VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CAPSTONE PROJECT

REPORT

aBridgeAI: An AI-Powered Career Readiness Learning Management System

---

Advisor(s):	Assoc. Prof. Vo Thi Ngoc Chau, PhD	
Student(s):	Le Thanh Duy	2352178
	Vo Hoang Ngan	2353373
	Nguyen Hoang Nam	2252502

HO CHI MINH CITY, OCTOBER 2026

Instructor's Signature

Approved by:

Signature	Date

Assoc. Prof. Vo Thi Ngoc Chau, Ph.D.
Associate Professor
Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology (HCMUT)

Declaration of Authenticity

We hereby certify that this project is our original work, conducted under the supervision of **Assoc. Prof. Vo Thi Ngoc Chau** at the Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (VNU-HCM):

- All external sources, data, and literature used in this report have been appropriately acknowledged and cited.

- We understand that any form of plagiarism will result in disciplinary action according to the university's regulations.

- The University shall not be held responsible for any copyright violations arising from this research.

Ho Chi Minh City, May 2026

Le Thanh Duy	**Vo Hoang Ngan**	**Nguyen Hoang Nam**
_____	_____	_____

Acknowledgement

This project would not have been possible without the support and contribution of many individuals.
First and foremost, we would like to express our deepest gratitude to our supervisor, **Assoc. Prof. Vo Thi Ngoc Chau**. Her profound expertise, insightful feedback, and unwavering support have been the cornerstone of this research. We are truly honored to have been guided by her intellectual mentorship throughout the development process.

We also extend our sincere appreciation to the faculty members of the Computer Science and Engineering department. Their constructive comments and keen insights into multimodal models have significantly refined our methodology and broadened our academic perspective.

Finally, a special thank you goes to our families and friends. Their constant encouragement and belief in our potential have been our greatest source of motivation and resilience, especially during the most challenging phases of this work.

Contents

1 Introduction **1**

1.1 Motivation 1
1.2 Goals 2
1.3 Scope 3
1.4 Significance of the Project 4
1.4.1 Scientific Perspectives 4
1.4.2 Practical Perspectives 5
1.5 Report Structure 6

2 Related Works **8**

2.1 The Bottlenecks of Traditional Spaced Repetition (SRS) Algorithms 8
2.2 Methodological Constraints in Retrieval Practice Applications 10
2.3 The Deficit in Contextual and Cognitive Adaptivity 12
2.4 The Path Toward aBridgeAI 16

3 Theoretical Background **17**

3.1 Cognitive Foundations of Learning 17
3.1.1 Cognitive Load Theory (CLT) 17
3.1.2 Handling Overly Complex Lessons 18
3.2 Algorithmic Learner Modeling and SR 19
3.2.1 The SM-2 Algorithm 20
3.2.2 Memory Models: Stability and Retrievability 22
3.2.3 Knowledge Tracing and Learner Modeling 24
3.3 Core Artificial Intelligence Technologies 28
3.3.1 LLMs in Education 28
3.3.2 Relational Reasoning: Overcoming Document Silos 29
3.3.3 Vector Databases and High-Dimensional Search 30
3.4 Technical Foundations and System Design Rationale 31
3.4.1 Component-Based Interface Design and Cognitive Load 31
3.4.2 Integration of the Adaptive Loop 31
3.5 Sumary about the Theoretical Background 32

4 System Requirements and Analysis **33**
4.1 System Overview . . . 33
4.1.1 Purpose . . . 33
4.1.2 Stakeholders and Users of the System . . . 33
4.2 Business Process . . . 35
4.3 Requirements . . . 39
4.3.1 User Requirements . . . 39
4.3.2 Functional Requirements . . . 42
4.3.3 Non-Functional Requirements . . . 43
4.3.4 Data Requirements . . . 43
4.4 System Modeling . . . 45
4.4.1 Overall Use Case Diagram . . . 45
4.4.2 Detailed Use Case Diagrams and Use Case Specifications . . . 46
4.4.3 Activity Diagrams . . . 53

5 System Design **55**
5.1 Technology Stack Selection . . . 55
5.1.1 Backend Framework . . . 55
5.1.2 Frontend Framework . . . 55
5.1.3 Real-Time Communication . . . 56
5.1.4 Primary Database & Vector Search . . . 56
5.1.5 Graph Database . . . 56
5.1.6 Caching & Task Queue . . . 57
5.1.7 Final Stack Summary . . . 58
5.2 System Architecture . . . 58
5.2.1 Actors and External Environment . . . 58
5.2.2 Internal System Components . . . 60
5.2.3 System Boundary Diagram . . . 61
5.2.4 Deployment Architecture . . . 62
5.3 Algorithm Design . . . 63
5.3.1 Career Pathway Progress and Completion . . . 63
5.4 API Design . . . 67
5.5 Class Diagrams . . . 68
5.6 Sequence Diagrams . . . 70
5.7 Database Design . . . 70

5.7.1 Entity-Relationship Diagram . . . 70
5.8 UI/UX Design . . . 73
5.8.1 Design Philosophy . . . 73
5.8.2 Design Tokens . . . 74
5.8.3 Key Screens . . . 75
5.8.4 Navigation Architecture . . . 78
5.9 Site Map . . . 78
5.10 Test Case Design . . . 81

6 Implementation and Testing **83**
6.1 Development Environment and Toolchain . . . 83
6.2 Project Structure . . . 84
6.3 Access Control . . . 86
6.4 Course and Curriculum . . . 88
6.5 Material Ingestion Pipeline . . . 92
6.6 AI Generation Pipelines . . . 94
6.6.1 Hybrid Retrieval . . . 94
6.6.2 Quiz Generation Pipeline . . . 95
6.6.3 Interview Question Generation . . . 96
6.6.4 LLM Gateway and Audit . . . 97
6.7 Adaptive Quizzing . . . 97
6.8 Mock Interview Sessions . . . 100
6.9 Spaced Repetition Implementation . . . 101
6.10 Career Pathways . . . 102
6.11 Programme and Path Governance . . . 106
6.12 Caching . . . 109
6.13 Platform Configuration, Policies and Audit . . . 111
6.14 Frontend Implementation . . . 112
6.15 Testing . . . 114
6.15.1 Test architecture . . . 114
6.15.2 Results . . . 116
6.15.3 Continuous integration and excluded suites . . . 118

7 Evaluation **120**
7.1 Frontend Evaluation . . . 120
7.2 Backend Performance . . . 121

7.2.1 Test Environment . . . 121
7.2.2 Test Scenarios . . . 122
7.2.3 Results . . . 122
7.2.4 Analysis against NFR Targets . . . 122
7.3 Requirements Scope . . . 125
7.4 AI Model Comparison for Quiz Generation . . . 125
7.4.1 Methodology . . . 125
7.4.2 Quality Results . . . 127
7.4.3 Latency and Cost . . . 127
7.4.4 Failure Analysis . . . 128
7.4.5 Recommendation . . . 128

8 Conclusion **130**
8.1 Achievements . . . 130
8.2 Limitations . . . 132
8.3 Future Work . . . 134
8.4 Concluding Remarks . . . 138

Appendices **144**

A Technology Selection Comparisons **144**
A.1 Backend Framework . . . 144
A.2 Frontend Framework . . . 145
A.3 Real-Time Communication . . . 146
A.4 Primary Database and Vector Search . . . 146
A.5 Graph Database . . . 147
A.6 Caching and Task Queue . . . 147

B Algorithm and Process Flow Specifications **148**
B.1 Hybrid SM-2 and Q Derivation . . . 148
B.2 Individualised Learner Modeling . . . 153
B.3 Student Performance Metrics . . . 155
B.4 Review Scheduling Flow . . . 158
B.5 Lesson Unlock Gate . . . 158
B.6 Material Ingestion . . . 159
B.7 Quiz Generation . . . 160

B.8 Quiz Attempt Lifecycle . . . 161
B.9 Career Pathway Stage Evaluation . . . 163
B.10 Path Change and Attempt Closure . . . 163

C Logical and Physical Database Design **165**
C.1 Logical Database Design . . . 165
C.2 Physical Database Design . . . 166

D Activity Diagrams Specification **167**
D.1 Authentication Flow . . . 167
D.2 Smart Content Ingestion Flow . . . 168
D.3 Human-in-the-Loop Quiz Generation Flow . . . 168
D.4 Native Mock Interview Flow . . . 169

E Sequence Diagrams Specification **171**
E.0.1 Authentication . . . 171
E.0.2 Smart Content Ingestion . . . 173
E.0.3 Human-in-the-Loop Quiz Generation . . . 173
E.0.4 Native Mock Interview Session . . . 174

F Class Diagram Specification **176**
F.1 Identity & Access . . . 176
F.1.1 Roles are assignments, not subclasses . . . 176
F.2 Course & Ingestion . . . 177
F.2.1 File bytes are not stored in the database . . . 177
F.3 Assessment & Interview . . . 179
F.4 Programme & Path Governance . . . 179
F.4.1 A held pathway is an attempt, not a foreign key . . . 179
F.4.2 Leaving a pathway is a reviewable record . . . 180

G Implementation Details **182**
G.1 Entity lifecycle reference . . . 182
G.2 Cross-cutting implementation notes . . . 186
G.3 The learner journey across subsystems . . . 187
G.4 Unlock evaluation in detail . . . 188
G.5 Work performed outside the request path . . . 188
G.6 Deployment topology . . . 189

G.7 Authentication and Identity 190
G.8 Notifications and Discussions 192
G.9 Test execution detail 193
G.9.1 Authentication and Access Control 193
G.9.2 Course and Material Management 194
G.9.3 Quiz Generation and Taking 194

List of Figures

3.1 Forgetting curves applied in SR learning. Source: [37]. 28
4.1 End-to-end business process by role: platform and organisation setup, programme and pathway governance, content authoring and assessment preparation, the student learning journey, and the career-readiness evidence it produces. 36
4.2 Overall Use Case Diagram of the aBridgeAI System 45
4.3 Detailed Use Case Diagram: Authentication and Access Control Module . . 46
4.4 Detailed Use Case Diagram: Organization and User Management Module . 47
4.5 Detailed Use Case Diagram: Course and Material Management Module . . 48
4.6 Detailed Use Case Diagram: Learning Loop and Adaptive Quiz Module . . 49
4.7 Detailed Use Case Diagram: Application Loop and Mock Interview Module 50
4.8 Detailed Use Case Diagram: Analytics, Reporting and Notifications Module 51
4.9 Activity Diagram: HITL Quiz Generation and Execution 53
4.10 Activity Diagram: Smart Content Ingestion and AI Parsing 54
5.1 aBridgeAI System Architecture Diagram 62
5.2 UML Class Diagram representing the core Domain Entities of aBridgeAI . 69
5.3 Overall Entity-Relationship Diagram 71
5.4 Entity-Relationship Diagram: Quiz, Attempts, Grading, and Spaced Repetition 72
5.5 Entity-Relationship Diagram: Interview Configuration, Sessions, Evaluation, Integrity, and Recording 72
5.6 Entity-Relationship Diagram: Remaining Cross-Domain Entities 73
5.7 Application sitemap — colour-coded by band: cool grey (public), warm grey (any signed-in user), blue (student), teal (instructor), violet (educational manager), amber (administrator). Dynamic URL segments are highlighted in amber as `$param`. Every node is one destination route; the 16 redirect-only aliases are omitted here and listed in Table F.8. 80
6.1 Top-level directory layout of the aBridgeAI backend. 85
6.2 Interview Question Generation Pipeline. 96
6.3 Activity Diagram: Mock Interview Main Handling Flow. 101
B.1 SM-2 review scheduling: from graded answer to persisted card state. 158
B.2 Lesson unlock gate: enrolment and visibility checks, the three unlock conditions, and the protected-content boundary. 159

B.3 Material ingestion: upload and scheduling, worker extraction and indexing, and teacher curation of the knowledge graph. 160
B.4 Quiz generation: synchronous validation and dispatch, the durable generation run, the topic and coverage pipelines, and the terminal poll. 161
B.5 Quiz attempt lifecycle: entry and resumption, attempt creation and freezing, the live answer loop with integrity monitoring, and submission. 162
B.6 Career pathway stage evaluation: from pinned version to per-stage completion, unlock, and overall progress. 163
B.7 Path change and attempt closure: request creation, the dean's review state machine, the approval transaction, and downstream entitlement reconciliation. 164
C.1 Logical Database Design 165
C.2 Physical Database Design 166
D.1 Activity Diagram: User Login with 2FA 167
D.2 Activity Diagram: Smart Content Ingestion and AI Parsing 168
D.3 Activity Diagram: HITL Quiz Generation and Execution 169
E.1 Sequence Diagram: Authentication 172
E.2 Sequence Diagram: Smart Content Ingestion 173
E.3 Sequence Diagram: HITL Quiz Generation and Execution 174

List of Tables

2.1 Comparative analysis of educational technologies across pedagogical dimensions. . . . 15
3.1 Quality response scale used in the SM-2 SR algorithm [5]. . . . 21
5.1 Selected Technology Stack — aBridgeAI 58
5.2 Core design tokens. . . . 74
5.3 Representative test coverage 82
6.1 Redis cache namespaces: key pattern, time-to-live, and what each entry saves. 110
6.2 Test layers, what each establishes, and what it requires to run. . . . 115
6.3 Executed test results. . . . 116
6.4 Backend tesing coverage by package. . . . 117
7.1 Lighthouse Audit Results (Lighthouse v13, Chrome 153, Production Deployment) 120
7.2 Peak-profile scaling matrix (3 s ramp, 3 min sustained per level; browse/quiz split 80/20) 123
7.3 Per-endpoint results at the NFR-2.1 target level (Config B/C, 50 VUs, 3 min, 23,473 requests, 0% errors) 123
7.4 Functional Requirements Scope: Specialized Project and Capstone Project 125
7.5 Quiz Generation Quality by Model (judge: `gpt-5-chat-latest`, reference-guided CoT) 127
7.6 Quiz Generation Latency and Cost by Model 128
A.1 Backend Framework Comparison 144
A.2 Frontend Framework Comparison 145
A.3 Real-Time Communication Comparison 146
A.4 Primary Database & Vector Search Comparison 146
A.5 Graph Database Comparison 147
A.6 Caching & Task Queue Comparison 147
B.1 Q value assignment rules based on correctness, hint usage, and time ratio ρ. Boundaries at $\rho = 1$ and $\rho = 2$ correspond to the teacher-defined expected response time and twice that value respectively; the boundary at $\rho = 0.5$ corresponds to half the expected time. . . . 150
B.2 Summary of student performance metrics and their roles within the aBridgeAI learner modeling layer. . . . 157
F.1 Identity & Access entities. . . . 177

F.2 Course & Ingestion entities. 178
F.3 Assessment & Interview entities. 179
F.4 Programme & Path Governance entities. 181
G.1 Entity lifecycle states enforced by database constraints. 183
G.2 Test Results: Authentication and Access Control 193
G.3 Test Results: Course and Material Management 194
G.4 Test Results: Quiz Generation and Taking 194

1 Introduction

In chapter 1, the motivation, goals, and scopes of the project are illustrated. The outline of the report is also presented.

1.1 Motivation

In the evolving landscape of higher education and vocational training, the demand for job-ready graduates is experiencing rapid growth, catering to diverse industry requirements. However, as the gap between academic theory and practical application widens, students often find themselves spending considerable effort struggling to transition from theoretical learning to actual career readiness. This transition is further complicated by a modern cognitive crisis described as the "age of interruption," where the constant barrage of digital stimuli from smartphones and social media leads to "continuous partial attention". In this state, learners perpetually shift their focus across multiple stimuli without fully immersing themselves in any single task, resulting in a superficial understanding of information. Research indicates that students in the presence of technological distractors can often only maintain sustained focus for a mere six minutes before being diverted. Consequently, the inefficiency of conventional Learning Management Systems (LMS) is not merely a structural issue but a psychological one; "attentional overload" occurs when environmental demands exceed an individual's finite attentional capacity, hindering the deep processing required for career-critical skills[1].

By introducing an AI-powered Career Readiness Learning Management System, we aim to revolutionize the way educational institutions prepare students for the workforce. A core objective of aBridgeAI is to mitigate the risk of "AI as Autopilot," a failure mode where generative tools provide complete solutions that reduce "germane load"—the productive mental work needed to build mastery—and thus prevent actual learning. Instead of doing the thinking for the student, aBridgeAI utilizes Large Language Models (LLMs) to provide "fading support" scaffolding, which transitions learners from fully solved worked examples to independent practice as their competence grows. The system is designed to force "System-2 level engagement" by prompting students to explain their reasoning in steps, identify underlying assumptions, or solve problems using alternative methods. To further respect the constraints of human working memory, the platform employs "segmenting," breaking high-intrinsic-load curriculum into bite-size, learner-controlled segments. This ensures that the learner has sufficient time to organize and integrate information from

one portion of the lesson before the next segment begins, preventing cognitive overload during the acquisition of complex technical skills[2, 3].

Moreover, the development of such a system addresses the pressing need for automation and scalable assessment in the educational sector while actively countering "digital dementia." This term refers to the cognitive decline and memory loss caused by overreliance on digital devices to store information that would otherwise be held in long-term memory. aBridgeAI combats this by integrating "active recall" and hybrid spacing algorithms that identify the optimal intervals for review just before information is likely to be forgotten. As students gain expertise, the system dynamically fades guidance to avoid the "expertise reversal effect," where redundant instructional support becomes an "extraneous load" that interferes with the schemas already constructed by the learner. This automated and context-aware approach allows educators to track authentic progress and provide personalized remediation, ensuring that mastery is internalized rather than offloaded to a tool. Ultimately, aBridgeAI answers the critical need to guide learners from being merely "Book Smart" to becoming "Job-Ready" candidates whose professional competencies are automated and resilient[4, 5].

1.2 Goals

The primary goal of this project is to research, design, implement and evaluate aBridgeAI, an intelligent LMS that bridges the gap between theoretical knowledge acquisition and practical career readiness. To achieve this aim, the project focuses on the following objectives:

- **Developing a Smart Content Ingestion and Retrieval Pipeline:** To engineer an automated system capable of processing multimodal educational materials. The system will transcribe, extract, and index these materials into a vector database for subsequent retrieval tasks.
- **Developing an Adaptive Theoretical Assessment Module:** To build an AI-driven quiz generation engine that dynamically creates context-specific questions based strictly on the ingested curriculum. This goal includes designing an adaptive feedback loop that directs struggling students back to precise timestamps or document sections for targeted remediation.
- **Providing Students with a Context-Aware Mock Interview System:** To build a conversational AI interviewer utilizing LLMs and Retrieval-Augmented Gen-

eration (RAG). The core innovation lies in dynamically generating technical and behavioral interview scenarios strictly grounded in both the teacher's specific lecture content and the student's personal resume, ensuring an authentic and highly relevant practical assessment.

- **Constructing a Comprehensive Analytics and Gap-Reporting Engine:** To develop an automated evaluation rubric that assesses technical accuracy, communication clarity, and problem-solving skills. The system will synthesize data from both the learning loop (quizzes) and the application loop (interviews) to generate a personalized "Gap Report," highlighting discrepancies between theoretical knowledge and practical articulation.

1.3 Scope

To ensure the feasibility of the aBridgeAI, the scope is constrained across five dimensions:

- **Technological Scope:** The system is developed as a responsive web application to ensure cross-platform compatibility. The stack encompasses React for the frontend, FastAPI (Python) for backend and AI control-plane orchestration, PostgreSQL with pgvector for transactional and vector data, Neo4j for the course knowledge graph, Redis for caching and background jobs, Garage for S3-compatible object storage, and LiveKit for WebRTC interview rooms. The target deployment runs a separate native LiveKit worker for conversational interviews; Google OAuth 2.0 / OpenID Connect provides delegated authentication with in-platform second-factor (TOTP) enrolment and session management.

- **Temporal Scope:** The development, testing, and deployment lifecycle of this Minimum Viable Product (MVP) is strictly bounded within Semester 252, commencing in early February and concluding in June.

- **Spatial and Contextual Scope:** The system's operational context is designed specifically for academic and structured learning environments. It targets higher education institutions, university departments, and coding bootcamps, focusing exclusively on bridging the gap between theoretical academic curricula and practical industry interview requirements.

- **Functional Scope:** The system's core functionalities are limited to four main interactive pillars:

1. *Smart Content Ingestion:* Parsing uploaded documents (PDF, DOCX) and media (MP4) to construct a localized Course Knowledge Graph.
2. *The Learning Loop:* Providing automated, adaptive quizzes based on the ingested content.
3. *The Application Loop:* Executing context-aware mock interviews supporting text-based and asynchronous/WebRTC voice-based interactions.
4. *Assessment Analytics:* Generating automated Gap Reports and tracking student performance matrices.

The following capabilities are explicit non-goals of the MVP and are excluded from this phase: synchronous human-to-human tutoring and live video conferencing; in-platform email/password registration, password reset, "remember me" device-trust, and per-device session pinning. Authentication is delegated exclusively to Google OAuth, and self-service password recovery is therefore handled by Google's identity infrastructure rather than in-platform.

- **Data Scope:** The system is scoped to process, store, and manage standard educational materials, user profiles, and assessment logs. For AI operations, the system limits retrieval data to vector embeddings (via pgvector) derived from institution-uploaded materials. Live interview audio is normally processed transiently for Speech-to-Text (STT) and Text-to-Speech (TTS). Optional audio recording is separately deployment-enabled, consent-gated by policy version, and retained through its own object-storage lifecycle; recording failure does not alter assessment outcome.

1.4 Significance of the Project

1.4.1 Scientific Perspectives

From a scientific standpoint, aBridgeAI makes several contributions to the intersection of educational technology, cognitive science, and applied machine learning.

The primary scientific contribution is the formalisation of an algorithmically derived quality response value for the SM-2 spaced repetition algorithm in institutional deployment contexts. Standard SM-2 relies on learner self-assessment as its sole input signal, an assumption that holds in solo study environments but breaks down when the quality score influences externally visible progression gates. This work derives Q from three objective,

system-observable signals — response correctness, hint usage, and a dimensionless time ratio $\rho = T_{\text{actual}}/T_{\text{exp}}$ — replacing a subjective input with a deterministic computation grounded in response latency theory [6]. The threshold boundaries in the Q assignment rule are derived as natural multiples of the teacher-defined expected response time rather than as independently calibrated constants, eliminating arbitrary parameterisation from the scheduling model.

A secondary contribution is the introduction of a calibration weight α applied asymmetrically to early EF increments, addressing the known cold-start overestimation problem in SM-2 initialisation. By dampening positive EF drift during the first three repetitions while applying negative increments at full magnitude, the model converges onto a student's true retention profile faster than the standard initialisation assumption permits. This mechanism is formalised within the SM-2 framework without requiring the full parametric complexity of successor algorithms such as FSRS [7].

At the assessment layer, the work contributes a teacher-defined outcome architecture for AI-conducted mock interviews that decouples approved question-bank selection, live progression, and terminal evaluation. Rather than scoring open-ended responses against a model answer key, the system uses teacher-specified outcomes, server-controlled state, and a durable transcript. The target native runtime may use an in-session LLM sufficiency probe for provisional progression, while asynchronous terminal evaluation independently determines the recorded outcome verdict. This separation preserves open-ended interaction without giving the conversational model sole authority over assessment.

Finally, the system introduces three formally defined learner-facing metrics — the Knowledge Retention Estimate $\hat{R}_{s,\ell}$, Progression Readiness, and Review Compliance Rate $\rho_{s,\ell}$ — that together characterise a student's cognitive standing within a lesson from complementary diagnostic, gate, and engagement perspectives. The formal definitions of these metrics, their derivation from per-student SM-2 state, and their relationship to the no-decay design decision constitute a contribution to the literature on competency-based progression systems in spaced repetition contexts.

1.4.2 Practical Perspectives

From a practical standpoint, aBridgeAI addresses a concrete and growing problem in institutional education: the inability of existing tools to bridge the gap between passive knowledge consumption and verified competency demonstration at scale.

For **teachers and instructors**, the platform eliminates the manual overhead of quiz creation by automating question generation from uploaded course materials, while preserv-

ing full editorial control through a mandatory review and configuration phase. Teachers can define expected response times and EF thresholds per card, configure lesson unlock criteria, and specify interview outcome requirements — all without requiring knowledge of the underlying scheduling algorithm. The monitoring dashboard surfaces per-student and cohort-level retention metrics, enabling data-informed intervention rather than reliance on periodic examination snapshots.

For **students**, the platform replaces fixed assessment schedules with individually paced review cycles governed by demonstrated memory stability. Students are notified when cards are due for review at algorithmically optimal intervals, reducing the cognitive burden of self-scheduling while reinforcing retention through spaced practice. The interview module provides repeated access to structured practice sessions in a low-stakes environment, building the practical communication and problem-solving skills that quiz-based assessment cannot evaluate. Career Readiness reporting gives students a transparent, continuously updated signal of their progression toward professional competency targets.

For **organisations and educational managers**, the system provides institution-level visibility into cohort progression through career-scoped reporting. Career Readiness scores aggregated across course completion percentages give managers a quantitative basis for identifying at-risk cohorts, evaluating curriculum effectiveness, and demonstrating learning outcomes to accreditation bodies or industry partners — without requiring manual data collection or post-hoc grade analysis.

From a broader industry perspective, aBridgeAI demonstrates that the cost and complexity barriers associated with AI-assisted personalised learning can be constrained architecturally: question banks are pre-generated and teacher-approved, real-time interview conversation is bounded by server-controlled state and tools, richer reconciliation and final evaluation are asynchronous, and scheduling is grounded in a deterministic algorithm. This separation limits live-model responsibility while retaining personalised interaction.

1.5 Report Structure

This report is organized as follows:

- **Chapter 1 – Introduction:** Introduces the motivation behind aBridgeAI, identifying the limitations of traditional educational systems in delivering personalised, competency-driven learning at institutional scale. Defines the project goals, delimits the system scope, and articulates the academic and practical significance of the proposed solution.

- **Chapter 2 – Related Works:** Reviews existing educational technologies and mock interview methodologies, identifying their current limitations and establishing the theoretical need for an integrated, context-aware solution.
- **Chapter 3 – Theoretical Background:** Establishes the theoretical foundation for the core technologies employed in this project, including LLMs, RAG pipelines, Vector Databases, and modern web development frameworks.
- **Chapter 4 – System Analysis and Requirements:** Details the systematic extraction of software requirements. It encompasses the definition of user roles, comprehensive Functional Requirements (FRs), quantifiable Non-Functional Requirements (NFRs), and Unified Modeling Language (UML) artifacts such as Use Case and Activity diagrams.
- **Chapter 5 – System Design:** Presents the high-level architectural blueprint of the system. It covers the system topology, PostgreSQL with pgvector, Neo4j, Redis, Garage object storage, LiveKit interview delivery, and User Interface/User Experience (UI/UX) wireframes.
- **Chapter 6 – Implementation and Testing:** Discusses the practical realization of the proposed pipeline, including the native LiveKit interview runtime and its automated-test strategy.
- **Chapter 7 – Evaluation:** Presents the measured frontend, backend, and quiz-generation evaluation evidence, together with FRS inventory and test-case traceability.
- **Chapter 8 – Conclusion:** Summarizes the technical contributions and educational impacts of the project, acknowledges current system limitations, and outlines strategic directions for future enhancements.

2 Related Works

The intersection of cognitive science and artificial intelligence has birthed powerful tools for learning and knowledge management. However, current implementations—ranging from SRS algorithms to RAG suffer from systemic fragmentation. This essay explores the critical limitations within these domains, establishing the theoretical necessity for a context-aware, integrated knowledge system like aBridgeAI.

2.1 The Bottlenecks of Traditional Spaced Repetition (SRS) Algorithms

While SRS is a cornerstone of evidence-based learning, traditional algorithms fail to account for the complexity of human cognition. Recent work such as LECTOR [8] highlights the need for concept-aware and test-oriented adaptive scheduling mechanisms.

Semantic Isolation and Interference.

Most popular algorithms, including SM-2, HLR, and even the more recent FSRS (Free Spaced Repetition Scheduler), treat flashcards as independent data points. This "atomistic" approach ignores the semantic relationships between concepts. In practice, this leads to semantic interference, where similar concepts (e.g., nuanced vocabulary in standardized testing) become muddled because the system fails to highlight their distinctions or shared contexts [8].

These theoretical limitations are directly observable in today's most widely adopted educational platforms. For instance, Anki, a standard utility for language acquisition and medical studies, relies heavily on SM-2 and FSRS to evaluate cards in strict isolation. When a user studies closely related terms, the system schedules them independently, failing to detect their semantic proximity to present them together for necessary contrastive learning. Similarly, Duolingo's foundational Half-Life Regression (HLR) models the decay rate of individual grammatical structures in a vacuum. If a learner struggles to distinguish between two similar concepts, the model merely decreases their respective half-lives, prompting more frequent, isolated reviews rather than recognizing the relationship itself as the cognitive bottleneck. Commercial platforms like Quizlet further exacerbate this issue by prioritizing rapid, item-by-item testing that strips concepts of their broader contextual frameworks[9].

Consequently, learners often experience retrieval failure when confronted with real-

world applications that require distinguishing between closely related ideas. When acquiring new language structures or technical definitions, studying synonymous terms on entirely separate review schedules prevents the brain from forming necessary contrastive distinctions. The learner might remember the general meaning but fail to grasp the specific usage constraints. To mitigate this, next-generation schedulers must incorporate structural architectures that map semantic proximities. By actively identifying clusters of related information, an advanced algorithm could intelligently group these items for side-by-side contrastive review, or strategically distance them to prevent memory overwriting. This shift would transform a fragmented, item-by-item database into a cohesive, associative web of knowledge that mirrors human cognition.

The Legacy of Limited Data.

The SM-2 algorithm, the default for platforms like Anki, was developed in 1987 based on a very narrow experimental dataset. Its reliance on fixed mathematical formulas makes it brittle" and incapable of adapting to the non-linear learning curves of modern learners. While machine learning variants exist, they often lack the granularity" required for complex conceptual mapping. Consequently, users often find themselves trapped in "review hell", where the algorithm stubbornly schedules repetitions based on superficial recall rather than deep comprehension. A foreign language vocabulary word, a historical timeline, and a complex syntax rule are all subjected to the exact same rigid decay functions[10].

In practice, this rigidity is acutely felt by users who attempt to manage diverse, multidisciplinary knowledge bases. For example, a software developer preparing for technical interviews might use standard SRS to memorize both simple programming syntax and complex, multi-layered frameworks like WCAG accessibility standards. Because the legacy algorithm lacks structural granularity, it cannot distinguish between the cognitive effort required for a simple rote fact versus a heavily interdependent concept. It simply subjects both to the same mathematical decay, frequently burying the user under an overwhelming backlog of due cards and forcing brute-force memorization over true understanding.

To move beyond these archaic constraints, next-generation SRS must evolve from isolated item schedulers into context-aware engines. By leveraging deep neural networks trained on vast, diverse sets of user interaction data, future algorithms could dynamically adjust intervals based on the intrinsic semantic difficulty of the material. Ultimately, a truly modern system would recognize the interconnected nature of knowledge, mapping how the mastery of one concept accelerates the retention of another, thereby replacing a

one-size-fits-all mathematical formula with a highly personalized cognitive model.

The Personalization Gap.

Traditional systems focus almost exclusively on temporal scheduling (when to review) rather than cognitive profiling (how to review). They do not dynamically adjust based on a learner's specific preparation scenarios, such as the high-pressure environment of a professional certification or the nuanced needs of language acquisition. For instance, the cognitive load required to distinguish visually similar logograms—such as complex Kanji characters—differs drastically from the effort needed to master basic phonetic scripts or standard vocabulary. Yet, legacy algorithms treat these distinct cognitive tasks uniformly. Furthermore, a robust cognitive profile should account for the learner's real-time state, including mental fatigue and specific error patterns, rather than relying solely on blunt, self-reported pass/fail metrics.

This gap is immediately evident in the user interfaces of major SRS platforms, which rely entirely on subjective feedback buttons (e.g., "Anki's Again, Hard, Good, Easy"). If a learner studying for a standardized language exam, such as the JLPT, repeatedly confuses visually similar Kanji, clicking "Again" merely resets the temporal interval. The system cannot diagnose the underlying nature of the error—whether it stems from semantic interference, a fundamental misunderstanding of the character's radical, or simply cognitive exhaustion. Furthermore, current platforms fail to adapt to the learner's physical and digital environment; a user squeezing in quick reviews on a mobile device receives the exact same cognitively demanding prompts as a user engaged in deep, focused study on a desktop machine.

By measuring multidimensional variables like response latency and contextual interference, a truly adaptive system could pivot its strategy on the fly. It might, for example, shift from demanding active recall to providing targeted recognition tasks when cognitive exhaustion is detected, transforming a rigid review schedule into an intuitive, highly personalized learning environment.

2.2 Methodological Constraints in Retrieval Practice Applications

Retrieval practice—the act of recalling information—is one of the most robust findings in educational psychology. Yet, its application in software remains superficial.

Confounded Experimental Control.

Many educational applications claim to leverage the "testing effect", but their implementations are often methodologically noisy. It is frequently unclear whether a learner's progress stems from the act of retrieval itself or from the feedback provided afterward. Without isolating these variables, systems cannot optimize the specific level of "cognitive effort" required for durable long-term retention. Furthermore, when platforms intertwine testing with immediate, high-information feedback, the assessment essentially transforms into an additional study trial. This conflation masks whether a memory trace was strengthened by the active, internal generation of the answer or merely refreshed by the external re-exposure to the correct information. Consequently, the concept of "desirable difficulties"—the foundational principle that learning should be suitably challenging to promote deep processing—is often compromised.

This methodological conflation extends far beyond mobile flashcard apps, deeply permeating both enterprise and academic LMS. In corporate or organizational training environments powered by platforms like Docebo, TalentLMS, 360Learning, and Absorb LMS, assessments are frequently designed for rapid compliance rather than long-term retention. Quizzes on these platforms often provide instant remediation; if a user fails a question, the system immediately highlights the correct protocol, effectively turning a test of recall into a passive reading exercise.

Similarly, massive open online course (MOOC) providers like Coursera and Khan Academy, alongside foundational open-source architectures like Moodle, heavily utilize immediate, high-information feedback to maintain student momentum. For instance, when a learner is tested on complex technical concepts—such as implementing WCAG accessibility standards—these platforms frequently offer progressive hints, automated corrections, or infinite module retakes with explicit error breakdowns. While platform creators like Gurucan incentivize this to scale course completion rates, this excessive scaffolding artificially inflates short-term performance. It inherently reduces the productive struggle necessary to forge durable neural pathways. To genuinely harness the testing effect, future iterations of educational technology must employ rigorous, factorial experimental designs that decouple retrieval difficulty from feedback mechanisms, thereby allowing algorithms to dynamically calibrate to a learner's optimal zone of cognitive friction [11].

The Monotony of Format.

There is a heavy over-reliance on Multiple-Choice Questions (MCQs) due to their ease of automation and objective grading scalability. However, research suggests that MCQs

are often significantly less effective than Short-Answer Questions (SAQs) or production-based tasks for cultivating higher-level conceptual mastery at the university level. Because MCQs inherently present the correct answer alongside distractors, they primarily stimulate recognition memory rather than the more cognitively demanding process of active, free recall. This paradigm trains learners to identify plausible answers instead of synthesizing and articulating knowledge independently, which frequently results in brittle, inert knowledge that fails to transfer to complex, real-world applications.

This structural bottleneck is ubiquitous across modern educational infrastructure. Global university LMS, such as Canvas and Blackboard, alongside massive MOOC platforms like edX and Udemy, rely almost exclusively on auto-graded MCQs to handle large student volumes. When these platforms do attempt to automate SAQs or fill-in-the-blank exercises, their legacy assessment engines typically rely on rigid regular expressions (RegEx) or exact string matching. Consequently, a student who deeply understands a concept—such as explaining the core loop of an algorithm or the nuance of a Japanese grammar rule—but phrases it uniquely is often penalized. This forces instructors and course designers to default back to the safety and scalability of MCQs.

Furthermore, current educational systems largely lack the generative and evaluative sophistication necessary to seamlessly produce diverse assessment formats capable of probing deep structural understanding. While modern artificial intelligence has made strides in natural language processing, reliably evaluating open-ended, production-based tasks requires nuanced semantic parsing to distinguish between genuine conceptual comprehension and mere keyword mimicry. Until automated assessment platforms can robustly evaluate these complex, constructed responses with human-like accuracy, the pedagogical landscape will remain bottlenecked by formats that prioritize superficial recognition over genuine cognitive generation.

2.3 The Deficit in Contextual and Cognitive Adaptivity

A significant bottleneck in current educational and automation frameworks is their inability to dynamically modulate the "pace" and "intensity" of interaction based on the user's fluctuating cognitive state. This lack of adaptivity manifests across several dimensions.

Static Control Architectures vs. High-Entropy Environments.

Most conventional learning systems are built upon implicitly Linear-Time-Invariant (LTI) operating assumptions. They presuppose a predictable, incremental trajectory

of knowledge acquisition. However, human learning is inherently non-linear and high-entropy: learners experience moments of rapid conceptual integration followed by extended consolidation plateaus. Because existing systems cannot dynamically pivot their instructional strategy in response to these shifting probabilistic states, they remain rigid—failing to capitalize on periods of high receptivity or provide calibrated intervention during unexpected cognitive stalls. To truly model the stochastic nature of human cognition, these architectures must transition away from deterministic, rule-based sequencing toward probabilistic control frameworks, such as Partially Observable Markov Decision Processes (POMDPs) or deep Bayesian Knowledge Tracing. In a high-entropy cognitive state—characterized by high working memory load or conceptual confusion—a rigid system blindly pushes the next scheduled module, often exacerbating cognitive overload and precipitating algorithmic fatigue. Conversely, during periods of low entropy and rapid assimilation, the system fails to accelerate the pacing or increase the structural complexity of the tasks, leading to boredom and disengagement. Achieving genuine adaptivity requires continuous, real-time telemetry—analyzing micro-interactions, response latencies, and specific error typologies—to accurately infer the user's hidden cognitive state. Only by closing this complex, multi-dimensional feedback loop can an educational platform dynamically modulate the "bandwidth" of information delivery, evolving from a static content repository into a highly responsive cognitive symbiote.

Failure to Maintain the "Goldilocks Zone" of Flow.

Effective knowledge construction requires maintaining the learner within the "Goldilocks zone"—a state of optimal challenge formally associated with Lev Vygotsky's Zone of Proximal Development and Mihaly Csikszentmihalyi's psychological theory of Flow. Current educational tools and automated tutors typically rely on fixed, deterministic review schedules or simplistic, performance-based branching mechanisms that only measure binary outcomes, such as whether a final answer is correct or incorrect. Without a more granular, continuous understanding of the user's internal affective state—derived from subtler telemetry like response latency, error self-correction rates, or dwell time—these platforms frequently drift into two counterproductive extremes:

- **Cognitive Overload (Anxiety):** Delivering complex material or requiring high-dimensional reasoning too rapidly. When systemic demands significantly exceed the learner's current schema capacity, it triggers working memory saturation and high affective resistance, ultimately resulting in frustration, algorithmic fatigue, and system abandonment.

- **Cognitive Underload (Complacency):** Providing overly simplistic tasks, excessive scaffolding, or highly predictable generative outputs that fail to stimulate semantic processing. Prolonged exposure to this state not only induces boredom and reduces immediate engagement, but also precipitates severe automation bias; the learner becomes passively dependent on the system's external cognitive labor, gradually eroding their internal critical reasoning and independent problem-solving faculties.

To sustain this delicate equilibrium, next-generation architectures must integrate dynamic difficulty adjustment (DDA) algorithms capable of micro-modulating pedagogical friction. By dynamically increasing abstraction when rapid mastery is detected and providing targeted, localized interventions during genuine struggle, these systems can actively engineer an enduring, personalized flow channel.

Synthesis of Limitations

These collective limitations—ranging from semantic isolation in SR and retrieval interference in rigid assessment formats, to structural fragmentation in RAG systems and the absence of dynamic cognitive feedback—reveal a deeply segmented technological landscape. Current frameworks treat knowledge as a collection of discrete artifacts rather than a living, interdependent cognitive network. This fragmentation underscores the necessity for an integrated, context-aware, and cognitively aligned architecture such as *aBridgeAI*, which aims to unify adaptive scheduling, relational reasoning, and real-time cognitive modulation into a cohesive intelligence partner.

Table 2.1: Comparative analysis of educational technologies across pedagogical dimensions.

Feature / Pedagogical Dimension	**Traditional SRS (Anki, Quizlet)** [12, 13]	**Enterprise / Academic LMS (Canvas, Moodle)** [14, 15]	**Gamified EdTech (Duolingo)** [16]	**aBridgeAI**
Knowledge Retrieval	Atomistic / Isolated flashcards	Modular / Static document silos	Linear / Skill-tree based	Semantic retrieval over structured learning materials (vector-indexed knowledge units)
Assessment Format	Binary recall (pass / fail)	MCQs, RegEx short answers	Drag-and-drop, translation	Algorithmically graded quiz with hint penalty; outcome-verified mock interviews
Adaptive Scheduling	Fixed multiplicative interval scheduling (SM-2, FSRS)	None (instructor scheduled)	HLR	Hybrid SM-2 with algorithmically derived Q via response latency normalisation and per-student EF isolation
Content Creation	Manual user generation	Manual instructor creation	Studio-produced content	Instructor-uploaded materials with AI-assisted breakdown; full manual creation and editorial override supported
Feedback Mechanism	Instant answer exposure	Instant remediation / scaffolding	Immediate error correction	Hint-penalised Q scoring discouraging passive retrieval; binary interview outcome without rubric exposure
Support applying knowledge and skills in practice	None (pure theory)	Limited (sandbox coding)	None (standardised translation)	Teacher-defined outcome verification via context-aware technical and behavioural mock interviews

2.4 The Path Toward aBridgeAI

The limitations outlined above—semantic isolation in SRS, insufficient experimental control in retrieval practice implementations, relational blindness in RAG, and the maintenance fragility of large-scale Knowledge Graph Construction (KGC)—converge toward a singular conclusion. Contemporary systems continue to treat knowledge as a collection of isolated data points rather than as a dynamic, interdependent cognitive ecosystem.

By integrating context-aware algorithms with rigorous dependency management and cognitively grounded retrieval strategies, *aBridgeAI* directly addresses these structural deficiencies. The system advances beyond simple recall optimization toward deep structural understanding, offering a unified and adaptive framework capable of supporting both individual learners and enterprise-scale knowledge infrastructures.

3 Theoretical Background

To address the structural deficiencies and semantic isolation of traditional educational systems identified in Chapter 2, aBridgeAI relies on a rigorous synthesis of cognitive psychology, algorithmic modeling, and advanced artificial intelligence. This chapter establishes the theoretical and technological foundations that empower the system to treat knowledge as a dynamic, interdependent cognitive ecosystem.

First, the chapter explores the baseline constraints of human memory through CLT, establishing the pedagogical necessity for adaptive knowledge structuring. These psychological principles are then translated into computational mechanics through the design of a hybrid SM-2 SR algorithm and individualized learner evaluation metrics. Finally, the chapter examines the sophisticated technological infrastructure required to execute these models at scale—specifically LLMs, RAG pipelines, Vector Databases, and modern web development frameworks. Together, these theoretical pillars form the architectural blueprint for aBridgeAI's context-aware learning environment.

3.1 Cognitive Foundations of Learning

3.1.1 Cognitive Load Theory (CLT)

Human cognitive architecture is characterized by a working memory that has severe limitations in capacity and duration when processing novel information. Working memory typically holds only about seven chunks of information at a time and is restricted in its ability to simultaneously process and organize interacting elements. All information must first pass through this limited working memory before it can be stored in the unlimited long-term memory as permanent schemas. When instructional materials exceed these processing constraints, cognitive overload occurs, leading the learner to lose focus and stop processing the content deeply. Consequently, total cognitive load—the sum of intrinsic, extraneous, and germane loads—must be managed to ensure that the learner's cognitive capacity is not overwhelmed[17].

Intrinsic Load

Intrinsic cognitive load refers to the inherent complexity of the material being learned, which is determined by its "element interactivity". This load is generated by the number of elements that a learner must attend to simultaneously to understand the subject matter. Because experts can use previously stored schemas in long-term memory to pro-

cess multiple interacting squiggles or concepts as a single chunk, intrinsic load is always relative to the learner's level of prior knowledge. It cannot be reduced by instructional design unless the learning objective itself is simplified or broken down. Therefore, a task that is high-load for a novice may impose minimal load on an expert due to their ability to activate existing knowledge structures in working memory[18].

Extraneous Load

Extraneous cognitive load is unnecessary mental strain caused by the way information is presented or the activities a learner is required to perform. Poor design choices, such as forcing learners to mentally integrate spatially separated text and diagrams (the split-attention effect), increase this bad form of load. Extraneous load consumes working memory resources that could otherwise be used for meaningful learning, thereby hindering mastery and schema construction. Instructional interventions aim to minimize this load by clarifying wording, simplifying navigation, and avoiding redundant information. Effective system design ensures that the cognitive capacity is focused on the essential content rather than on managing a messy user interface or unclear instructions[2].

Germane Load

Germane cognitive load refers to the productive mental effort that learners expend to construct schemas and build a permanent store of knowledge. Modern conceptualizations view germane load as a redistributive function that moves working memory resources away from extraneous activities and toward the intrinsic aspects of the learning task. Well-structured instructional methods increase germane load by prompting learners to engage in reflection, retrieval practice, and self-explanation. This type of load is essential for deep understanding, as it represents the actual mental work required to automate skills and integrate new information with prior knowledge. By minimizing extraneous distractions, systems can maximize the cognitive resources available for this critical germane processing[19].

3.1.2 Handling Overly Complex Lessons

To mitigate cognitive overload, the system employs dynamic chunking and segmenting to break high-intrinsic-load material into digestible, bite-size nodes. This "segmenting effect" ensures that learners have sufficient time and capacity to organize and integrate information from one portion of the lesson before the next segment begins. By breaking a continuous unit into learner-controlled segments, the system prevents the "zoning out"

that happens when novel information is presented at a pace too fast for deep processing. This approach essentially creates interconnected nodes of knowledge, allowing the working memory to focus on building component models before attempting to understand the entire complex causal system[3].

The system also utilizes isolated elements sequencing, where individual interacting components are presented sequentially before being integrated into a fully complex form. This strategy is particularly effective for very high-element interactivity tasks that would otherwise exceed working memory limits if all interactions were taught at once. Once these isolated elements are stored in long-term memory, the learner only needs to learn the interactions between them, which significantly reduces the working memory load during the final integration phase. By managing the order of presentation, the system ensures that the intrinsic complexity of the subject matter is introduced at a rate that aligns with the recovery and limits of working memory resources[17].

Finally, aBridgeAI provides "fading support" or scaffolding to transition learners from studying worked examples to independent practice. Initially, the system provides fully solved examples to reduce extraneous load, as solving a conventional problem requires high-load strategies like means-ends analysis which leave little capacity for learning. As the learner's expertise increases, the system provides partially worked examples and completion tasks, eventually requiring independent problem solving. This adaptive approach avoids the "expertise reversal effect," where redundant guidance becomes an extraneous load for learners who have already acquired sufficient domain knowledge[4].

3.2 Algorithmic Learner Modeling and SR

SR is a memory consolidation technique grounded in the psychological phenomenon of the spacing effect, which holds that information reviewed across distributed intervals is retained more durably than information reviewed in massed practice [20, 21]. Algorithmic implementations of SR automate the scheduling of these intervals by maintaining a model of each learner's memory state and adjusting review frequency accordingly. This subsection surveys the foundational algorithm underpinning aBridgeAI's learner modeling layer, its theoretical memory model, and its known limitations in institutional deployment contexts.

3.2.1 The SM-2 Algorithm

The SuperMemo-2 (SM-2) algorithm, introduced by Wozniak in 1987 and subsequently refined [5], is among the most widely adopted SR scheduling algorithms in educational software. It introduced item-level difficulty differentiation through a scalar parameter termed the Easiness Factor (EF), which governs the rate at which review intervals grow for each knowledge item independently. This allowed the algorithm to adapt review schedules to the relative difficulty of individual items rather than applying a uniform spacing function across all material.

Interval Scheduling

SM-2 schedules reviews through a recursive interval function. The first two intervals are fixed:

$$I(1) = 1 \text{ day}, \qquad I(2) = 6 \text{ days}. \tag{3.1}$$

All subsequent intervals are computed as:

$$I(n) = I(n-1) \cdot EF, \quad n > 2, \tag{3.2}$$

where EF is initialised at $EF_0 = 2.5$ for all items and bounded below by $EF_{\text{min}} = 1.3$ to prevent pathologically short intervals on items of high difficulty. The interval therefore grows multiplicatively with each successful review, reflecting the empirical observation that successfully recalled items can tolerate progressively longer delays before the next review without meaningful loss of retention [5].

Easiness Factor Update

After each review, EF is updated according to the learner's reported quality response value $q \in \{0, 1, 2, 3, 4, 5\}$:

$$EF_{\text{new}} = EF_{\text{old}} + (0.1 - (5 - q)\,(0.08 + (5 - q) \cdot 0.02))\,. \tag{3.3}$$

This update rule penalises EF for low q values and rewards it for high q values, with the neutral point at $q = 4$ producing approximately zero net change. Items recalled with difficulty accumulate lower EF values and are therefore scheduled more frequently; items recalled effortlessly accumulate higher EF values and are scheduled less frequently. The EF floor of 1.3 ensures that even the most difficult items eventually stabilise at a reviewable cadence rather than collapsing into daily repetition indefinitely.

Failure and Reset

If $q < 3$, the item is considered failed. The interval resets to $I = 1$ day, the repetition counter returns to zero, and the item re-enters the learning queue regardless of its prior history. The EF penalty is nonetheless applied, ensuring that repeated failures progressively reduce the item's interval growth rate even as the absolute interval resets.

Quality Response Scale

The quality response value q is the sole input the algorithm receives from the learner per review. In the original SM-2 specification, q is assigned entirely through learner self-assessment after the correct answer has been revealed. The semantic definitions of each value are summarised in Table 3.1.

Table 3.1: Quality response scale used in the SM-2 SR algorithm [5].

q	**Response Quality Description**
5	Perfect response; the correct answer is recalled immediately and without effort.
4	Correct response after a brief hesitation.
3	Correct response recalled with noticeable difficulty.
2	Incorrect response; the correct answer feels familiar upon seeing it.
1	Incorrect response; the correct answer is only partially recognised after review.
0	Complete blackout; no recognition of the correct answer whatsoever.

Known Limitations

Although SM-2 has demonstrated practical effectiveness across decades of deployment in self-study tools [22], several structural limitations bear directly on its applicability in institutional learning environments.

First, the algorithm treats q as a honest self-assessment, assuming the learner has no incentive to misreport. This assumption is well-founded in solo study contexts but breaks down when q accumulates into metrics that influence externally visible outcomes such as lesson progression or instructor-facing dashboards.

Second, interval growth is governed solely by EF, which encodes only the historical pattern of q values for a given item. The algorithm applies no model of item-level response demand — a brief factual question and a complex multi-step problem receive identical scheduling treatment once their EF values converge, despite the substantially different cognitive effort required to answer each.

Third, SM-2 does not model the passage of time between reviews beyond the binary distinction of whether the review occurred before or after the scheduled date. A student who reviews every card one day late receives identical scheduling to one who reviews on time, and a student who never reviews an overdue card suffers no algorithmic consequence — their EF remains frozen at its last recorded value indefinitely [5].

Fourth, the interval recurrence has no upper bound. Each successful review multiplies the previous interval by EF, so an item answered correctly several times in succession recedes geometrically: at the EF ceiling of 2.5 the eighth interval already exceeds four years. In solo study that is the intended behaviour and costs nothing, because the learner's engagement with the material is open-ended. Within a course of fixed duration it means an item can fall due after the enrolment that introduced it has ended. SM-2 offers no concept of an item ceasing to require review, and neither do the widely deployed implementations of it: Anki's *Maximum Interval* bounds how long a card may wait but never stops scheduling it, and its only automatic removal — leech suspension — is triggered by repeated failure rather than by mastery.

These limitations motivate the design adaptations described in Section B, where aBridgeAI's hybrid scheduling model is specified.

3.2.2 Memory Models: Stability and Retrievability

Whilst SM-2 provides a practical scheduling heuristic, its interval recurrence function does not derive from an explicit model of how memory traces form, strengthen, or decay over time. A more theoretically grounded account of memory dynamics is provided by the Two-Component Model of Memory, which characterises the state of any memorised item through two independent variables: memory stability and memory retrievability [23].

The Forgetting Curve

The empirical foundation of SR theory is Ebbinghaus's forgetting curve, first described in 1885 [24]. Ebbinghaus demonstrated through systematic self-experimentation that the probability of successful recall declines as a negatively exponential function of elapsed

time since the last successful review:

$$R(t) = e^{-\frac{t}{S}} \tag{3.4}$$

where $R(t) \in [0, 1]$ denotes the probability of recall at time t days after the last review, and S denotes a stability parameter governing the rate of decay. Items with higher stability retain acceptable recall probability over longer intervals; items with lower stability decay rapidly toward chance-level performance. This formulation implies that forgetting is not a discrete event but a continuous process, and that any fixed review interval will produce different retention outcomes for items of differing stability.

Memory Stability

Memory stability S represents the durability of a memory trace — formally, the inter-repetition interval at which the probability of successful recall equals a target retention threshold, typically set at $R = 0.9$ [23]. Stability is not an intrinsic property of the material alone but reflects the interaction between item difficulty, the learner's prior knowledge, and the history of prior retrievals. Each successful retrieval increases stability, a phenomenon termed the retrieval practice effect [25], with the magnitude of the stability increase depending on the retrievability at the moment of practice: retrievals attempted when recall probability is low produce larger stability gains than retrievals attempted when recall probability is high [26]. This is the theoretical basis for spacing reviews at intervals that allow some forgetting to occur before the next retrieval rather than reviewing while recall is still effortless.

Memory Retrievability

Memory retrievability R represents the probability of successful recall at a specific point in time, computed from the forgetting curve given the current stability S and the elapsed time since the last review t. Unlike stability, which changes only at the moment of review, retrievability decays continuously between reviews. At the moment of a successful review, retrievability resets to 1.0 and then begins declining again according to the exponential function above. Retrievability thus provides a real-time estimate of a learner's current knowledge state for any given item, independent of when the next scheduled review falls [23].

Stability Increase and the Role of Difficulty

The Two-Component Model further specifies that the increase in stability ΔS produced by a successful retrieval is governed by an Optimum Factor (OF), which encodes the relationship between retrievability at the time of review and the resulting stability gain. Items reviewed at low retrievability — that is, items that the learner has nearly forgotten — yield disproportionately large stability increases, consistent with the desirable difficulties framework described by Bjork [26]. This property motivates scheduling reviews near the point of forgetting rather than significantly before or after it, which is the theoretical justification for the interval lengths produced by SM-2 and its successors.

Absolute item difficulty, termed the A-Factor in SuperMemo's formulation [5], modulates stability growth independently of retrievability. High A-Factor items require more repetitions to achieve equivalent stability to low A-Factor items, reflecting genuine differences in material complexity that persist across learners. Separating item difficulty from learner-specific memory dynamics allows the model to distinguish between an item that is universally difficult and a learner who is individually struggling, a distinction that has practical consequences for both scheduling and content design.

Relationship to SM-2

SM-2 can be understood as an approximation of the Two-Component Model operating without explicit stability or retrievability estimates. The Easiness Factor EF functions as a proxy for stability — items with high EF grow intervals rapidly, consistent with high stability — but it encodes no continuous retrievability estimate and applies no forgetting curve between reviews. The consequence is that SM-2 responds only to review outcomes, not to the passage of time between reviews. An item due for review three months ago carries the same EF as the day it was last reviewed, despite the intervening decay in its true retrievability. More recent algorithms, including SM-17 and the FSRS. [27], address this by maintaining explicit stability and retrievability estimates updated continuously over time, at the cost of greater computational and parametric complexity. The theoretical properties of stability and retrievability nonetheless inform the design of aBridgeAI's learner-facing performance metrics, as described in Section B.

3.2.3 Knowledge Tracing and Learner Modeling

SR algorithms model memory at the level of individual items, scheduling each card independently based on its own retrieval history. A complementary line of research, termed knowledge tracing, addresses a broader question: given a sequence of observed practice

attempts, what is the probability that a learner has mastered an underlying knowledge component at any given point in time? Where SR optimises *when* to review, knowledge tracing estimates *whether* mastery has been achieved [28].

Bayesian Knowledge Tracing

The foundational model in this field is Bayesian Knowledge Tracing (BKT), introduced by Corbett and Anderson [28] in the context of the ACT programming tutor. BKT models each knowledge component as a latent binary variable — either mastered or not mastered — and updates the probability of mastery after each practice attempt using Bayes' theorem. The model is parameterised by four values:

- $P(L_0)$: the prior probability that the learner has already mastered the knowledge component before any practice.
- $P(T)$: the probability of transitioning from unmastered to mastered after a single practice opportunity.
- $P(S)$: the probability of a slip — answering incorrectly despite having mastered the component.
- $P(G)$: the probability of a guess — answering correctly without having mastered the component.

After each observed response, the posterior probability of mastery $P(L_n)$ is updated as:

$$P(L_n \mid \text{correct}) = \frac{P(L_{n-1})(1 - P(S))}{P(L_{n-1})(1 - P(S)) + (1 - P(L_{n-1})) \cdot P(G)}, \tag{3.5}$$

$$P(L_n \mid \text{incorrect}) = \frac{P(L_{n-1}) \cdot P(S)}{P(L_{n-1}) \cdot P(S) + (1 - P(L_{n-1}))(1 - P(G))}. \tag{3.6}$$

The updated posterior is then carried forward as the prior for the next practice attempt, allowing the model to track mastery probability as a continuous trajectory over the learner's practice history. A mastery threshold — typically $P(L_n) \geq 0.95$ — determines when a learner is considered to have sufficiently mastered a component and may progress [28].

BKT established several principles that remain influential in educational data mining: the separation of latent knowledge state from observed performance, the explicit modeling of slip and guess probabilities as noise sources, and the use of a mastery threshold as a

progression gate. However, BKT operates on discrete knowledge components modeled independently, with no mechanism for capturing relationships between components or for modeling the continuous decay of knowledge over time [29].

Deep Knowledge Tracing

Piech et al. [30] extended knowledge tracing to sequence modeling using recurrent neural networks, introducing Deep Knowledge Tracing (DKT). Rather than maintaining an explicit probability for each knowledge component, DKT encodes the learner's entire interaction history as a hidden state vector within a long short-term memory (LSTM) network, and predicts the probability of correct response on any future knowledge component from this compressed representation. This allows the model to implicitly capture relationships between knowledge components — for instance, that mastery of TCP flow control is predictive of performance on TCP congestion control — without requiring these relationships to be explicitly specified by a domain expert.

DKT demonstrated substantially improved predictive accuracy over BKT on large-scale educational interaction datasets [30], and subsequent variants have incorporated attention mechanisms, graph structures, and forgetting functions to further improve performance [31]. However, DKT and its successors require substantial interaction data to train reliably, operate as largely opaque models offering limited interpretability, and are computationally intensive relative to the closed-form updates of BKT or SM-2 [32].

Per-Student State Isolation

A principle common to both BKT and SM-2 is that knowledge state is modeled at the level of the individual learner. Each student maintains an independent set of parameters — mastery probabilities in BKT, Easiness Factors in SM-2 — that evolve according to their own practice history without influence from peer performance. This isolation is not merely a computational convenience but a theoretical necessity: the same observable response pattern may reflect different underlying knowledge states in different learners depending on their prior experience, learning strategies, and domain familiarity [28, 33]. Aggregating or sharing state across learners would systematically misrepresent individual knowledge trajectories and undermine the validity of any progression gate derived from the model.

This principle of per-student state isolation is directly adopted in aBridgeAI's learner modeling design, as elaborated in Section B.

Item-Level Demand and Latency Normalisation

A challenge in using raw response latency as a cognitive signal is that observed latency confounds retrieval effort with item-level reading and processing demand. A learner responding to a lengthy, multi-clause technical question will necessarily exhibit higher latency than a learner responding to a brief factual question, regardless of their underlying memory state for the relevant knowledge. Meaningful interpretation of response latency therefore requires normalisation against an item-specific baseline that accounts for the inherent processing demand of the question itself [34].

Several approaches to this normalisation have been proposed in the literature. Standardisation against population response time distributions — dividing individual latency by the mean or median latency for that item across all learners — removes item-level demand while preserving individual differences in retrieval speed. Where population data is unavailable, item-level demand can be estimated from surface features of the stimulus: reading time models based on word count and syntactic complexity provide principled baselines against which individual latency can be compared. Technical content specifically warrants a conservative reading rate estimate, as domain-specific vocabulary and conceptual density slow comprehension relative to general prose. Difficulty ratings assigned by domain experts provide a further adjustment factor, capturing the additional cognitive demand imposed by items that require not only reading but active problem construction or multi-step reasoning [17].

Response Time in Automated Assessment

The use of response time as a formal scoring input in automated assessment systems has gained traction alongside the growth of computer-based testing. Item Response Theory extensions incorporating response time — most notably the hierarchical model of van der Linden [35] — treat latency as a second observed variable alongside accuracy, jointly informing estimates of both item difficulty and learner ability. In adaptive learning systems, response time has been used to adjust item selection, to detect disengaged or random responding, and to calibrate difficulty estimates in the absence of sufficient historical response data [36]. These applications share the common principle that response latency, when properly normalised, provides diagnostic information about the learner's cognitive state that accuracy-only models systematically discard.

The theoretical basis established here — that normalised response latency reflects memory strength beyond what correctness alone captures, and that item-level demand must be accounted for in any latency-based signal — directly motivates the expected

time formulation and Q derivation model specified in Section B.1.

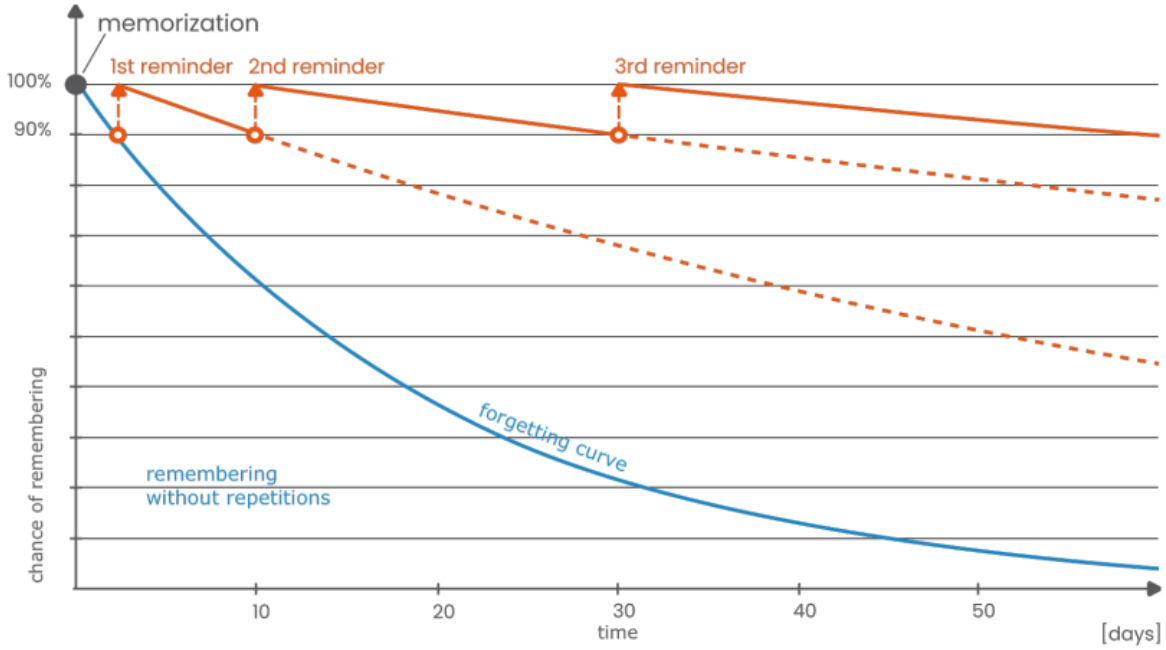


Figure 3.1: Forgetting curves applied in SR learning. Source: [37].

3.3 Core Artificial Intelligence Technologies

3.3.1 LLMs in Education

LLMs, characterized by transformer architectures and self-attention mechanisms, have shifted educational technology from static content delivery toward context-aware generative systems capable of deep semantic understanding. In the aBridgeAI framework, LLMs serve as the primary engine for "fading support" scaffolding, dynamically adjusting the complexity of hints and explanations based on the learner's demonstrated competence. These models facilitate "System-2 level engagement" by generating automated questioning prompts that require students to explain their reasoning in steps, identify underlying assumptions, or solve problems through alternative methodologies. This approach prevents the "AI as Autopilot" failure mode, where a tool provides complete solutions that reduce germane load and hinder mastery; instead, the LLM enforces the productive mental work necessary for schema construction. Furthermore, LLMs enable the large-scale production of high-quality practice questions and quizzes, satisfying the need for a continuous supply of assessment materials in adaptive learning environments without the high cost and resource constraints of manual question construction [38].

Beyond generative scaffolding, LLMs fundamentally resolve the assessment bottlenecks prevalent in legacy LMS. Traditional platforms rely heavily on rigid RegEx or exact string matching, forcing a systemic dependence on MCQs that primarily stimulate recognition memory. LLMs, however, possess the high-dimensional semantic parsing capabilities required to reliably evaluate open-ended, production-based tasks. By analyzing the semantic intent of a student's constructed response—whether written or transcribed via voice—the LLM can accurately distinguish between genuine structural comprehension and superficial keyword mimicry, enabling nuanced, automated grading of short-answer questions and complex technical explanations.

Specifically within the aBridgeAI architecture, LLMs are operationalized to drive the context-aware mock interview module. This functionality requires the model to maintain conversational state and context over multi-turn dialogues, dynamically adapting its follow-up inquiries based on the learner's real-time articulation. To ensure the LLM strictly adheres to these pedagogical objectives rather than defaulting to a subservient, answer-providing assistant, the system employs advanced prompt engineering frameworks, including few-shot prompting and strict persona constraints. By functioning as an adaptive Socratic evaluator, the LLM continuously probes the boundaries of the student's knowledge, actively identifying and reporting the discrepancies between theoretical recall and practical application.

3.3.2 Relational Reasoning: Overcoming Document Silos

Traditional RAG pipelines often suffer from "document silos," where the system treats individual documents or text chunks as isolated units of information. This structural blindness limits relational reasoning, making it difficult for an AI to answer complex queries that require linking configuration rules, transactional dependencies, or conceptual hierarchies distributed across multiple sources[39]. aBridgeAI addresses this through a structured lecture ingestion pipeline, where uploaded source materials — spanning documents, videos, and presentation files — are parsed and decomposed into typed knowledge units rather than stored as raw, undifferentiated text chunks. Each unit preserves its relational origin within the lesson it belongs to, allowing the system to reason across source boundaries rather than treating each file as a self-contained silo. Knowledge is represented not as a flat collection of passages but as a structured set of concepts, summaries, and examples that together form a coherent pedagogical context.

At the retrieval layer, the system combines parametric memory — the LLM's pretrained knowledge — with non-parametric memory held in a dense vector index built

over these structured materials, following the hybrid retrieval architecture described by Lewis et al. [39]. This ensures that AI-generated assessments are grounded in verified, teacher-uploaded content rather than produced purely from the model's internal weights, reducing the risk of hallucination and improving factual specificity. The generation layer is further constrained by teacher-defined configuration. For quiz generation, instructors can supply additional contextual instructions that narrow the retrieval scope to lesson-relevant material. For interview generation, teacher-defined outcome requirements serve as semantic anchors that guide which regions of the knowledge index are consulted when producing sample questions — without requiring a pre-defined answer key from the teacher. This mirrors the principle of cognitive load management described by Sweller et al. (2019)[17], where scaffolded knowledge structures reduce extraneous cognitive load by narrowing the problem space to what is pedagogically relevant, benefiting both the learner and the generative process itself. Together, these design choices position aBridgeAI's knowledge layer not as a collection of flat files but as a structured cognitive ecosystem — one where the relational context between materials, lessons, and assessment outcomes is preserved architecturally, enabling more coherent, traceable, and pedagogically grounded AI-generated content[40].

RAG pipelines address the inherent limitations of purely parametric LLMs, such as their tendency to produce "hallucinations" and their inability to easily expand or revise stored world knowledge. By combining a pre-trained sequence-to-sequence model (parametric memory) with a dense vector index of verified instructional material (non-parametric memory), RAG anchors the AI's output to specific, factual lesson contexts. This architecture ensures that the "supportive information" provided to a learner—which often involves high element interactivity and complex reasoning—is retrieved accurately from a ground-truth source before the LLM generates a response. The retrieval mechanism allows the system to be "human-readable" and "human-writable", enabling educators to update the knowledge base by simply editing the document index without retraining the underlying model parameters. Ultimately, this hybrid approach generates language that is more factual, specific, and diverse than parametric-only baselines, ensuring the learner is presented with information that is both reliable and contextually relevant to their current learning objective[39].

3.3.3 Vector Databases and High-Dimensional Search

Vector databases solve the problem of "semantic isolation" by embedding knowledge into high-dimensional vector spaces, where information is represented as mathematical

coordinates based on semantic meaning. Unlike traditional keyword search, which relies on literal word overlap, high-dimensional search utilizes Maximum Inner Product Search (MIPS) and cosine similarity to identify conceptually related documents. This capability transforms the system from a collection of isolated data points into a "dynamic, interdependent cognitive ecosystem" where semantically related "isolated elements" are sequenced to manage the learner's working memory. When a student struggles with a concept, the vector database can retrieve "just-in-time" procedural information or supportive theoretical backgrounds by identifying the nearest neighbor concepts in the knowledge space. By mapping the dependencies between item difficulty, memory stability, and semantic relevance, high-dimensional search ensures that new information is integrated into a coherent mental model, effectively automating the "isolated elements effect" to prevent cognitive overload during the integration of complex causal systems[3, 17, 39].

3.4 Technical Foundations and System Design Rationale

3.4.1 Component-Based Interface Design and Cognitive Load

The practical software framework for a learning system must be architected to respect the human cognitive architecture, specifically the capacity and duration limits of working memory. To support low-latency and highly interactive applications, the frontend architecture utilizes component-based UI rendering, which facilitates the "segmenting effect" by breaking complex, high-element interactivity lessons into manageable, learner-controlled segments. This approach ensures that learners are not overwhelmed by a single continuous unit of information, allowing working memory resources to recover between processing episodes. Furthermore, a strongly-typed backend architecture is necessary to manage the "state-heavy" nature of individualized learner modeling, where the system must track millions of unique parameters—such as item-specific Easiness Factors (EF), Memory Stability (S), and Memory Retrievability (R)—across high-dimensional matrices. By minimizing extraneous load through a simplified, responsive user interface, the framework ensures that the learner's limited cognitive resources are focused entirely on the germane task of schema construction[2, 3, 17].

3.4.2 Integration of the Adaptive Loop

The system operates through a continuous "use, verify, and correct" feedback loop that synchronizes the web client, vector database, LLM, and SM-2 based algorithms. The

loop begins when the web client presents a learning item; once the student provide a self-assessed quality grade (0–5), this "retrievability" data is sent back to the algorithm to verify the accuracy of the predicted interval. If the grade is low, indicating forgetting, the system immediately "corrects" the multidimensional matrices by reducing the stability increase factors and resetting the repetition schedule. Simultaneously, the LLM utilizes RAG to pull "supportive information" from the vector database, identifying semantically related knowledge nodes through high-dimensional search to provide "just-in-time" procedural hints. This integration ensures that the algorithm "learns the learner" in real-time, shifting from a fixed, heuristic model to a personalized forgetting curve that adapts to individual memory decay rates[5, 37].

3.5 Sumary about the Theoretical Background

The theoretical frameworks and technological paradigms examined in this chapter provide the essential blueprint for aBridgeAI. By grounding the system in the principles of CLT and individualized learner modeling, the system establishes a cognitively aware baseline that respects human memory constraints. The integration of a hybrid SM-2 algorithm and dynamic evaluation rubrics translates these psychological principles into an actionable, mathematical engine capable of adaptive spacing. Furthermore, LLMs, RAG pipelines, and Vector Databases supports the requisite technological infrastructure to maintain semantic context and relational depth at scale.

Together, these theoretical and computational elements form a cohesive ecosystem that directly addresses the limitations of traditional, isolated SRS.

Having established why aBridgeAI is designed the way it is and the core technologies that power it, the focus must now shift to implementation. Chapter 4 – System Requirements and Analysis will translate these theoretical foundations into concrete software requirements, architectural diagrams, and functional specifications, detailing exactly how this cognitive ecosystem is operationalized in practice.

4 System Requirements and Analysis

This chapter provides a comprehensive analysis of the aBridgeAI system. It details the stakeholders and users of the system, business process, functional, non-functional and data requirements, and visualizes the system's behavior through Use Case and Activity diagrams.

4.1 System Overview

This section establishes the foundational context of the aBridgeAI system, outlining its core mission, the entities invested in its success, and the human actors who interact with the platform.

4.1.1 Purpose

The purpose of aBridgeAI is to bridge the gap between theoretical academic curricula and practical industry expectations. By operating as an AI-driven LMS and Mock Interview system, it transforms traditional, passive learning into an active, context-aware application loop. The system aims to automate the generation of assessments and provide personalized interview practices to ensure students are "Job-Ready."

4.1.2 Stakeholders and Users of the System

It is essential to distinguish between stakeholders (those impacted by or invested in the system's outcome) and users (those directly interacting with the software).

Stakeholders

- **Higher Education Institutions and Coding Bootcamps:** The intended institutional adopters of aBridgeAI, which can use the platform to reduce assessment overhead and inspect progression evidence within their authorised scope.
- **Accreditation Bodies, Industry Partners, and Hiring Companies:** External stakeholders that may benefit indirectly from institutional progression evidence and improved graduate preparation. They are not direct system users, and the current platform specifies no direct reporting, data-sharing, or candidate-verification workflow for them.

- **Academic Supervisor and Project Review Committee:** The faculty advisors and reviewers responsible for evaluating the scientific and engineering contributions of this work against academic criteria.

Users of the System

- **Student:** The primary end-user. Consumes multimedia lessons, participates in AI-driven quiz and spaced-repetition quiz review cycles whose intervals are governed by the hybrid SM-2 scheduler, takes AI-conducted mock interviews in a unified hybrid room supporting typed and spoken interaction, and tracks personal progression via the Knowledge Retention Estimate, Progression Readiness, and Review Compliance Rate metrics. The student receives a binary pass/fail outcome together with qualitative remediation and a study plan; numeric rubric scores, per-outcome verdicts, and model reasoning remain hidden.

- **Teacher:** Designs lessons, uploads raw materials for AI ingestion, and is responsible for reviewing AI-drafted quiz and interview questions before publication — including editing quiz cards, confirming the per-card expected response time T_{exp} that anchors Q derivation, defining target outcomes for the mock interview, and reviewing the AI-generated interview question bank. Material ingestion and derived retrieval units are processed automatically rather than individually teacher-approved. Teachers additionally configure the SM-2 unlock parameters ($EF_{\text{min}}^{\text{unlock}}$, coverage threshold τ, prerequisite criteria) per lesson and monitor cohort retention and compliance metrics via the teacher dashboard.

- **Educational Manager:** An academic operations role acting within the organisation or faculty scope granted by its permissions (e.g. a programme lead). May manage pathways, teaching assignments, programmes, and enrolments only within that resolved scope; bulk enrolment and organisation-scoped reporting remain subject to the same boundary. Faculty Deans retain the distinct authority to adjudicate pathway-change requests. An Educational Manager has no cross-tenant visibility.

- **Faculty Dean:** The academic authority over a single faculty or department within an organisation (e.g. a Dean or Vice Dean). The Dean holds the full authority of an Educational Manager within their assigned organisational unit—the dean permission set subsumes the manager one—and additionally owns the governance decisions that belong to a faculty rather than to the organisation as a whole. Chief among

these is authoring learning programmes within their faculty and adjudicating student requests to change career pathway or to drop one: the Dean acknowledges, approves, or rejects each request, and an approval either transfers the student's earned progress onto the incoming pathway or closes the dropped one against a snapshot of what it had earned. The Dean's authority is bounded by the faculty they administer.

- **IT Administrator:** The super-user responsible for cross-tenant operations. Provisions new organisations and their email domains, tunes the platform's runtime configuration both deployment-wide and per organisation, invites the initial Educational Manager for each newly provisioned tenant, monitors system health, processing backlog and AI spend, and enforces platform-wide security and compliance policies. The IT Administrator holds the full platform permission set, since recovering a tenant from an operational failure requires it; in normal operation, however, course content and assessment configuration remain the responsibility of the Educational Manager and the Teachers of each tenant, and every administrative action taken inside a tenant is recorded in an append-only audit trail during its configured retention period.

4.2 Business Process

Traditional educational workflows rely heavily on manual content creation, fixed assessment schedules, and uniform grading criteria, creating systematic bottlenecks in delivering personalised, competency-driven feedback at scale. aBridgeAI reengineers this workflow through a five-stage AI-orchestrated process that separates programme governance, content preparation, knowledge validation, practical assessment, and longitudinal reporting into distinct, traceable phases.

Figure 4.1 shows the same process organised by the role that performs each step. The two views are deliberately orthogonal: the stages below say *what* happens and in what order, while the lanes say *who* is accountable and where work is handed between them. They do not line up one to one — Stage 1 spans the administrator and dean lanes, Stages 2 and 3 sit almost entirely with the teacher and the student respectively, and Stage 5 appears as the dashed return edges that carry learner evidence back to the teacher and progress data back to the dean. Those return edges are the reason the process is a loop rather than a pipeline, and they are the hardest part to see in a numbered list.

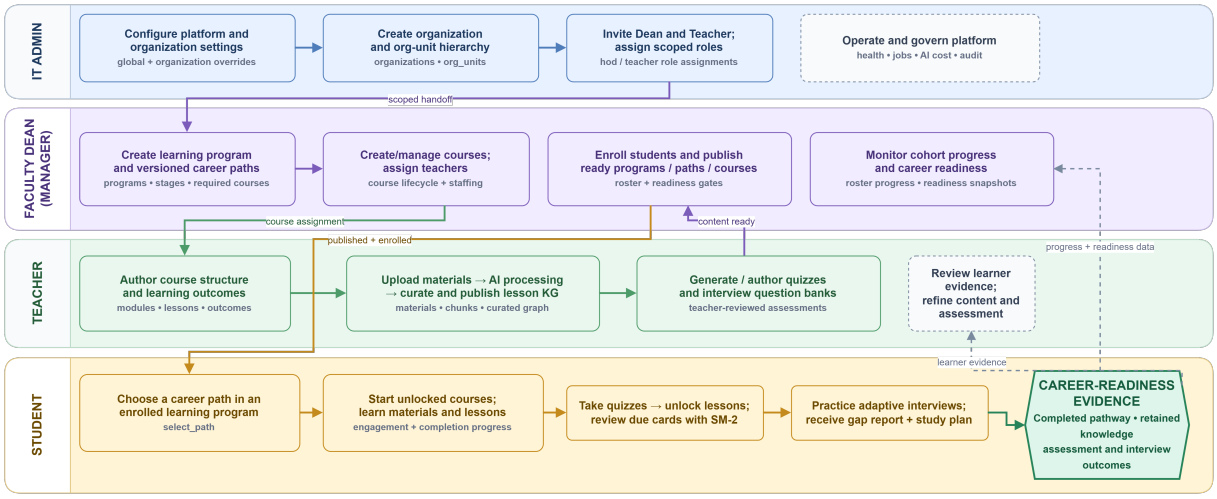


Figure 4.1: End-to-end business process by role: platform and organisation setup, programme and pathway governance, content authoring and assessment preparation, the student learning journey, and the career-readiness evidence it produces.

1. **Programme Governance and Enrolment (Admission).** Before any content is authored, the structures that give a course its meaning must exist. Within a faculty, a Faculty Dean or an Educational Manager defines a *learning programme* that bundles one or more career pathways — the ordered, staged sequences of courses that constitute a professional competency — and publishes it. Publication freezes a version: subsequent edits open a new draft, so a student who enrolled under one version continues to see the curriculum they joined. Authorised operators then admit students, individually or by bulk import, subject to an organisation-configurable ceiling on concurrent enrolments. Each programme version nominates one of its pathways as the default, and admission attempts to activate it when it is eligible and the concurrent-path constraints permit. Otherwise the enrolment remains awaiting pathway selection. A student may afterwards take on further pathways of their own accord, up to a limit the programme sets. Enrolment is what grants access to the courses that Stages 2 through 4 operate upon; without it a student has no curriculum and the loop cannot begin.

2. **Content Ingestion (Preparation).** Teachers upload raw learning materials in supported formats including PDF documents, MP4 video recordings, and presentation files. The ingestion pipeline parses each source, extracts typed knowledge units — summaries, concepts, examples, and indexes them in a dense vector store to support semantic retrieval. The resulting knowledge base supports retrieval-grounded

AI drafting where source material is available. Teachers may additionally author assessment questions manually and supply learning outcomes or supplementary instructions that guide assessment generation and evaluation.

3. **The Learning Loop (Knowledge Validation).** Students consume published learning materials and undertake quizzes whose questions may be teacher-authored, AI-drafted, or imported from curated banks to validate baseline knowledge retention. Each quiz card carries a teacher-confirmed expected response time T_{exp}. The system derives the quality response value Q algorithmically from response correctness, hint usage, and the time ratio $\rho = T_{\text{actual}}/T_{\text{exp}}$, updating the student's per-card Easiness Factor EF via the hybrid SM-2 scheduling model. Due reviews may generate in-app reminders when enabled; students remain in the learning loop until their EF distribution across the lesson's cards satisfies the teacher-configured unlock condition.

4. **The Application Loop (Practical Assessment).** Upon satisfying the relevant learning and access conditions, students gain access to a mock interview. Interview questions are selected from a pre-generated, teacher-reviewed question bank. Questions may be linked to one outcome; the system reports incomplete outcome coverage as a readiness warning rather than universally blocking publication. Each session uses a unified hybrid interaction space that supports both typed and spoken turns.

   When a session is completed or times out after assessment has begun, an asynchronous evaluation worker assesses recorded answer evidence against the configured outcomes, produces diagnostic rubric information, and generates a gap report and study plan. After successful evaluation, the student receives a binary pass/fail verdict together with qualitative remediation guidance. Pending, recovery, or terminal evaluation-failure states do not invent a verdict. Numeric rubric scores, per-outcome verdicts, and model reasoning remain hidden from the student, while authorised teachers may review the transcript and outcome-level assessment. A curriculum unit is considered complete only when its configured completion conditions, including any required interview pass verdict, are satisfied.

5. **Evaluation and Reporting (Feedback).** The system continuously aggregates per-card SM-2 state into three learner-facing metrics: the Knowledge Retention Estimate $\hat{R}_{s,\ell}$, Progression Readiness, and Review Compliance Rate $\rho_{s,\ell}$. These metrics

are surfaced in per-student and class-wide dashboard views accessible to both students and teachers. At the career level, course completion percentages are aggregated into a Career Readiness score reported to the student, the Educational Manager, and the Faculty Dean who owns the programme. Teachers can inspect cohort-level patterns — including systematic retention gaps and declining compliance rates — through learning-loop dashboard views, enabling intervention at the content or configuration level rather than purely at the individual student level.

Reporting also feeds a slower, governance-level response. A student whose progression shows that they are on the wrong pathway may request a change of career pathway, or — where they carry more than one — that a pathway be dropped, in each case with a written justification; the Faculty Dean who owns the programme acknowledges the request and then approves or rejects it against the same progression evidence, and may never adjudicate their own. Taking on a pathway is thus the student's to decide and laying one down is not, and no approval may leave a student with none. An approval snapshots the progress earned on the outgoing pathway and preserves completed courses as awards that carry across, so that work shared between two pathways is credited once and never repeated. The decision, and the reason behind it, are returned to the student.

The five stages form two nested feedback loops that turn at different speeds. The *pedagogical loop* closes within a course: content ingestion informs quiz generation, quiz performance governs interview access, interview outcomes determine curriculum completion, and reporting surfaces the gaps that guide both student remediation and teacher reconfiguration of content and assessment criteria. It turns with every review session.

The *governance loop* closes around the programme and turns rarely. Enrolment in Stage 1 fixes the pathway a student is measured against; the reporting of Stage 5 evidences whether that pathway remains the right one; and a dean-adjudicated pathway change feeds back into Stage 1 by opening a new attempt under a different pathway, without discarding what the student has already earned. Separating the two matters because they fail differently: a defect in the pedagogical loop misjudges a single student's readiness on one lesson, whereas a defect in the governance loop misdirects an entire programme of study.

4.3 Requirements

4.3.1 User Requirements

User Requirements articulate the functional needs of the system from the perspective of the end-users.

Student Requirements

- **UR-STU-01:** As a Student, I want to consume multimedia course materials and complete spaced-repetition quiz reviews scheduled by the system so that I can build durable retention of lesson content.
- **UR-STU-02:** As a Student, I want to participate in context-aware mock interviews through a unified hybrid experience supporting typed and spoken turns so that I can practise articulating my understanding under realistic conditions.
- **UR-STU-03:** As a Student, I want to receive the binary pass/fail outcome and qualitative remediation guidance for each interview session — without numeric rubric scores, per-outcome verdicts, or model reasoning — so that I can understand how to improve while being evaluated on overall demonstrated competence rather than on isolated answers.
- **UR-STU-04:** As a Student, I want to track my own progress through learner-facing metrics — per-lesson Knowledge Retention Estimate, Progression Readiness, and Review Compliance Rate — so that I can self-regulate my study pace without depending on instructor intervention.

Teacher Requirements

- **UR-TCH-01:** As a Teacher, I want to upload and organise learning materials so that I can build a structured knowledge base for each lesson.
- **UR-TCH-02:** As a Teacher, I want the system to draft quiz questions and interview prompts from my uploaded materials, and I want to review, edit, approve, or reject each draft before publication, so that AI-generated assessments retain pedagogical accuracy through Human-in-the-Loop oversight.
- **UR-TCH-03:** As a Teacher, I want to confirm the expected response time T_{exp} for every published quiz card and define the outcome criteria for every published

interview so that the system can compute response-time ratios and binary pass/fail verdicts against my domain judgment rather than against generic heuristics.

- **UR-TCH-04:** As a Teacher, I want to configure per-lesson spaced-repetition parameters — the minimum Easiness Factor for unlock ($EF_{\min}^{\text{unlock}}$), the coverage threshold (τ), and lesson prerequisites — so that progression stringency can be tuned to the criticality of each lesson.

- **UR-TCH-05:** As a Teacher, I want a cohort monitoring dashboard exposing the class-wide distribution of Knowledge Retention Estimates, a per-student breakdown of an individual's review history, an at-risk list flagging low compliance, frozen retention and a widening gap between recognition and explanation, and the cards the cohort finds hardest, so that I can identify at-risk learners and detect systemic weak points in my source material.

Educational Manager Requirements

- **UR-MGR-01:** As an Educational Manager, I want to enrol Students onto a course or a learning programme in bulk from a CSV file — with accounts created automatically for addresses the platform has not seen, and per-row failures reported without losing the rest of the batch — and to invite individual Teachers and Students into my organisation with the role they are to hold, so that I can provision a cohort without creating every account by hand.

- **UR-MGR-02:** As an Educational Manager, I want to define career pathways as versioned sequences of ordered stages, each grouping the courses a Student must satisfy before the next opens, and to assign Teachers to those courses, so that my institution's curriculum structure is reflected in the platform's progression model and a revision published mid-enrolment does not change what an enrolled Student must finish.

- **UR-MGR-03:** As an Educational Manager, I want to view organisation-scoped Career Readiness snapshots — aggregated across all enrolled Students in a pathway, with drill-down to individual progress — so that I can report on cohort outcomes without exposing per-interview rubrics.

- **UR-MGR-04:** As an Educational Manager, I want all my actions to be strictly scoped to my single tenant organisation so that I cannot accidentally or intentionally access data belonging to other institutions on the platform.

Faculty Dean Requirements

- **UR-DN-01:** As a Faculty Dean, I want to review, acknowledge, approve, or reject eligible student pathway-change and pathway-drop requests within programmes owned by my faculty, with a recorded reason and no self-approval, so that pathway governance is accountable and scoped to the appropriate academic authority.

IT Administrator Requirements

- **UR-ITA-01:** As an IT Administrator, I want to provision new tenant organisations and invite their initial Educational Manager so that institutions can be onboarded onto the platform without my involvement in their internal user management.

- **UR-ITA-02:** As an IT Administrator, I want to tune the platform's operational parameters at runtime — enrolment ceilings, retention windows, AI timeout and rate-limit behaviour, ingestion and retrieval thresholds, and the toggles that enable or disable optional processing stages — with an organisation-level override where a tenant needs to differ, so that the deployment can be adjusted to institutional demand without a code change.

- **UR-ITA-03:** As an IT Administrator, I want to monitor cross-tenant system health, infrastructure metrics, and retained append-only audit records so that I can ensure platform stability and respond to security or compliance incidents.

- **UR-ITA-04:** As an IT Administrator, I want every action I take inside a tenant — restoring deleted content, disabling an account, amending an organisation's configuration — to be recorded in retained append-only audit records, so that platform authority broad enough to recover a tenant from an operational failure remains accountable, and pedagogical decisions stay in practice with the Educational Manager and Teachers even where the permission model does not forbid me from making them.

- **UR-ITA-05:** As an IT Administrator, I want the platform to enforce centralised authentication and identity controls — including Google SSO, two-factor authentication, session management, and role-based access control — so that user access is uniformly secured across all tenant organisations.

4.3.2 Functional Requirements

The functional requirements of aBridgeAI specify the concrete system behaviours, processes, and user interactions that the platform must support. They are organised into eight functional modules: (1) Authentication and Access Control, (2) Organisation and User Management, (3) Course and Material Management, (4) Learning Loop and Adaptive Quiz, (5) Application Loop and Mock Interview, (6) Analytics, Reporting, and Notifications, (7) Learning Programs and Path Governance, and (8) Platform Configuration and Policy Documents. Each requirement is traced to one or more User Requirements to ensure comprehensive coverage and end-to-end traceability. The complete specification — one hundred requirements, each with its actors, traced user requirement, feasibility and testability assessment, and priority — is given in Appendix A (Volume II), and is not reproduced here.

The modules divide along the lines the system itself is built on. Modules 1 and 2 establish who a user is and what they may do: federated sign-in, second-factor verification, and the dynamic role and permission model, then the tenant structure of organisations, faculties and memberships within which every later permission is resolved. Modules 3 to 5 carry the teaching loop in the order a learner meets it — a course and its ingested materials, the quiz cycle and the spaced-repetition scheduling that governs progression, and the mock interview that tests explanation rather than recognition. Module 6 turns what those produce into reporting, notification and audit.

The final two modules are the ones most often mistaken for administration rather than pedagogy. Module 7 governs the enrolment structure above individual courses — versioned learning programmes, the career pathways they bundle, and the dean-adjudicated workflow by which a student moves between pathways without losing earned progress. Module 8 covers the runtime configuration catalogue and the versioned policy documents, the two surfaces through which the platform itself is tuned and its terms published.

Two properties hold across the assessment requirements and are worth stating once rather than repeating in each requirement. AI-drafted quiz and interview questions enter a review state and require a human decision before publication; automatically derived ingestion units instead support retrieval and generation. Rules that judge a learner — such as the integrity policy on an attempt, the pathway version on an enrolment, and the programme version on a student — are frozen when they start applying, so that editing the curriculum does not retroactively change what somebody already under way is measured against.

4.3.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of aBridgeAI, specifying how the system behaves rather than what it does. They govern performance, scalability, security, and usability properties that collectively determine the reliability and institutional suitability of the platform. The complete Non-Functional Requirements specification is provided in Appendix B (Volume II). Representative points are summarised below:

- Performance and latency targets ensure AI interview responses and core learning workflows remain responsive for users.
- Scalability and reliability requirements target modest peak concurrency while prioritising graceful degradation and uptime.
- Security and compliance mandates cover encryption at rest, RBAC, audit logging, session management, and the input and output guards that protect the AI interviewer against prompt injection and content leakage.
- Usability and accessibility requirements emphasise responsive UI and WCAG 2.1 AA alignment.

Two of these carry more weight than their priority suggests. The concurrency target is modest by design — the deployment is institutional rather than public, and the measured headroom against it is reported in Section 7 — but the accessibility requirement is the one most likely to be treated as optional and least likely to be recoverable late, since retrofitting keyboard navigation and contrast onto a finished component library costs far more than building to them. The nineteen requirements, with their actors and priorities, are enumerated in Appendix B (Volume II).

4.3.4 Data Requirements

This section summarises the data demands of aBridgeAI across its core functional domains. Rather than specifying a physical schema, the data requirements identify what information must be captured, maintained, and related to support each system capability. The complete specification — fifteen domains, covering the full lifecycle of content creation, assessment, learner modeling, and progress reporting — is provided in Appendix C (Volume II). The governing demands are as follows.

- **Audit and traceability.** Mutable, auditable domain records capture the timestamps and responsible actors needed for compliance reporting and dispute resolution; soft-deletable records additionally record deletion metadata. Append-only event and history records may retain only their creation timestamp and actor context appropriate to that event.
- **Identity and access.** One account belongs to at most one organisation at a time. Permissions are not hardcoded to roles but defined at runtime, and role assignments are scoped to one of four levels — global, organisation, organisational unit, or course — so that the same person may hold different authority in different contexts.
- **Curriculum structure.** Organisations contain faculties and organisation-owned career pathways. Faculty-scoped learning programmes bundle pinned pathway versions; courses divide into modules, and each module holds lessons, quizzes, and interviews as sibling curriculum items.
- **Content provenance.** Uploaded file content is retained through material versions while material metadata remains editable. Quiz and interview questions record their generation provenance and review state; automatically derived ingestion units support retrieval but are not individually teacher-review artefacts.
- **Assessment records.** Spaced-repetition state is held per student-card pair in isolation from every peer, while quizzes additionally maintain a single defensible grade of record per student. Unlock and completion are stored separately, because entering a piece of curriculum and discharging it are distinct facts.
- **Integrity and safety.** Session activity signals and AI guard decisions are recorded as evidence for human review; the guard trail is redacted by design, holding a fingerprint of an offending input rather than the input itself. The integrity policy is frozen onto an attempt when it starts, so a later edit cannot re-judge work already under way. Where a quiz requires a camera, the current implementation checks only that an active video track is present and keeps the stream local to the learner's browser.
- **Operational telemetry.** External AI calls record available token usage, latency, and outcome. When token usage and a model-price entry are configured, an estimated cost is persisted at call time so that later price changes cannot rewrite that estimate.

4.4 System Modeling

This section provides a visual and structured representation of the functional behavior of the aBridgeAI system. The system is modeled using Unified Modeling Language (UML) artifacts, primarily Use Case Diagrams, to describe how each actor interacts with the major functional modules of the platform.

The system is decomposed into eight functional modules: Authentication and Access Control, Organization and User Management, Course and Material Management, Learning Loop and Adaptive Quiz, Application Loop and Mock Interview, Analytics, Reporting and Notifications, Learning Programs and Path Governance, and Platform Configuration and Policy Documents.

4.4.1 Overall Use Case Diagram

The Overall Use Case Diagram provides a macroscopic view of the aBridgeAI system boundary and the major interactions between the system and its authorised actors. The primary actors include Students, Teachers, Educational Managers, Faculty Deans, and IT Administrators. A Faculty Dean inherits every use case available to an Educational Manager within their faculty, so the diagram shows only the governance use cases that are distinctively theirs.

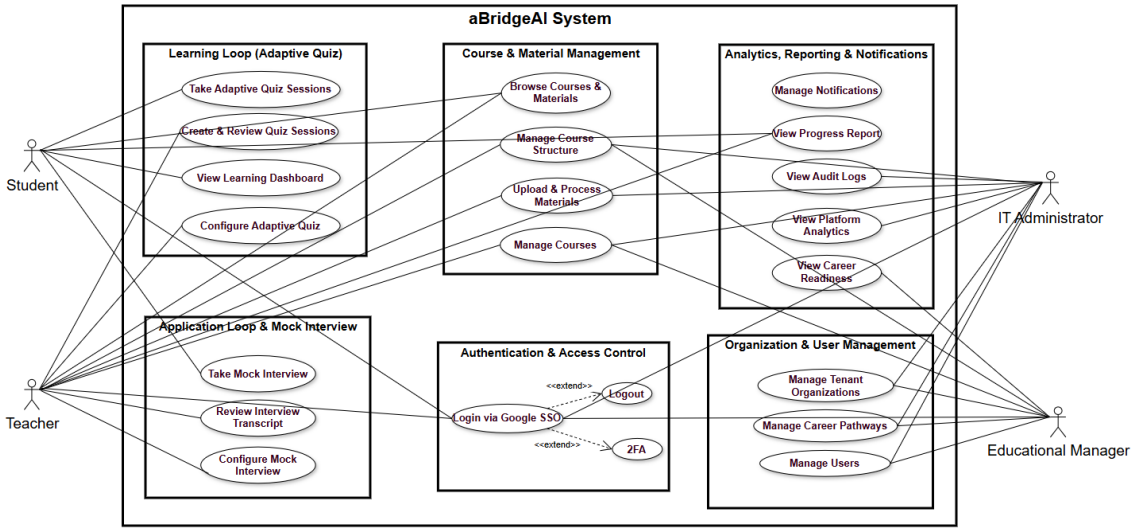


Figure 4.2: Overall Use Case Diagram of the aBridgeAI System

Note. In this diagram, authentication is treated as a prerequisite for all protected use cases rather than as a repeated *«include»* relationship.

4.4.2 Detailed Use Case Diagrams and Use Case Specifications

The detailed use case diagrams decompose the overall system into module-level views. Each module-level diagram is followed by use case specifications that illustrate the most important workflows of that module. To keep the main report concise, only the most critical use cases are described in this section. The complete set of use case specifications is provided in Appendix D (Volume II).

Authentication and Access Control Module

This module covers Google SSO authentication, logout, authenticated session management, multi-factor authentication, role-based and permission-based access control, and security-related audit events.

The diagram highlights that all users must authenticate before accessing protected features. It also shows that sensitive account-security operations require recent second-factor verification, while access to backend routes and frontend UI actions is controlled by the authenticated user's active roles and effective permissions.

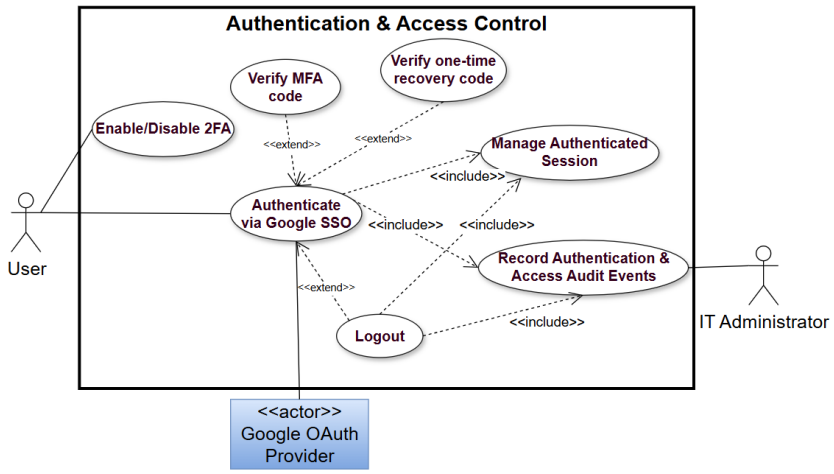


Figure 4.3: Detailed Use Case Diagram: Authentication and Access Control Module

The full set of use case specifications for this module is provided in Appendix D (Volume II).

Organization and User Management Module

This module supports multi-tenant organisation management, user onboarding, organisation memberships, user profiles, domain-based auto-provisioning, and tenant-scoped access control.

IT Administrators are responsible for provisioning and managing tenant organisations, organisation domains, and account status. Educational Managers manage learning operations such as bulk student enrolment, teacher-course assignment, and pathway administration within the organisation or faculty scope resolved from their permissions. The module ensures that Educational Manager actions and queries remain tenant-scoped.

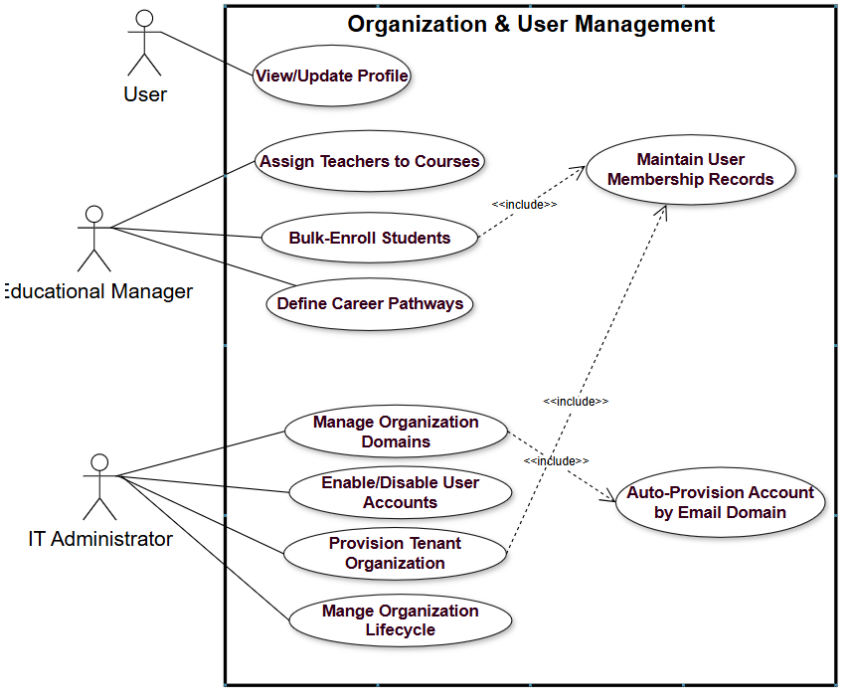


Figure 4.4: Detailed Use Case Diagram: Organization and User Management Module

The use case specifications for this module — including tenant organisation management, which exercises the provisioning, lifecycle and domain-attachment flows in full — are given in Appendix D (Volume II).

Course and Material Management Module

This module allows Teachers and Educational Managers to manage course lifecycles, organise modules and learning items, configure prerequisites and unlock rules, upload materials, manage material versions, and monitor ingestion progress.

The diagram also shows the student-facing side of the module, where enrolled Students can browse published courses, view course details, and access approved materials. Material visibility is controlled separately from course publication status, ensuring that students only access content that is both published and explicitly marked visible.

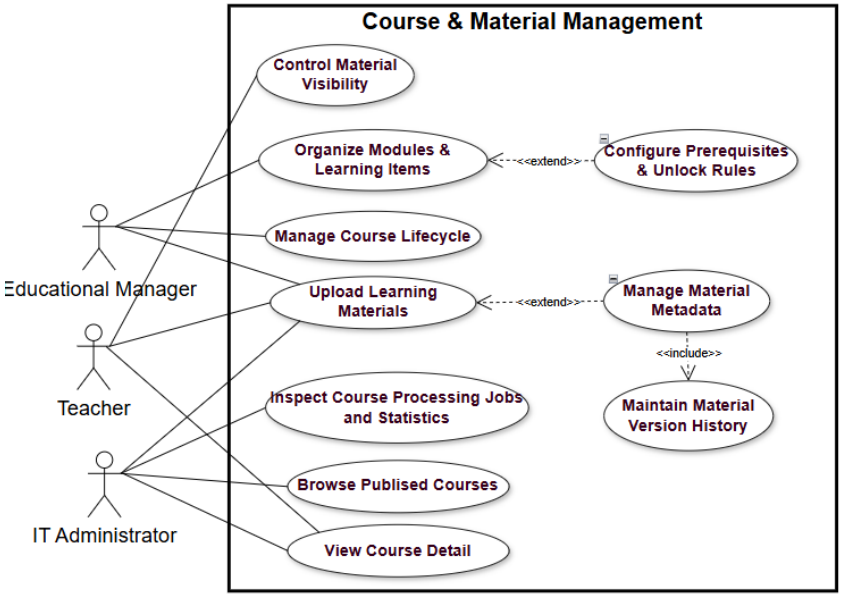


Figure 4.5: Detailed Use Case Diagram: Course and Material Management Module

The use case specifications for this module — including the upload and ingestion workflow for learning materials — are given in Appendix D (Volume II).

Learning Loop and Adaptive Quiz Module

This module represents the core learning mechanism of aBridgeAI, combining AI-assisted quiz question authoring, teacher review, adaptive quiz attempts, quality-score derivation, and hybrid SM-2 spaced-repetition scheduling.

Teachers create or review quiz questions, configure expected response time T_{exp}, spaced-repetition parameters, unlock thresholds, and prerequisites, and may require a camera for a given quiz. Students take quiz sessions, during which correctness, hint usage, and response time are recorded. These values are used to derive the per-card quality score Q and update each student's spaced-repetition state. A live attempt admits only one client at a time, with the learner able to transfer it to another device by explicit confirmation; where a camera is required, the current implementation checks only that an active video track is present and keeps the stream local to the browser. The module also supports lesson access gating, due-card reminders, remediation links, and learning-loop dashboards.

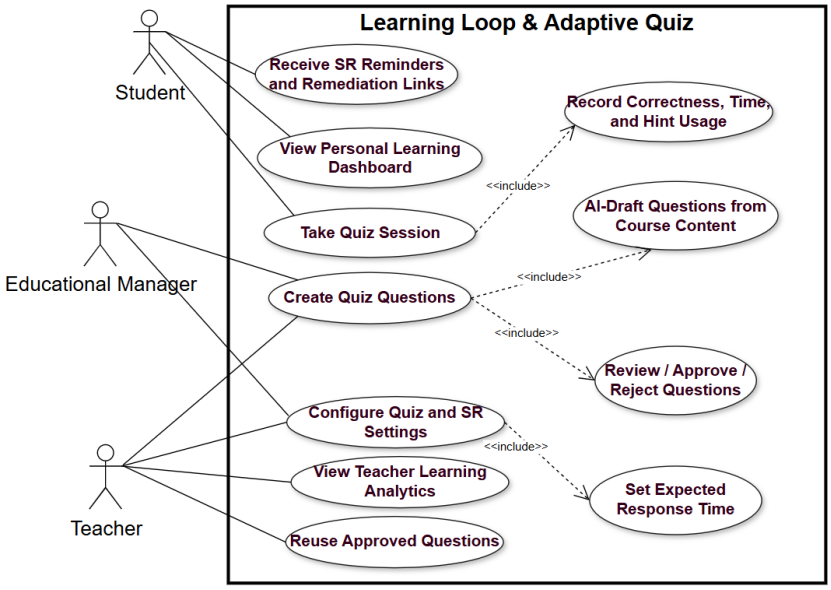


Figure 4.6: Detailed Use Case Diagram: Learning Loop and Adaptive Quiz Module

The use case specifications for this module — including the adaptive quiz workflow, with Q derivation and hybrid SM-2 scheduling — are given in Appendix D (Volume II).

Application Loop and Mock Interview Module

This module focuses on transforming learned knowledge into applied interview performance. Teachers define desired outcomes that students must demonstrate, while the system can draft questions from ingested course content or accept manually authored questions. Each question may be linked to one desired outcome; unlinked questions and incomplete outcome coverage produce readiness warnings rather than universally preventing publication.

Students participate in mock interviews through a unified hybrid session supporting typed and spoken turns. The system persists the interview transcript and session metadata, then evaluates recorded answer evidence against teacher-defined desired outcomes asynchronously. After successful evaluation, students receive a binary pass-or-fail verdict together with a qualitative gap report and study plan; numeric rubric scores, per-outcome verdicts, and model reasoning remain hidden. Pending, recovery, and terminal evaluation-failure states do not invent a verdict. Teachers can review the transcript and outcome-level assessment for remediation purposes. The module also records browser activity signals to support post-session integrity review; these signals support human review but do not constitute conclusive evidence of misconduct or automatically terminate an interview.

Because the assessment is conducted through a conversational model, the module

additionally guards the exchange in both directions: learner input is screened for prompt-injection attempts, and model output is screened for leakage of protected content before it reaches the learner. The guard defaults to shadow mode, which assesses and records security signals without changing learner-facing flow. Explicit enforcement configuration may refuse, redirect, or end the relevant turn rather than failing the whole session. Redacted security events are persisted, and authorised review is scope- and permission-controlled without exposing raw attack content. In the deployed configuration, a native LiveKit agent maintains conversational context while server-side selectors, state writers, and tool gates retain authority over progression. A routed legacy progression path remains available as a rollback configuration rather than the normal spoken-turn path.

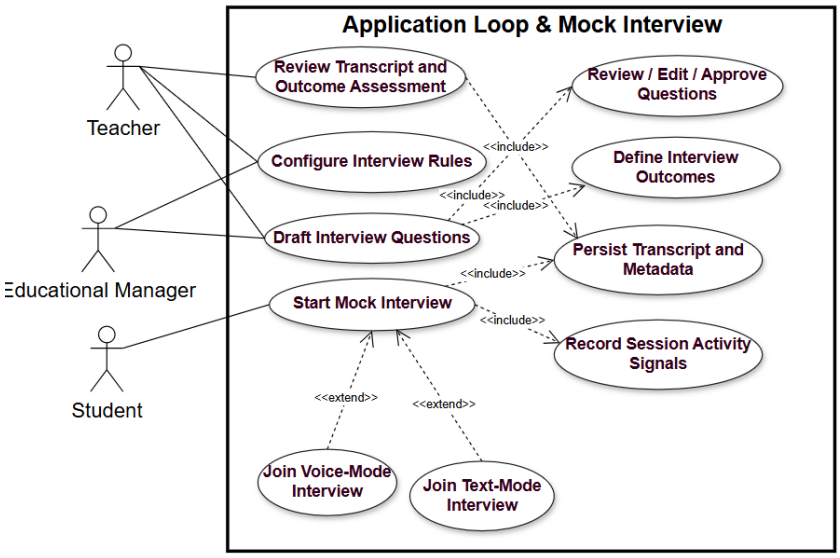


Figure 4.7: Detailed Use Case Diagram: Application Loop and Mock Interview Module

The full set of use case specifications for this module is provided in Appendix D (Volume II).

Analytics, Reporting and Notifications Module

This module consolidates progress tracking, notification delivery, administrative analytics, audit-log querying, AI cost telemetry, processing-job inspection, and Career Readiness reporting.

Students can view completion progress and receive notifications, while Teachers can monitor roster progress and identify at-risk learners. Educational Managers can view organisation-scoped Career Readiness snapshots and drill down into individual learner

progress. IT Administrators can inspect platform statistics, AI usage, processing jobs, and append-only audit records retained under the configured retention policy for security and compliance review.

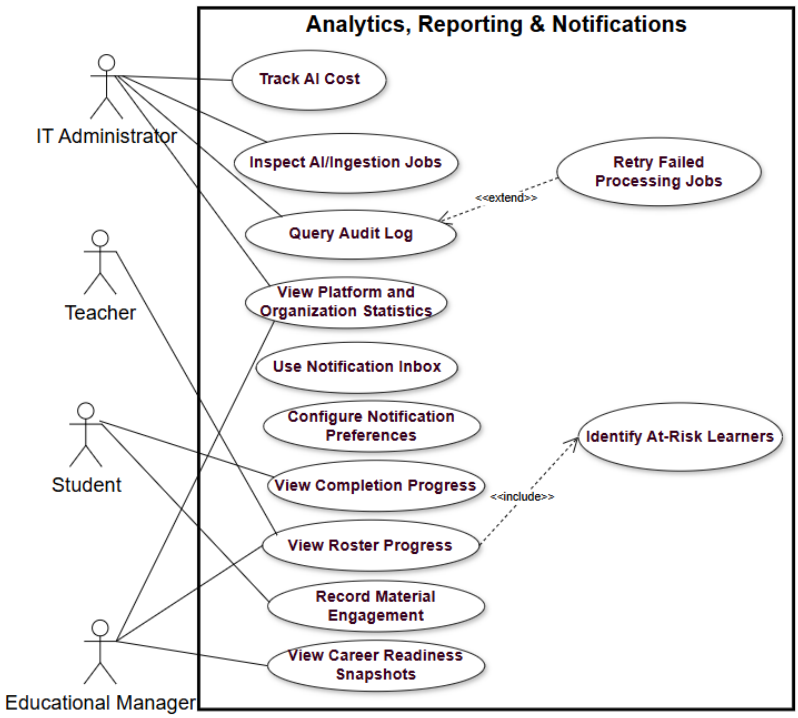


Figure 4.8: Detailed Use Case Diagram: Analytics, Reporting and Notifications Module

The full set of use case specifications for this module is provided in Appendix D (Volume II).

Learning Programs and Path Governance Module

This module governs the enrolment structure that sits above individual courses. A *learning programme* bundles one or more career pathways into a single enrolment offered by a faculty, and this module covers programme authoring, versioning, student enrolment, pathway selection, and the dean-adjudicated workflow by which a student changes or relinquishes a pathway.

Faculty Deans and Educational Managers author programmes within their own faculty and move them through draft, published, and archived states. Authoring is version-based: an edit opens a new draft rather than mutating a published version, so a student who enrolled under version 2 continues to see version 2 for the life of their enrolment. Authorised operators enrol students individually or by bulk import, subject to an organisation-

configurable ceiling on concurrent programme enrolments.

On the student-facing side a programme version nominates one of its pathways as the default, and enrolment attempts to activate a system-originated attempt on it when the path is eligible and the concurrent-path constraints permit. Otherwise the enrolment remains awaiting selection. A student may then take on further pathways without approval, up to a limit the programme sets, which may exceed neither the pathways it bundles nor an organisation-configurable ceiling. Relinquishing a pathway is governed rather than free: a student may request either a change of pathway or, holding more than one, that a pathway be dropped, in each case with a mandatory written justification. Both kinds share a single open request per enrolment and a single finite switch allowance, which an approval spends and nothing restores. A request is adjudicated by a Faculty Dean who owns the programme and who may not be the subject of it. Approval closes the outgoing attempt with a progress snapshot, opening a replacement attempt for a change and none for a drop; no approval may leave a student with no pathway in progress, that being a withdrawal rather than a pathway decision. Course completions already earned are preserved as immutable awards, so work shared between pathways is never repeated, and the enrolment completes only once every pathway the student holds is itself complete.
A dedicated rendered use case diagram for this module is not yet included; the programme authoring, conditional default-path activation, pathway selection, and dean-adjudicated change-request workflows are specified textually in Appendix D (Volume II).

Platform Configuration and Policy Documents Module

This module covers the operational surfaces through which the platform itself is tuned and through which its governing documents are published. It comprises two related concerns: a catalogue of runtime-tunable settings, and a versioned store of policy documents.

Every setting declares its key, grouping, data type, default value, and description. The effective value of a setting is resolved by a fixed precedence of organisation override, deployment-wide stored value, environment configuration, and code default, so that an IT Administrator tunes the deployment while an Educational Manager tunes only their own organisation. Because a mis-set value can affect every tenant, a change may first be previewed as a validated dry run that reports what it would alter; every applied change carries a mandatory written reason and is recorded so that it can later be cleared or rolled back to the value it replaced. Should a stored setting become unreachable, unreadable, or invalid, the system falls through to the next precedence layer rather than failing.

Policy documents — each classified as legal or academic — hold their content in inde-

pendently numbered versions, so that the text in force on a given date remains recoverable. The index of published policies and the current text of any published policy are served without requiring authentication, since a prospective user must be able to read the terms before holding an account.

A dedicated rendered use case diagram for this module is not yet included; runtime-setting precedence, preview, rollback, and policy-document workflows are specified textually in Appendix D (Volume II).

4.4.3 Activity Diagrams

Activity diagrams model the dynamic behavior of the system by illustrating the flow of activities, decision points, and asynchronous AI-driven processes. The system activity diagrams focus on four major workflows: Authentication, Smart Content Ingestion, Human-in-the-Loop Quiz Generation, and Context-Aware Mock Interviews.

The following activity diagram illustrates the Human-in-the-Loop quiz generation workflow. The full set of activity diagrams is provided in Appendix D.

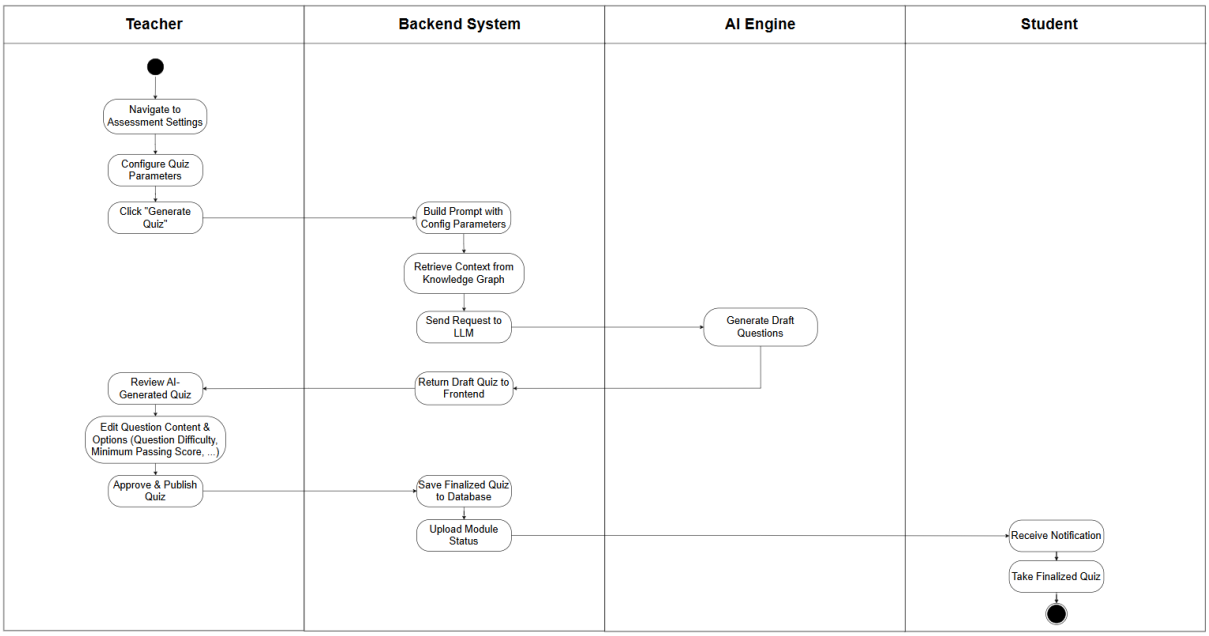


Figure 4.9: Activity Diagram: HITL Quiz Generation and Execution

The following activity diagram details the material ingestion workflow in aBridgeAI. It shows how teachers upload learning materials directly to object storage, how the backend verifies the upload and queues processing, and how the ingestion worker extracts, chunks, embeds, and stores content for retrieval.

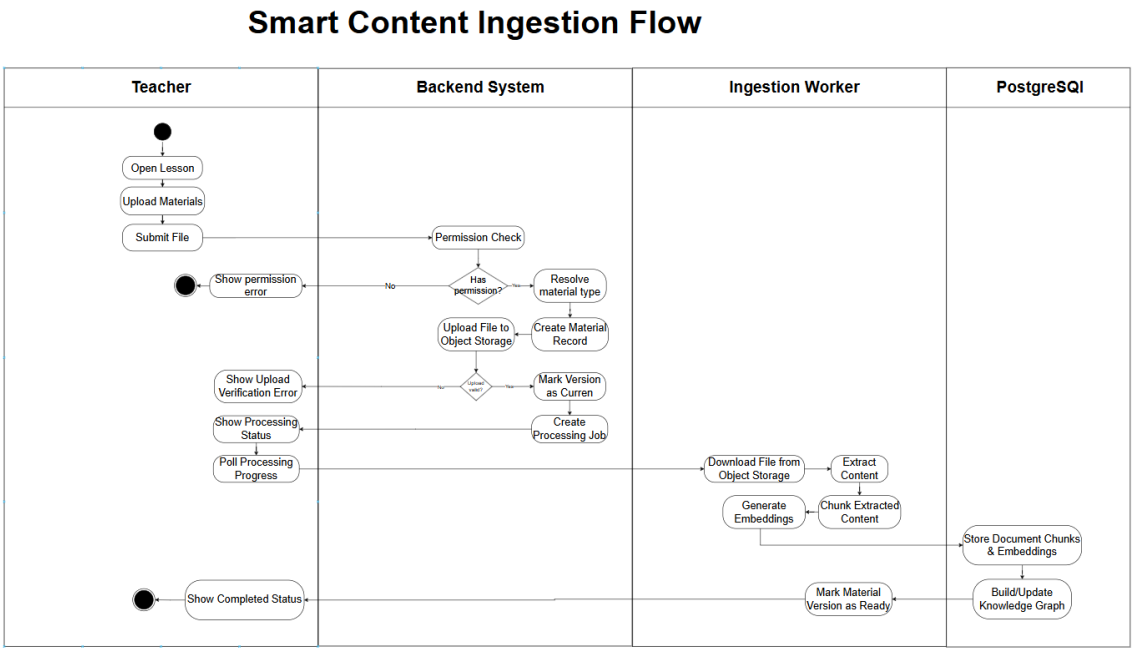


Figure 4.10: Activity Diagram: Smart Content Ingestion and AI Parsing

Detailed activity diagrams are provided in Appendix D.

5 System Design

5.1 Technology Stack Selection

The technology stack for aBridgeAI was selected by evaluating each major system component against its most viable alternatives. Each decision is grounded in the system's core technical constraints: native AI/ML orchestration, real-time WebRTC voice processing, high-throughput vector search, and graph-based knowledge representation.

Each subsection below states the constraint the component must satisfy and the decision it led to. The comparison matrix behind each — the chosen option scored against the alternatives on the dimensions that mattered — is given in Appendix A.

5.1.1 Backend Framework

The backend must orchestrate adaptive SR scheduling, knowledge retention estimation, asynchronous LLM calls, and interview lifecycle operations. The implementation uses first-party Python modules for retrieval, state policy, and model integration, together with standard scientific libraries where appropriate; it does not depend on a chain-composition framework.

Decision: FastAPI. FastAPI combines Python-native AI library access with a fully asynchronous request model. It provides the lifecycle, authorization, token, dispatch, and persistence APIs for the native LiveKit interview worker, while independent worker processes hold long-running real-time sessions and asynchronous `arq` jobs. Its built-in OpenAPI generation also accelerates API contract development across the team.

5.1.2 Frontend Framework

The frontend must support both a standard LMS interface (course catalog, quiz editor, progress dashboards) and a real-time voice interview module with live transcript rendering and session state management. As a fully authenticated SPA with no public-facing SEO requirements, the system does not benefit from Server-Side Rendering (SSR) — making a lightweight CSR-first toolchain the optimal choice.

Decision: Vite + React + TanStack Query. aBridgeAI is a fully authenticated application — all routes require login, so SSR and SEO optimizations provided by Next.js yield no benefit. Next.js App Router introduces significant architectural complexity: React Server Components (RSC) require developers to reason about server/client boundaries

throughout the codebase, the Flight payload serialization adds overhead at every server-client crossing, and the framework has accumulated a reputation for instability across major versions [41, 42]. Vite produces a 54% smaller initial bundle (∼42KB vs ∼92KB) and delivers sub-50ms Hot Module Replacement, directly improving developer iteration speed [43, 44]. TanStack Query replaces Next.js's coarse fetch caching with fine-grained server-state management — cache invalidation, background refetching, and optimistic updates — using standard React patterns that require no framework-specific mental model [45].

5.1.3 Real-Time Communication

The AI interview feature requires low-latency voice transport, streaming STT/TTS integration, and conversational agent turn-taking. The communication layer must handle continuous audio streams without TCP head-of-line blocking.

Decision: LiveKit. The deployed architecture uses LiveKit Cloud for the WebRTC room and a separate native LiveKit Agents worker for the interview runtime. The worker combines configured STT, LLM, and TTS plugins with Silero VAD and persistent conversational context; application code retains question selection and assessment state authority. Socket.io's TCP transport causes head-of-line blocking that makes continuous voice streams unsuitable for real-time AI interviews [46].

5.1.4 Primary Database & Vector Search

The system requires ACID-compliant relational storage for user profiles, enrollment records, and session history, combined with high-throughput vector similarity search for course knowledge graph retrieval during peak concurrent usage.

Decision: PostgreSQL with pgvector. PostgreSQL with `pgvector` delivers 11.4× higher vector query throughput than Qdrant at 99% recall [47], while simultaneously providing full ACID compliance for transactional data. The ability to combine vector similarity search with SQL JOINs in a single query eliminates the need for a separate vector database service, significantly reducing operational complexity. Pinecone's managed-only model introduces vendor lock-in and prevents on-premise deployment.

5.1.5 Graph Database

The Course Knowledge Graph requires efficient multi-hop traversal to identify concept relationships, prerequisite chains, and knowledge gaps for adaptive question generation.

These traversal patterns are prohibitively expensive in purely relational models.

Decision: Neo4j. Neo4j's index-free adjacency model makes multi-hop traversals (e.g., "find all prerequisite concepts of topic X within 3 hops") orders of magnitude faster than ArangoDB's document-oriented storage layer [48]. The APOC plugin library provides ready-made graph algorithms (PageRank, community detection) that directly support knowledge gap analysis. ArangoDB's multi-model approach adds flexibility but sacrifices graph traversal performance compared to a purpose-built graph engine.

5.1.6 Caching & Task Queue

The system requires a high-speed in-memory store for JWT session tokens, API rate limiting, and background task scheduling (SR card scan, quiz generation jobs). Offloading these workloads from PostgreSQL prevents write contention during peak examination periods.

Decision: Redis. Redis uniquely serves three roles in a single service: session cache, rate limiter, and task queue (via `arq`). Memcached matches Redis on raw throughput but lacks the data structures needed for rate limiting (sorted sets) and task queuing (streams/lists). RabbitMQ is purpose-built for message queuing but requires a separate caching solution, doubling operational overhead. Redis's optional persistence (AOF) also ensures queued jobs survive container restarts [49].

5.1.7 Final Stack Summary

Table 5.1: Selected Technology Stack — aBridgeAI

Component	**Technology**	**Primary Justification**
Backend	FastAPI (Python)	Native AI/ML, async I/O, auto OpenAPI
Frontend	Vite + React + TanStack Query	Authenticated SPA — no SSR needed; 54% smaller bundle vs Next.js, sub-50ms HMR
Real-Time	LiveKit	WebRTC UDP, built-in AI Agents SDK
Primary DB	PostgreSQL + pgvector	Full ACID + 11.4× vector throughput vs Qdrant
Graph DB	Neo4j	Native multi-hop traversal for KG
Cache / Queue	Redis	Session cache + rate limit + task queue in one
Object Storage	Garage (S3-compatible)	Self-hostable, no vendor lock-in

5.2 System Architecture

The system architecture of aBridgeAI is designed as a modern, decoupled structure that clearly delineates the **system boundary**—separating internal processing components from external actors and third-party service providers. This separation ensures that the system can evolve independently of its environment while maintaining well-defined interfaces for all external interactions.

5.2.1 Actors and External Environment

Before describing the internal architecture, it is essential to identify all actors and external services that exist *outside* the system boundary and interact with it through defined interfaces.

System Actors (Users)

Five distinct user roles interact with the aBridgeAI system, each with specific access privileges and operational responsibilities:

1. **Student:** The primary end-user who accesses course materials, completes SR review sessions, takes assessments, and participates in AI-driven voice interviews. Students interact with the system exclusively through web browsers or mobile devices.

2. **Instructor:** The content creator and pedagogical authority. Instructors design courses, define learning outcomes, configure assessment parameters (e.g., SR thresholds, interview difficulty levels), and review aggregated student performance analytics including knowledge retention reports.

3. **Education Manager:** The organizational administrator responsible for provisioning the learning environment. This role creates and manages user accounts, establishes course structures, assigns instructors to courses, enrolls students, and oversees institutional-level analytics and reporting.

4. **Faculty Dean:** The academic authority over one faculty, holding the full authority of an Education Manager within that scope and, in addition, the governance decisions that belong to a faculty rather than to the organisation — authoring learning programmes and adjudicating student requests to change or drop a career pathway.

5. **IT Administrator:** The technical operations role responsible for system health monitoring, infrastructure maintenance, deployment management, and troubleshooting.

External Service Providers

The system integrates with several third-party services that reside outside the system boundary:

- **Identity Provider (Google OAuth 2.0):** Provides delegated authentication via the user's Google account. The platform issues its own session JWTs and persists second-factor enrolment (TOTP) in its internal database.

- **LLM and Embedding Provider:** Supplies inference and embedding capabilities consumed by the AI layer. The platform targets the OpenAI-compatible chat-completions and embeddings interface rather than one vendor's SDK, so the provider

is a configured base URL and model identifier per role; the models evaluated in Chapter 7 are open-weight models served through that interface.

- **Speech Services:** Configured STT and TTS providers are integrated through the LiveKit native agent pipeline. English sessions use the configured Deepgram streaming path; Vietnamese synthesis may use Cartesia or an OpenAI-compatible gateway. Provider choice remains configuration-dependent.

5.2.2 Internal System Components

The internal architecture is segregated into specialized layers within the system boundary:

- **Frontend Application (Vite + React + TanStack Query):** Serves as the primary client interface, providing a responsive SPA environment. All routes require authentication; there is no public-facing content, making SSR unnecessary. TanStack Query manages server-state with fine-grained cache invalidation and background refetching.
- **Reverse Proxy & API Gateway (Nginx):** Acts as the single ingress point into the system boundary. Handles HTTPS/TLS termination, load balancing, and routes API requests and WebSocket upgrades to the internal services. Rate limiting is additionally enforced in the backend, so a request reaching the API directly is still bounded.
- **Backend Core & Orchestration (FastAPI):** The central processing hub. Built on Python for native AI/ML ecosystem access, it handles business logic, RESTful API endpoints, async I/O via `asyncio`, SR scheduling (SM-2), knowledge retention computation, and AI pipeline orchestration.
- **Real-Time Interview Agent (LiveKit):** LiveKit Cloud carries browser WebRTC media and data streams. A separate native LiveKit Agents worker joins the room, coordinates configured STT, LLM, TTS, VAD, and a persistent chat context, then publishes audio, transcription, and control state. The model generates dialogue wording; backend policy and durable state retain authority over question selection, progression, and completion.
- **AI Layer (first-party):** A purpose-built package rather than a general orchestration framework, comprising a model gateway that routes each call by its role and

records tokens, latency and cost; an extraction, preprocessing and chunking chain; a retrieval stack implementing dense vector search, sparse full-text search, rank fusion, diversification and access filtering as separate modules; and versioned prompt templates. The decision not to adopt a chain-composition framework keeps the retrieval steps individually testable and the per-call audit record exact, which the cost telemetry and the generation-quality evaluation both depend on.

- **Primary Data Store (PostgreSQL + pgvector):** Unified relational and vector database with full ACID compliance. Stores structured data and high-dimensional vector embeddings via `pgvector`, enabling combined SQL + vector similarity queries within single transactions.
- **Graph Database (Neo4j):** Stores the Course Knowledge Graph. Enables native multi-hop traversal for concept relationship queries, prerequisite chain resolution, and knowledge gap analysis via Cypher queries and APOC algorithms.
- **Cache & Task Queue (Redis):** In-memory store serving four roles: session caching on the authenticated hot path, API rate limiting (sorted sets), background job scheduling via the `arq` worker framework for SR card scans and generation jobs, and ownership of a live quiz attempt under compare-and-set scripts. The first three degrade to the database on failure; the last deliberately fails closed.
- **Object Storage (Garage S3):** Self-hosted S3-compatible object store for course material files, uploaded documents, and media assets. Eliminates vendor lock-in while maintaining AWS S3 API compatibility.

5.2.3 System Boundary Diagram

Figure 5.1 illustrates the high-level system architecture, delineating the system boundary that separates internal components from external actors and third-party service providers.

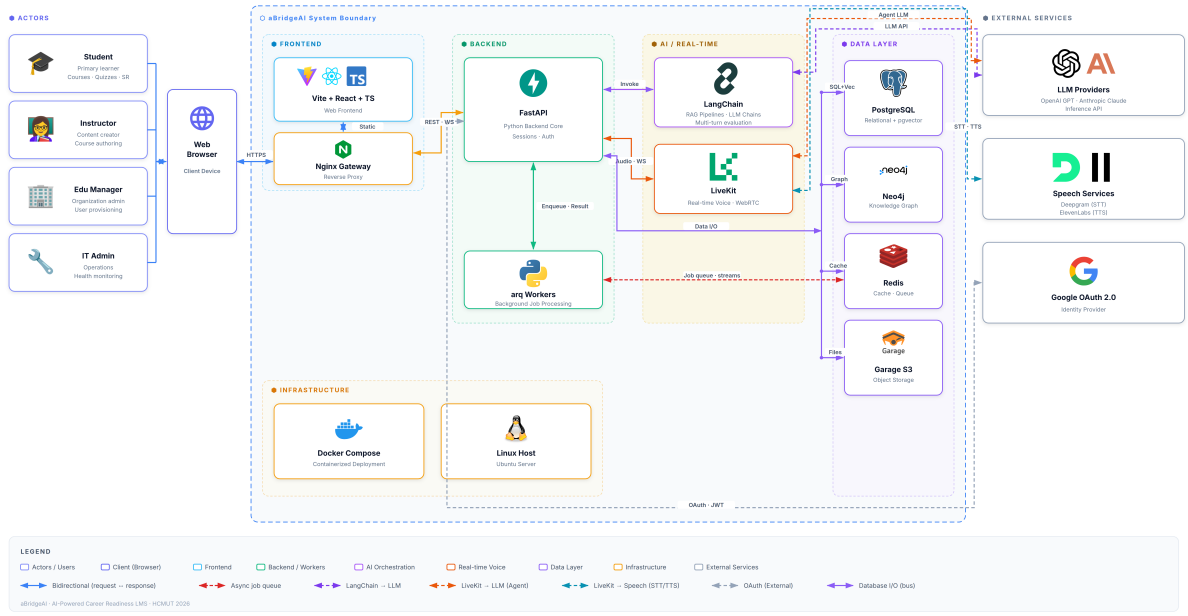


Figure 5.1: aBridgeAI System Architecture Diagram

5.2.4 Deployment Architecture

The aBridgeAI system is deployed as a set of long-running processes. Nginx serves static assets and reverse-proxies HTTP API traffic, while the LiveKit WebRTC media connection is established directly between the browser and LiveKit Cloud rather than being proxied as application media. The application runs as three supervised PM2 processes: a **FastAPI** service under Uvicorn serves lifecycle, authorization, token, and dispatch APIs; an `arq` **worker** consumes reconciliation and evaluation jobs; and a **LiveKit interview agent** runs independently because a real-time session has a lifetime measured in minutes and must survive an API deployment.

The native worker receives server-minted session metadata after a learner joins a room. With native mode enabled it initializes the conversational session and returns from branch selection; the older routed runtime remains a configuration-controlled compatibility path. FastAPI is therefore a control-plane service for the interview, not a media intermediary. PostgreSQL remains the authoritative store for session state, transcript, and assessment data, while Redis/`arq` carries deferred reconciliation and final evaluation.

The backing services — **PostgreSQL with pgvector** for relational and vector data, **Neo4j** for the Course Knowledge Graph, **Redis** for caching, rate limiting, queueing and attempt ownership, and **Garage** as a self-hosted S3-compatible object store — are con-

tainerised. Clients transfer material to and from the object store through presigned URLs issued by the API rather than streaming bytes through it, so upload size is bounded by storage policy rather than by request timeouts.

The AI layer is first-party code calling OpenAI-compatible model and configured speech-provider interfaces over HTTP, with no chain-orchestration framework between them. This architecture separates routing, application policy, durable storage, background processing, and real-time delivery, making each boundary independently deployable and testable.

5.3 Algorithm Design

Two algorithmic decisions shape what a learner may do next: when a card is scheduled for review, and when a stage of a career pathway opens. The first is the hybrid SM-2 model, whose full specification — the derivation of Q from objective response signals, the calibration weight, the interval rules, and the learner-facing retention metrics built on them — is given in Appendix B. It is placed there because this iteration does not evaluate it: a scheduling policy's effect on retention can only be established over a cohort observed across months, which is outside the scope of the present work. The pathway rules below are specified here because they are not a research claim but a piece of academic bookkeeping the system must get exactly right, and because every progress figure the product reports is derived from them.

5.3.1 Career Pathway Progress and Completion

The scheduling model governs movement *within* a course. A career pathway is the structure above it: an ordered sequence of stages, each bundling required courses with a set of electives from which the stage asks for a stated number. Progress along a pathway, and the decision that a student has finished one, are therefore computed over stages rather than over cards.

Stage Satisfaction and the Elective Quota

A course is *satisfied* when the student holds a completion award for it. Partial lesson progress does not satisfy a course: a course is either awarded or it is not, and progress short of the award counts for nothing at this level.

Each stage of the pathway version the student is pinned to therefore *demands* two things — all of its required courses, and a stated number of its electives, which is the

stage's quota. Against that, the stage is *credited* with every required course the student has satisfied, plus the electives they have satisfied, counted only as far as the quota.

Stopping the elective count at the quota is the whole of the rule. A stage offering five electives and asking for one is finished by the first of them, and the remaining four are never owed. This is a deliberate departure from treating every attached course as outstanding work: an elective set exists to offer a choice, and a design under which choosing one obliged the student to the other four would make that choice meaningless. Because the elective count is capped at the quota, a stage can never be credited with more than it demands.

Pathway Progress

Progress along the pathway is the share of demanded work that has been credited, totalled across every stage and expressed as a percentage.

Stages that demand nothing — no required courses and a quota of zero — are left out of both halves of that comparison. Such a stage has nothing outstanding and nothing to measure; counting it as unmet work would hold a finished pathway permanently below one hundred per cent.

Deciding That a Pathway Is Finished

Whether a student has finished a pathway is decided from the stages themselves: the pathway is complete when it has at least one stage and every one of those stages has met its demand in full. It is not read off the progress percentage.

Because a stage can never be credited beyond what it demands, the two presentations agree exactly — the percentage reaches one hundred precisely when every stage is complete. Keeping them as one rule under two presentations is what prevents a pathway reading as finished on screen while the records built on completion remain open.

The requirement of at least one stage is deliberate. A pathway with no stages has not been shown to be finished; it has merely offered nothing to examine, and what completion writes is a durable academic record. Publication requires at least one stage per pathway and at least one course per stage, so the case does not arise for any pathway a student can be enrolled onto.

Stage Unlocking

Completion says what a student has finished; unlocking says what they may open. The two are computed from the same evaluation but answer different questions, and a pathway needs both: a stage may be finished without ever having been reachable, and the design turns on recognising that as a coherent state rather than a contradiction.

Each stage carries an unlock policy, and there are three:

- `always` — the stage is open regardless of what precedes it.
- `after_previous` — the stage opens once the stage before it is complete, quota included, and was itself unlocked.
- `after_previous_required` — the stage opens once the required courses of the stage before it are satisfied and that stage was itself unlocked, letting its electives run on unfinished.

Two rules sit above the policies. The first stage of a pathway is unconditionally open whatever its stored policy says, because a pathway whose opening stage were locked could not be started by anyone. And a policy the system does not recognise resolves to locked rather than open, so a malformed rule withholds content instead of releasing it.

The difference between the two gated policies is exactly the elective quota: one waits for the previous stage to be finished outright, the other releases the next stage as soon as the previous stage's required work is done.

Why Unlocking Chains

Requiring the previous stage to have been *unlocked*, and not merely complete, is the part that is easy to leave out and costly to leave out. Satisfaction is a property of a student and a course, with no pathway in it, so a stage's courses can be satisfied by work done somewhere else entirely. Where a programme permits several pathways at once, a second pathway whose course set mirrors one stage of the first will complete that stage in the first the moment it is joined — correctly, because the work was genuinely done, and it counts toward that pathway and is recorded as permanently earned.

It must not, however, open what follows it. Without the conjunction, a stage the student had never been permitted to enter would serve as the prerequisite pass for the stage after it: a curriculum whose first stage is a hard requirement could be entered at its third stage without the first ever being touched, by taking a second pathway that mirrors the second stage. Requiring the previous stage to have been reachable as well as finished closes that route, and does so without weakening the credit: the mirrored stage remains complete, remains counted, and remains permanently recorded.

What this admits is a stage that is complete and locked at the same time. That pair is not a contradiction but the precise description of the situation — the work is done and will never have to be done again, while the pathway's own sequence has not yet reached it.

An `always` stage is unaffected throughout, being unlocked by its own policy; it propagates the chain rather than interrupting it, so an unfinished stage of pure electives can never shut the pathway behind it.

Unlocking governs presentation and access rather than eligibility to progress. A locked stage blocks a student from starting its courses only where its author marked the stage as hard-enforced; the softer settings record the lock, warn the student, and let them through, because an author who chose to advise rather than to block should not be silently overruled.

One Evaluation, Every Surface

The same evaluation answers the question wherever it is asked: on the learner's own pathway view, on the manager's roster of every student following a pathway, and in the frozen snapshot written when a student leaves one. None of these recomputes progress for itself.

This is a stronger requirement than it appears, because the three surfaces have different shapes and invite different shortcuts. A roster reports many students at once, and a single aggregate query is the obvious way to produce it: each student's percentage, and the count of courses behind it, derived in one statement from the mean lesson completion of every attached course. Such a figure is reasonable in isolation and wrong in company. A course counts as satisfied when the student has been awarded its completion, which is not the same condition as its lessons averaging 100, so a roster built that way can report a student at 100% against a course count that says otherwise, in the same row.

The correction is not a better query but one fewer. The roster's query now returns identity and avatar columns only; the percentage, the number of satisfied courses and the denominator are all read from the stage evaluation, which is the sole definition any of them has. A second derivation of a value that already has one authority does not add a check — it adds a disagreement that nothing is watching for.

Evaluating per student is what this costs, since satisfaction is per student by nature. The stage structure, however, is a property of the pathway version and not of the reader, so it is resolved once for the whole roster rather than re-read for each row.

Programme Completion

Completion propagates upward but never sideways. Finishing a pathway completes the attempt that held it, and the programme enrolment behind that attempt completes only once none of its attempts remain active — so a student holding two pathways completes the programme on the second, not the first. An attempt ended by an approved change

or drop is closed rather than active, and therefore does not hold the enrolment open. Each attempt is judged against the pathway version pinned to it, so a revision published mid-enrolment cannot alter what an already-enrolled student must finish.

The evaluation in one pass

Figure B.6 in Appendix B traces the rules above as the evaluator applies them: the ordered stage walk, the quota arithmetic, the latch that preserves a completion already earned, and the forward dependency by which each stage's unlock reads the stage before it.

5.4 API Design

The backend is a Python FastAPI service exposing a versioned RESTful API at `/api/v1`, and it is the single data contract between the single-page application and the server. This section states the decisions that shape that contract; the full specification — the authentication and MFA flows, the error model, pagination and idempotency, and the complete endpoint catalogue with its Swagger screenshots — is given in Appendix E (Volume II).

Vertical slices, audience-scoped routers

Each feature owns a self-contained package with its own routers, services, queries and schemas, and routers translate HTTP to application calls without touching persistence directly — a boundary enforced by an import-linter contract in continuous integration rather than by review. Within a feature, routers are split by *audience* rather than by entity: a learner router under `/me/...`, an authoring router under `/teacher/...`, and where the domain needs them, administration, department-assignment and management routers under their own prefixes. The access context is therefore legible from the URL alone, which is what allows the permission guard on a route to be read against the route's own path rather than inferred from its handler.

Asynchronous by default, with long work moved off the request

Every router function and database session is asynchronous, and work that cannot complete in a request — material ingestion, quiz and interview generation, material reprocessing — is dispatched to a worker pool over Redis rather than executed inline. The HTTP layer therefore returns within hundreds of milliseconds even for AI-driven mutations, and the client follows progress by polling the generation run the call returns. This

is the decision that keeps an AI platform's API latency governed by its database rather than by its slowest model provider.

Uniform contract mechanics

Three conventions are applied across every feature rather than chosen per endpoint. List endpoints use cursor pagination over a single indexed column, which avoids the cost of deep offsets and tolerates inserts between fetches without duplicating or skipping rows. Errors are returned in one structured envelope so the client has a single failure shape to handle. Mutating endpoints that a client may retry accept an idempotency key and deduplicate against it, which matters most where a retry would otherwise create a second quiz attempt or a second generation run.

Session verification on the hot path

Because every authenticated request must confirm that its session is still live, non-revoked and MFA-cleared, session state is cached in Redis and read through to the database on a miss, so a cache outage costs latency rather than availability. A live quiz attempt carries a second, deliberately short-lived key binding it to one client; that path is the single deliberate exception to the fallback policy, failing closed rather than open, because admitting two concurrent clients to a graded attempt is worse than refusing the write. Both are described in full in Appendix E (Volume II).

5.5 Class Diagrams

The class diagram captures the static structure of the backend: the core domain entities, their attributes, and the relationships between them. The model is organised along Domain-Driven Design lines into four packages — *Identity & Access*, *Course & Ingestion*, *Assessment & Interview*, and *Programme & Path Governance*. The four paragraphs below state the design decision that shapes each package; the full entity listing, with attributes, cardinalities and the rationale behind each relationship, is catalogued in Appendix F.

- **Identity & Access.** Authorisation is **role-based, not inheritance-based**. There is a single concrete `User` entity; a person's capabilities come from rows in `user_role_assignments` that bind them to a `Role` at a declared scope (global, organisation, org-unit or course), and each role carries a set of `Permission` codes. Modelling "student" and "teacher" as subclasses was rejected because the same account is routinely both — a teaching assistant enrolled on another course — and

because a scoped assignment expresses "instructor *of this course*", which a subclass cannot.

- **Course & Ingestion.** A `Course` composes `LearningMaterial`s, each of which is versioned: a `LearningMaterialVersion` references bytes held in object storage through `storage_object_id`, and the database stores that reference rather than the file. Each version decomposes into `DocumentChunk`s carrying a 3072-dimensional embedding, which is what the retrieval pipeline searches.

- **Assessment & Interview.** Maps the path from theory validation (`QuizAttempt`) to practical application (`InterviewSession`), and how the per-question record of an interview is aggregated into a personalised `GapReport`.

- **Programme & Path Governance.** A student's career pathway is held as an **attempt record, not as a field on the enrolment**. `ProgramEnrollment` owns many `ProgramPathAttempt`s, each pinned to a career-path *version* and carrying how it began — assigned from the programme default, chosen by the student, or produced by an approved change. A single field could not express a student holding two pathways at once, nor survive one ending, and the closed attempt is what the audit is written against. `PathChangeRequest` makes leaving a pathway reviewable: one open request per enrolment, discriminated by `kind` into a change or a drop.

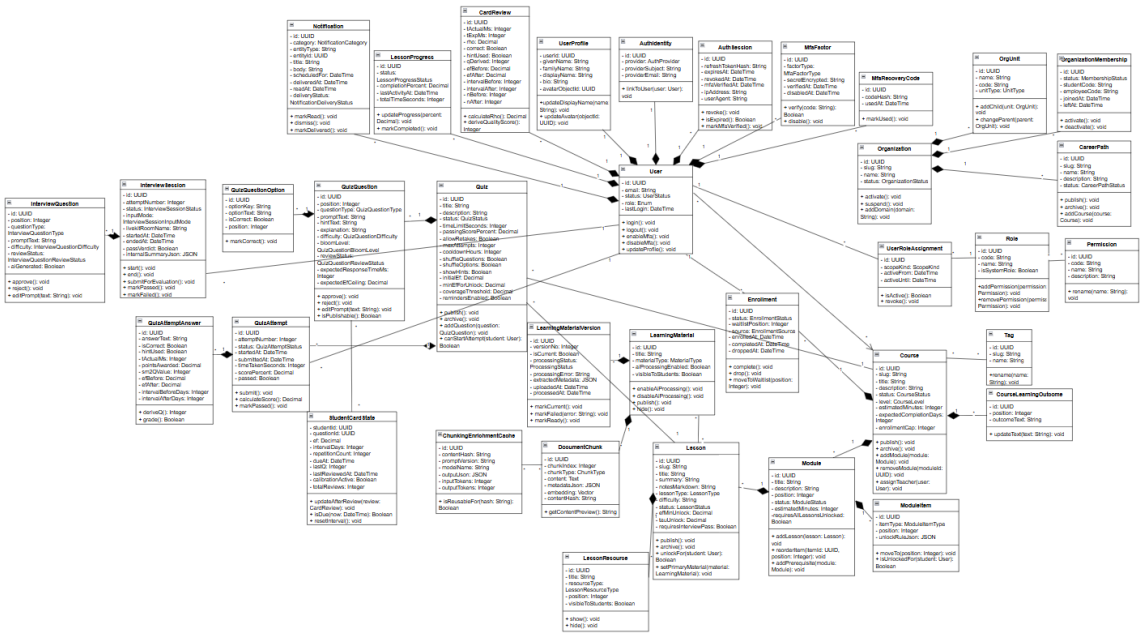


Figure 5.2: UML Class Diagram representing the core Domain Entities of aBridgeAI

5.6 Sequence Diagrams

Sequence diagrams illustrate the time-based interactions between system components, including the client application, backend services, AI engines, and database infrastructure.

The sequence diagrams focus on the core workflows of the aBridgeAI platform, including authentication, smart content ingestion, and Human-in-the-Loop quiz generation. These diagrams emphasize asynchronous processing, AI integration, and communication across the hybrid system architecture.

Reproducing a representative subset here would duplicate the appendix without adding to the argument, since each diagram is only legible at full width and each carries the same weight as the others. The retained rendered diagrams — authentication, smart content ingestion, and Human-in-the-Loop quiz generation — are presented in Appendix E. The mock-interview raster sequence was replaced with a code-backed native textual trace in that appendix because its former routed/Qdrant narrative no longer represents the deployed runtime; Section 6.8 provides the full implementation detail.

5.7 Database Design

This section presents the Entity-Relationship Diagram of the main database — the structure every feature reads and writes. The logical and physical designs derived from it, which restate the same structure at the level of relations and of storage, are given in Appendix C.

5.7.1 Entity-Relationship Diagram

The schema is presented first as an overall map, then through focused diagrams for the quiz, interview, and remaining cross-domain entities. Splitting the ERD makes the primary keys, foreign keys, and domain-specific relationships readable without losing the overall context.

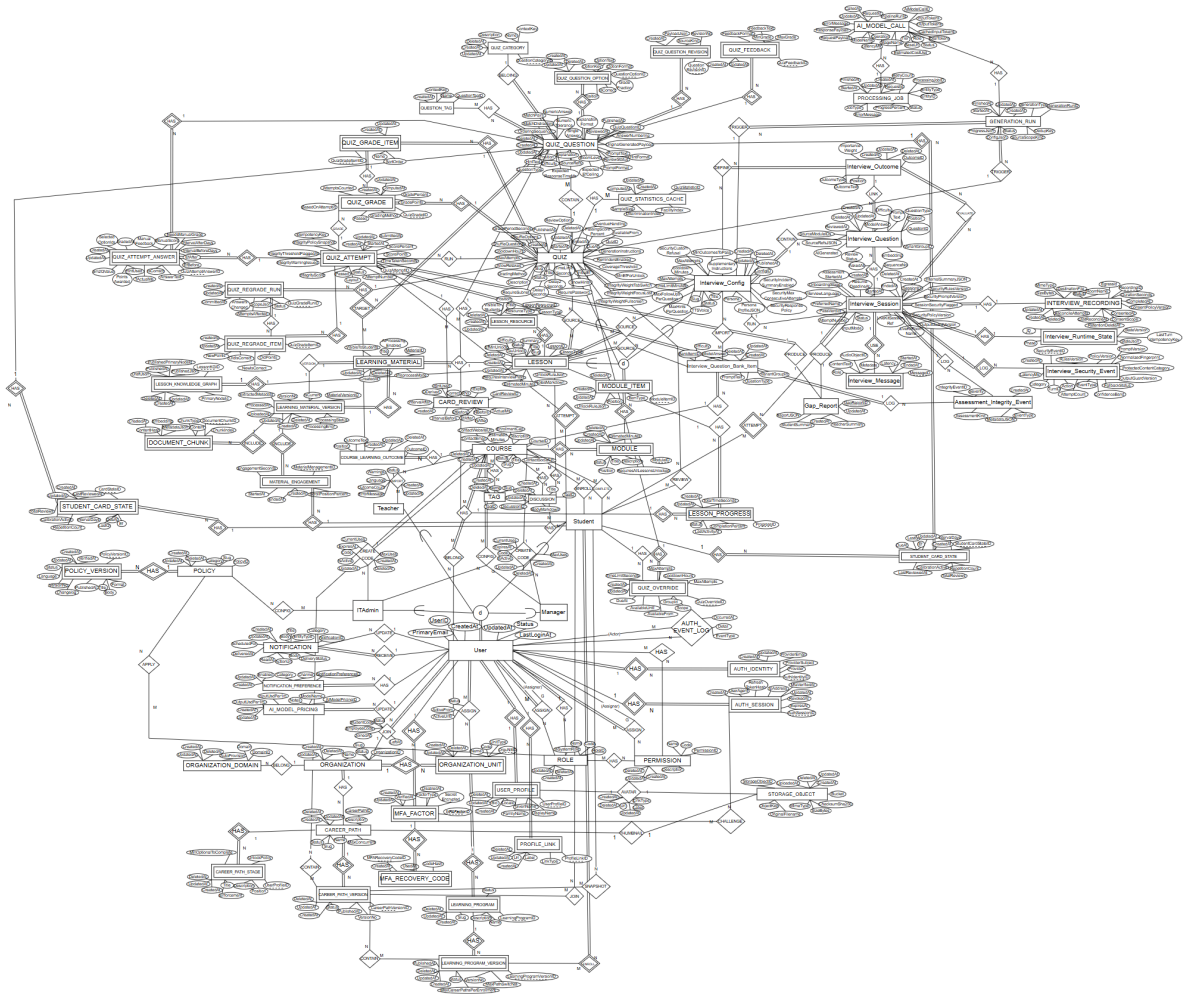


Figure 5.3: Overall Entity-Relationship Diagram

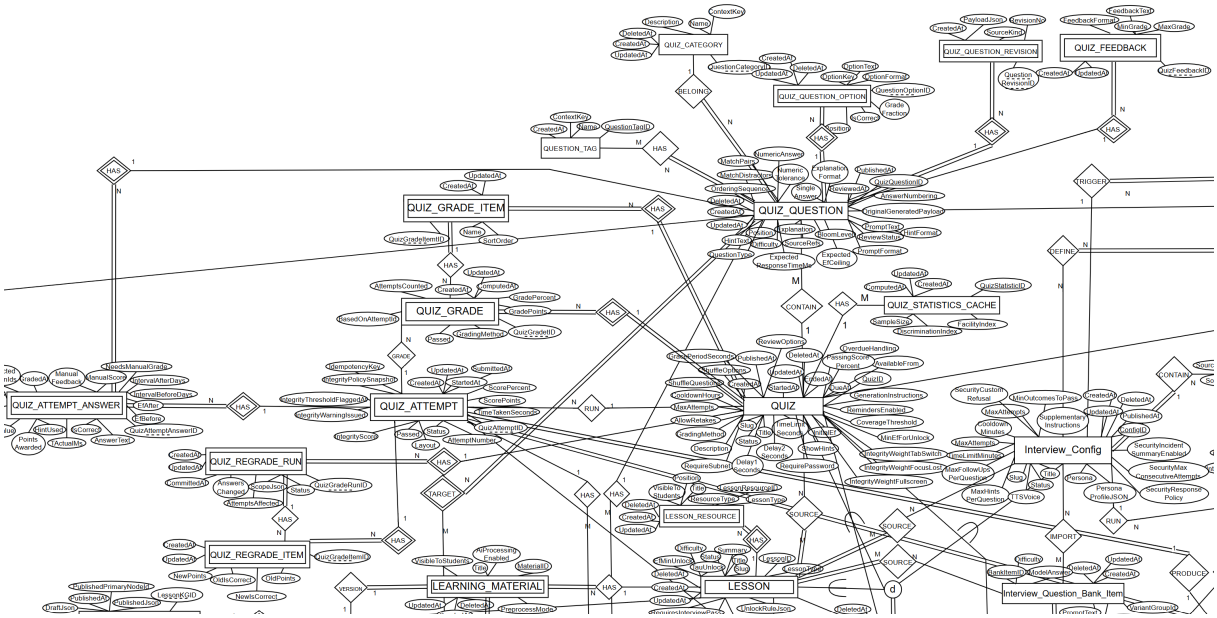


Figure 5.4: Entity-Relationship Diagram: Quiz, Attempts, Grading, and Spaced Repetition

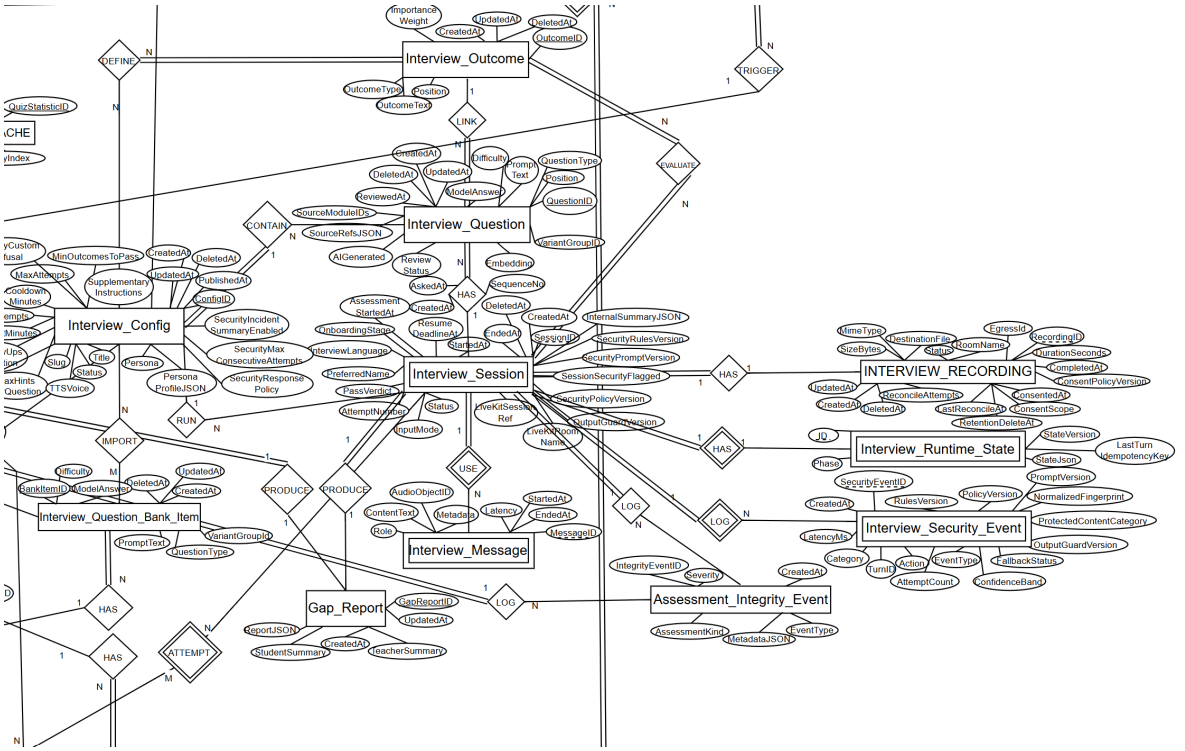


Figure 5.5: Entity-Relationship Diagram: Interview Configuration, Sessions, Evaluation, Integrity, and Recording

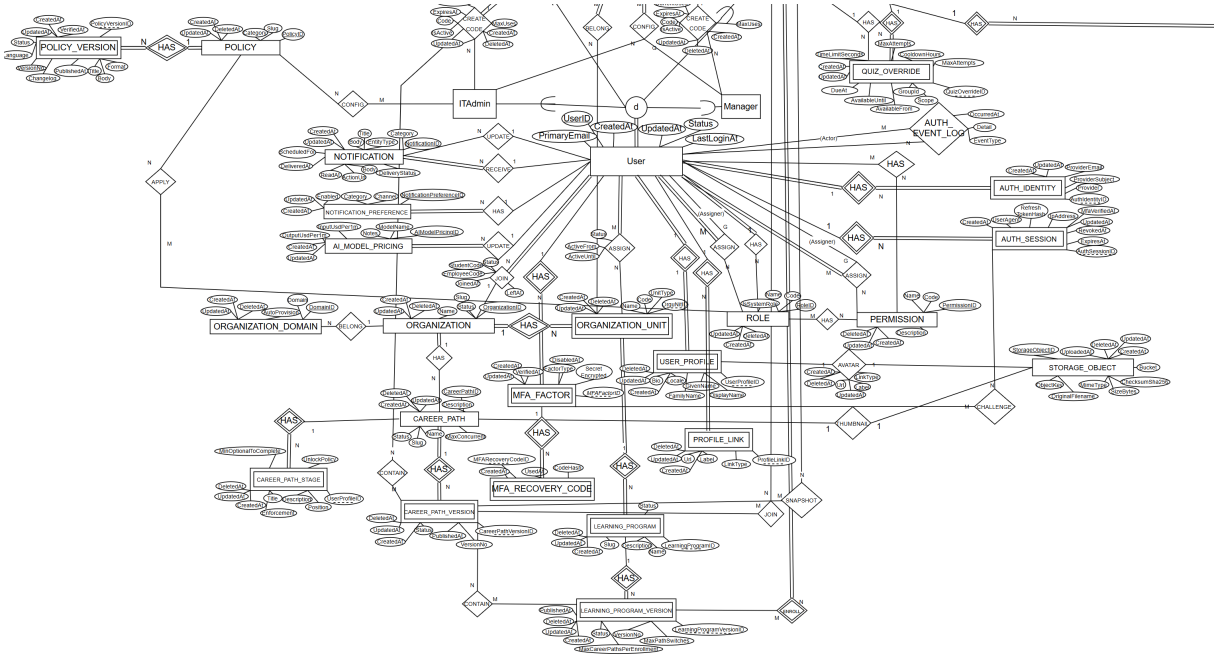


Figure 5.6: Entity-Relationship Diagram: Remaining Cross-Domain Entities

5.8 UI/UX Design

5.8.1 Design Philosophy

aBridgeAI adopts a **light-first, academic** design language inspired by enterprise dashboards (Metronic Demo 6) — dense rows, crisp 1 px borders, restrained 4–10 px corner radii. The interface deliberately reads as a serious learning tool rather than a consumer product. Four guiding principles shape every screen:

1. **Role-based clarity** — students, instructors, and administrators see distinct navigation structures and role-appropriate destinations; the same shell component renders all three contexts from a per-role `navItems` prop.

2. **Progressive disclosure** — AI features (quiz generation, processing queue, cost telemetry) are surfaced contextually inside their host screens, only when the user has the required role and prerequisites.

3. **Accessibility-first typography** — body copy uses *Be Vietnam Pro*, a humanist sans-serif with full Vietnamese diacritic support, paired with a 14 px base size and 1.5 line-height for sustained reading.

4. **Responsive shell architecture** — a persistent `AppShell` renders a desktop left rail that can collapse from 256 px (`w-64`) to 64 px (`w-16`) by user toggle. On narrow screens the rail is replaced by an off-canvas navigation menu opened from the mobile top bar.

The implementation uses **shadcn/ui** components on top of **Tailwind CSS v4**; the colour system is defined as CSS custom properties (`var(--primary)`, `var(--surface)`, ...) so that the light and dark themes share a single component layer with full parity.

Internationalisation (i18n).

Every authenticated page exposes a globe icon in the `ContentTopBar` that opens a language selector; the active language is shown next to the icon (*e.g.* "EN"). Locale strings live in `src/i18n/locales/<lang>.json` — English and Vietnamese — with English as the fallback, and a test asserts that the two catalogues carry the same key set, so a translation added on one side but not the other fails the build rather than degrading silently at runtime. Navigation labels and form fields reference an `i18nKey` (such as `nav.dashboard`, `nav.career_paths`) defined in `src/lib/navigation.ts`, so adding a new locale requires only a JSON dictionary — no component changes. The platform was developed with English and Vietnamese as first-class locales, matching the choice of Be Vietnam Pro as the primary typeface.

5.8.2 Design Tokens

Table 5.2 consolidates the design tokens that govern the visual system. Full per-rule rationale and the complete spacing, layering, and interaction rules are documented in Appendix G (Volume II).

Table 5.2: Core design tokens.

Token	Value	Usage
Page background	`#f8fafc` (slate-50)	Page surface in light mode.
Surface / Card	`#ffffff`	Cards, panels, dialogs.
Primary accent	`#1e40af` (blue-800)	Primary actions, active nav, links, chart series 1.
Secondary accent	`#3b82f6` (blue-500)	Secondary buttons, focus rings, chart series 2.

Table 5.2 *(continued)*

Token	**Value**	**Usage**
CTA / accent	`#f59e0b` (amber-500)	High-attention CTAs and AI affordances.
Success	`#16a34a` (green-600)	Published / completed states.
Warning	`#ea580c` (orange-600)	Drafts, pending review.
Danger	`#dc2626` (red-600)	Failed jobs, destructive actions.
Border	`#e2e8f0` (slate-200)	1 px borders on cards, inputs, tables.
Typography	Be Vietnam Pro (system-ui fallback)	All body and UI text; full Vietnamese diacritic support.
Component library	shadcn/ui + Radix UI + Lucide React	Dialogs, dropdowns, sheets, tooltips, icons.
Charts	Recharts	Bar, line, area; reuses palette for chart series 1–5.
i18n keys	`src/lib/navigation.ts` + `src/i18n/locales/<lang>.json`	Translatable nav labels and copy.
Sidebar widths	64 px collapsed / 256 px expanded	`w-16` / `w-64` on desktop; narrow screens use the off-canvas mobile navigation menu.
Border radius (card)	8 px (`rounded-lg`)	10 px hard cap project-wide.
Base spacing unit	4 px	All spacing is a multiple of 4 px.
Max modal depth	1 stacked modal	Prevents nested-modal anti-patterns.

5.8.3 Key Screens

This subsection sketches the principal screens by role. Per-screen layouts and interaction details are illustrated in Appendix G (Volume II).

Public surfaces.

The **Landing Page** (`/`) is the marketing entry point; it carries a hero, stats strip, category grid, and featured-courses carousel under a dedicated `TopNavBar` that is unique to this route. The **Login** screen (`/login`) offers Google SSO as the single sign-in path; on success, accounts with MFA enabled are routed to `/login/mfa` for a TOTP or recovery-code challenge before a session is issued. A `next` query parameter preserves deep links across the auth round-trip.

Student.

The **Student Dashboard** (`/dashboard`) keeps the layout calm: stat cards summarise enrolled courses, in-progress lessons, and recently earned milestones; a recommendations strip suggests the next course drawn from the career paths the student currently holds. The **Course Catalog** (`/courses`) opens with a sticky search and filter bar (query, category, difficulty); enrolled courses display an inline progress ring so students can pick up where they left off. The **Course View** (`/courses/$slug/learn`) splits into a sticky curriculum accordion on the left and the active lesson's content on the right (video, reading, attached resources, embedded quiz launcher). The **Quiz Session** (`/courses/$slug/quiz/$quizId`) is a focused, distraction-free interface with a countdown timer and a question-navigation grid; on submission students land on the attempt-review screen with correct answers and per-question explanations.

Instructor.

The **Teacher Dashboard** (`/teacher`) summarises authoring activity (total / published / draft / AI-enabled) and surfaces the most recently edited course with a one-click jump into management. The **Course Manage** screen (`/teacher/courses/$courseId`) is the authoring core: modules render as drag-handle cards, expanding to reveal ordered lesson and quiz items, each with a publish-state badge and inline **+ Video** / **+ Reading** / **+ Quiz** quick-add buttons. The **Lesson Editor** (`/teacher/courses/$courseId/lessons/$lessonId`) splits into a Markdown content area with attached resources on the left and a settings sidebar (Visibility, Lesson Type, Duration, Difficulty, Prerequisites, AI Material Hub) on the right.

The **Quiz Editor** (`/teacher/courses/$courseId/quizzes/$quizId`) is structured around three sub-tabs — *Questions*, *Settings*, and *Preview*. The Questions tab lists every item with its type badge, per-question time limit, and approval status; bulk-action controls let the instructor apply uniform timing or bulk-approve. A persistent banner reports

how many questions are pending review, enforcing a deliberate approval gate between AI generation and student exposure. An **Open generator** button launches a modal that drives the AI quiz pipeline: source-lesson selection, question count, difficulty band, question types (multiple choice, true/false, short answer, fill in the blank), generation mode (*Topic* balanced spread vs. *Coverage* guaranteeing per-section coverage), and an *Append* or *Replace* policy for existing questions. Generated items land in *Pending Review* state and are visible to students only after explicit instructor approval.

Assessment surfaces.

Two screens exist to hold a learner's full attention and are deliberately stripped of the surrounding shell. The **Quiz Session** runs a countdown against the server's authoritative clock, so closing the tab neither pauses nor forfeits the attempt; where the quiz requires a camera the workspace blocks while no live video track is present, after a short grace period that absorbs a momentary device glitch. Because an attempt admits one client at a time, opening it a second time presents a conflict screen offering an explicit takeover rather than silently splitting the session in two. The **Interview Session** begins through the course interview workspace. It requires staged identity, audio, language, preparation, and readiness checks before assessed questioning and timer start. A native LiveKit room is dispatched after readiness in the target deployment; if microphone permission or voice availability is absent, the same hybrid workspace retains typed input. Results present the learner-facing verdict and qualitative remediation after asynchronous evaluation rather than exposing assessment internals.

Progression surfaces.

The **Study** screens (`/me/study`, with `cards-due` and `review`) drive the spaced-repetition loop, presenting what is due rather than asking the learner to choose. **Progress** (`/me/progress`), **Career Paths** (`/me/career-paths`) and **Learning Programmes** (`/me/learning-programs`) expose the learner's standing at successively wider scope, the last of these carrying pathway selection and the change-request form whose outcome a Faculty Dean decides.

Manager and Dean.

The management surface (`/management/...`) is where institutional structure is authored: learning programmes and their versions, career paths, organisational units, course enrolments, and user administration, all scoped to the operator's own organisation. The programme detail screen carries the dean's review queue, where pathway change and drop

requests are acknowledged and then approved or rejected against a fixed reason vocabulary.

Administrator.

The **Operations** page (`/admin/operations`) gives platform operators visibility into the AI background-job pipeline — material ingestion, embedding, quiz generation, validation — with stat cards reporting the backlog, status-pill filters narrowing the table, and a per-row **Retry** on failed jobs. The **AI Costs** page (`/admin/ai-costs`) tracks language-model and embedding spend across a selectable range, broken down by role and by pipeline stage so that cost can be correlated with backlog health without a context switch. Beyond these sit **Organisations**, **Users**, **Audit Logs**, **Policies** and **System Configuration** (`/admin/settings`), the last presenting each runtime setting with its effective value, the level that supplied it, and a dry-run preview before any change is applied.

5.8.4 Navigation Architecture

The authenticated experience is built around a single role-agnostic `AppShell` component that orchestrates a fixed-position `SideNavBar` and a sticky `ContentTopBar` around the main content. The shell receives a `navItems` array as a prop (defined in `src/lib/navigation.ts`), so the same layout code serves the student, teacher, and admin route groups; role differentiation is communicated through the `navItems` configuration and shared design tokens. On desktop the sidebar can collapse to an icon rail; on narrow screens it is replaced by a mobile top-bar menu, keeping the main column readable without permanently consuming horizontal space.

The `TopNavBar` and `Footer` components are reserved exclusively for the landing page, ensuring a clean visual handoff between the marketing surface and the authenticated product.

5.9 Site Map

The application is a single-page web client built with **Vite** and **TanStack Router**, served as a static bundle from a CDN and consuming the FastAPI backend over HTTPS. All authenticated routes are nested under a single `authenticatedRoute` parent that enforces a session check and an MFA gate before any child route is allowed to mount; deep-linked routes round-trip through `/login` with a `next` query parameter so users land

back on the originally requested URL after authentication.

Figure 5.7 summarises the full route tree, organised by the six bands the platform exposes: *Public*, *Any signed-in user*, *Student*, *Instructor*, *Educational Manager*, and *Administrator*. The router declares 97 routes in total: one pathless authenticated layout, 80 that render a destination page, and 16 that are redirect-only aliases retained so that previously published links keep resolving. The 80 destinations occupy 79 distinct URLs, because the instructor course workspace declares a layout route and an index child at the same path. The complete per-route URL listing — with page name, dynamic parameters and purpose — is provided in Appendix F (Volume II).

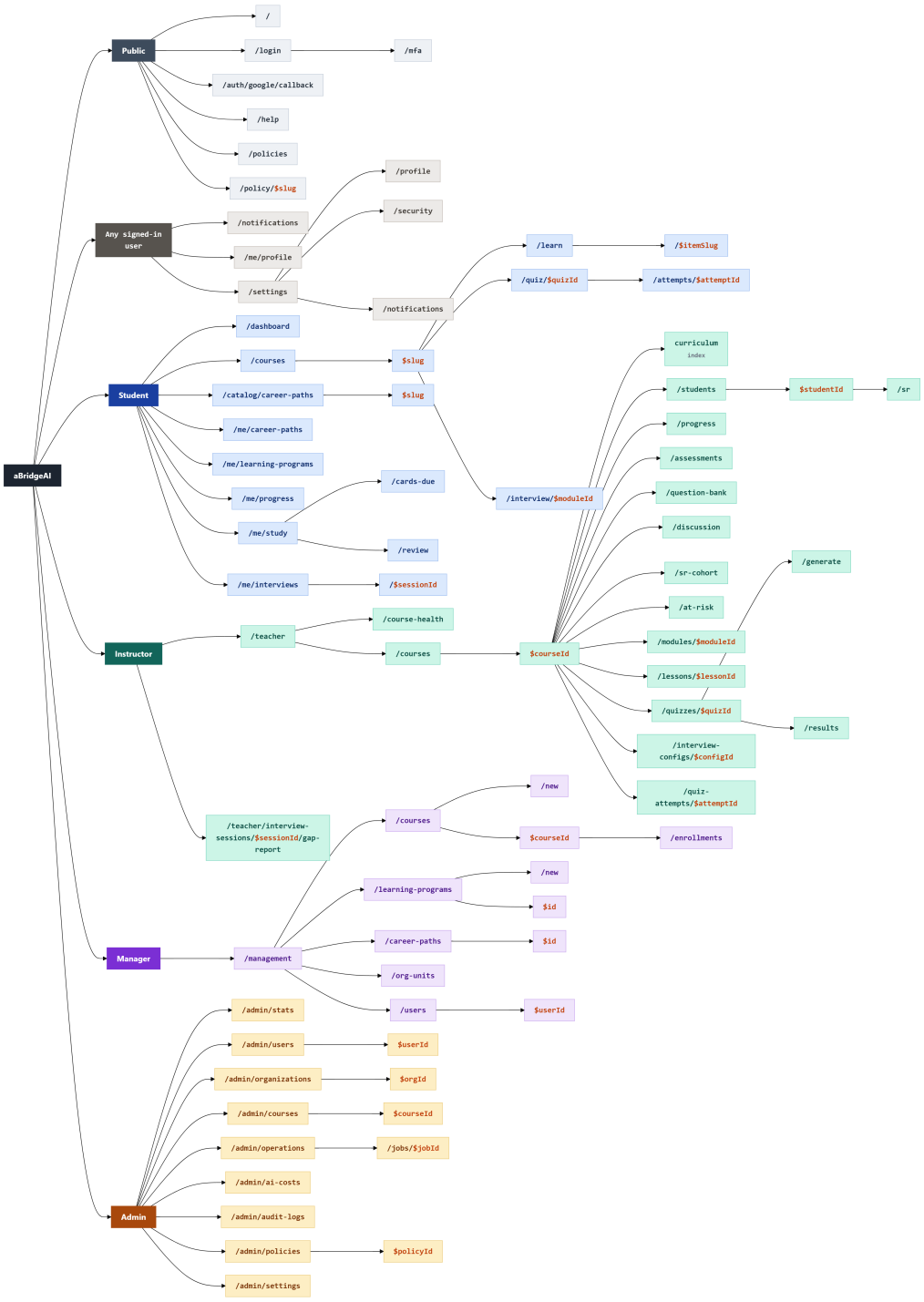


Figure 5.7: Application sitemap — colour-coded by band: cool grey (public), warm grey (any signed-in user), blue (student), teal (instructor), violet (educational manager), amber (administrator). Dynamic URL segments are highlighted in amber as `$param`. Every node is one destination route; the 16 redirect-only aliases are omitted here and listed in Table F.8.

The sitemap is intentionally flat within each role group: there are no role-specific subdomains, only path prefixes, so a single deployment serves every persona. Three prefixes are privileged — `/admin`, `/teacher` and `/management` — and each maps to exactly one permission set, so the section a URL belongs to and the permission it demands are the same fact written once. A band names the persona a route is designed for, not the permission it demands, and the two are deliberately not the same partition. The notification inbox, the account settings pages, and the reader's own profile hang off the top bar, which the application shell renders for every authenticated role, so they belong to no single persona and are drawn in a band of their own. The remaining learner routes are equally ungated — a teacher who types `/me/study` reaches it — and sit under *Student* only because that is the sidebar which offers them. The learner surfaces carry two unprivileged prefixes of their own: `/me` for everything scoped to the signed-in user, and `/catalog` for the organisation-wide browsable inventory, which keeps *my* career paths distinct from *the* career paths. Role differentiation is enforced by the `beforeLoad` guard on the parent route, a prefix-to-permission check in the authenticated layout, and the role-specific navigation groups driving the sidebar. This keeps the bundle small (one client-side router for the whole platform) and makes it cheap to add new role-scoped routes — a new admin page is one new `createRoute` call that joins the `authenticatedRoute` children list.

5.10 Test Case Design

The main text reports representative coverage only; the complete test-case specification, including inputs, expected results, and exception conditions, is provided in Appendix H (Volume II).

Table 5.3: Representative test coverage

Area	**Representative coverage**	**Evidence of the expected behaviour**
Authentication and access control	Google SSO, MFA verification, and permission-protected routes	Successful authentication grants access; invalid second factors and insufficient permissions are rejected.
Learning loop and adaptive quiz	Quiz configuration, response recording, and SM-2 review updates	Published settings reach student attempts, response evidence is persisted, and the derived card state is updated.
Application loop and mock interview	Outcome-based interview configuration, adaptive turn-taking, and session completion	Eligible students can complete an interview and receive a persisted transcript, metadata, and verdict.

These representatives demonstrate the three principal test levels used by the system: system behaviour at user-facing boundaries, integration behaviour across feature boundaries, and persistence of the resulting learning evidence. The appendix maps the individual cases to their functional requirements.

6 Implementation and Testing

This chapter describes the concrete implementation of aBridgeAI, covering the development environment, project structure, and the realisation of each major subsystem on both the server and the client. The backend follows a feature-first architecture in which each domain — identity, access control, courses, materials, quizzes, interviews, spaced repetition, learning programmes, notifications and discussions, configuration and policies — is a self-contained package exposing its own models, queries, services, and API routers, with cross-feature calls confined to a declared public surface. Cross-cutting concerns — caching, audit logging, observability, and access control — are isolated in a shared `core` layer consumed by all features. Identity and authentication, and notifications and discussions, are built entirely to established patterns and are therefore documented in Appendix G rather than here.

The sections that follow are ordered as the system is built up rather than as the packages happen to be named: platform foundations first, then the curriculum and the material that fills it, then the AI pipelines and the two assessment modes they feed, then the learning loop and the governance that sits above it, and finally the cross-cutting services, the client, and testing. Each subsystem states its design decision and the shape of its solution here; the full lifecycle vocabulary the system enforces, the two subsystems built to established patterns, and the per-case test detail are catalogued in Appendix G.

6.1 Development Environment and Toolchain

The backend is implemented in Python 3.12 using the FastAPI framework for asynchronous HTTP serving. Dependency management is handled by `uv`, which resolves and locks the full dependency graph in `uv.lock` to guarantee reproducible builds across developer machines and CI. The project declares its metadata and tool configuration in a single `pyproject.toml`, covering package discovery, mypy type-checking settings, bandit security rules, and ruff linting profiles.

Runtime dependencies

The core runtime stack consists of:

- **FastAPI + Uvicorn** — ASGI web framework and server.
- **SQLAlchemy 2 (async)** — ORM and query layer over PostgreSQL via the `psycopg` driver.

- **Alembic** — schema migration management; 16 versioned migration files track every DDL change from the baseline schema to the current state.
- **Redis (via `arq`)** — task queue for background workers and cron jobs.
- **Neo4j (async driver)** — knowledge graph storage for concept-relationship extraction.
- **Pydantic v2** — request/response validation and settings management via `pydantic-settings`.
- **tiktoken** — token counting for chunking budget enforcement.
- **Jinja2** — prompt template rendering for all LLM calls.

Code quality gates

A `.pre-commit-config.yaml` wires 13 hooks that run before every commit: `ruff` for linting and formatting, `mypy` for static type checking, `bandit` for security scanning, and `gitleaks` for secret detection. The same hooks are replicated in the GitHub Actions CI pipeline (`.github/workflows/backend-new-ci.yml`) so violations are caught both locally and on every pull request.

Local infrastructure

A `docker-compose.yml` defines local backing services: PostgreSQL 16 with the `pgvector` extension, Redis 7, Garage object storage, and Neo4j 5. Developers run `docker compose up -d` before starting the application with `uv run uvicorn`. This Compose file is development infrastructure, not the target production topology. The latter separates FastAPI, `arq`, and the configuration-controlled native LiveKit agent process; browser media connects directly to LiveKit Cloud as described in Chapter 5.

6.2 Project Structure

The codebase follows a feature-first layout in which every domain is a self-contained package under `abridgeai/features/`. Each feature package contains the same internal layers: `models.py` (SQLAlchemy ORM), `schemas/` (Pydantic DTOs), `queries/` (database access functions), `services/` (business logic), `routers/` (FastAPI route handlers), and optionally `workers/` (background tasks) and `ai/` (AI pipeline stages). This uniform layout makes it straightforward to locate any piece of logic and to add new features without touching unrelated code.

```
abridgeai/
+-- api/                    # Application entry point, router registration
+-- core/                   # Cross-cutting: config, db, cache, audit, security
|   +-- audit/              # Async audit-log listener and worker actor
|   +-- cache/              # Redis client, decorators, cache-key registry
|   +-- db/                 # Soft-delete mixin, conflict mapper, hard-delete guard
|   +-- observability/      # Structured logging, HTTP audit log
|   +-- pagination/         # Cursor-based pagination
|   +-- security/           # JWT creation, token hashing, security headers
+-- ai/                     # Shared AI primitives
|   +-- chunking/           # Token-aware, semantic, timestamp-aware chunkers
|   +-- extraction/         # Multimodal extractors (PDF, DOCX, audio, video, ...)
|   +-- knowledge_graph/    # Neo4j builder and retrieval
|   +-- llm/                # LLM gateway, embedding client, pricing, audit
|   +-- prompts/            # Jinja2 prompt loader
|   +-- retrieval/          # pgvector, BM25, RRF fusion, MMR, role filter
+-- features/
|   +-- access_control/     # Organisations, org units, RBAC
|   +-- admin/              # Admin dashboards, AI cost tracking, audit viewer
|   +-- career_paths/       # Career path authoring and learner enrollment
|   +-- courses/            # Course, module, lesson authoring and learner view
|   +-- discussions/        # Lesson and course discussion topics and comments
|   +-- enrollments/        # Enrollment lifecycle management
|   +-- identity/           # Users, auth sessions, Google OAuth, MFA
|   +-- interviews/         # AI interview generation, session taking, evaluation
|   +-- learning_programs/  # Programme authoring, enrollment, path governance
|   +-- materials/          # Learning material upload, ingestion pipeline
|   +-- notifications/      # In-app and email notification dispatch
|   +-- policies/           # Versioned legal and academic policy documents
|   +-- progress/           # Lesson progress tracking and analytics
|   +-- quizzes/            # AI quiz generation, taking, grading
|   +-- spaced_repetition/  # SM-2 scheduler, card review, dashboards
+-- infrastructure/         # Third-party adapters: Neo4j, S3/Garage, Google OAuth
```

Figure 6.1: Top-level directory layout of the aBridgeAI backend.

Layered dependency rule

Dependencies flow strictly downward: routers depend on services, services depend on queries and AI primitives, queries depend on models. No layer imports from a layer above it. Cross-feature communication is mediated through the `api/public` module of each feature, which exposes a stable internal API surface. This prevents circular imports and keeps feature boundaries explicit.

Database migrations

Schema evolution is managed by 121 Alembic migration files running from `0001` to `0122`. Migration `0001` establishes the full baseline schema ported from a hand-managed SQL file. Subsequent migrations add partial unique indexes, cascade rules, audit columns, pgvector half-precision embeddings (`halfvec(3072)`), full-text search vectors, and the append-only trigger that protects the audit log. Every migration is reversible via a `downgrade()` function; where reversal would destroy records that cannot be reconstructed, the downgrade refuses rather than proceeding silently.

6.3 Access Control

The access control subsystem implements a hierarchical Role-Based Access Control (RBAC) model that supports multi-tenant organisations with nested organisational units. It is implemented in `abridgeai/features/access_control/` and `abridgeai/access_control/`.

Permission catalog

All permissions are declared in a YAML catalog (`access_control/permissions/catalog.yaml`) that is validated at startup against a Pydantic schema. Each permission entry specifies a `resource`, an `action`, and an optional `scope` (e.g., `course:read`, `quiz:generate`, `admin:users:write`). The catalog is the single source of truth for the permission namespace; adding a new permission requires only a catalog entry and a migration to seed it into the `permissions` table.

Role seeds

Default roles (`student`, `instructor`, `admin`) and their permission assignments are declared in `role_seeds.yaml` and applied by migration `0004_seed_permission_catalog.py`. Custom roles can be created at the organisation level and assigned any subset of catalog permissions.

Organisation hierarchy

Organisations are structured as trees of `OrgUnit` nodes. Membership in a parent unit implicitly grants membership in all descendant units. The recursive membership resolution query (`queries/sql/org_unit_tree.sql`) uses a PostgreSQL recursive CTE to traverse the tree in a single query, returning the full set of unit IDs a user belongs to.

Permission resolution

At request time, the `PermissionsLookupService` (`services/permissions_lookup.py`) resolves the caller's effective permission set by:

1. Loading all role assignments for the user across all their organisation memberships.
2. Expanding each role to its permission set via the catalog.
3. Merging direct permission grants (`user_permission_grants`) with role-derived permissions.
4. Caching the resolved set in Redis with a short TTL to avoid repeated database queries on every request.

Policy enforcement

Route handlers declare their required permissions via FastAPI dependencies. The `policies.py` module in each feature defines the permission checks for that feature's endpoints. A failed permission check raises `ForbiddenError` (HTTP 403), which the global exception handler converts to a structured JSON error response.

Audit logging

Every write operation that modifies a soft-deletable entity is recorded in the `http_audit_log` table by a SQLAlchemy event listener (`core/audit/listener.py`). The listener captures the actor's user ID, the HTTP method and path, the affected table and row ID, and a JSON diff of the changed columns. Audit writes are dispatched asynchronously via an actor pool (`core/audit/_worker_actor.py`) to avoid adding latency to the request path.

Holding a permission is not the same as holding it here

Resolution answers what the caller may do; it does not answer where. Because every assignment is scoped — globally, to an organisation, to an organisational unit, or to a

single course — the guard must additionally decide whether the scope the caller holds *contains* the resource being touched. A dean holding `course.edit` over one faculty must be refused on a course belonging to another, even though the permission resolves identically in both cases.

Containment is a walk up the organisational-unit tree, expressed as a recursive SQL CTE rather than an application loop: a unit-scoped assignment covers the target if the target's unit is that unit or any descendant of it. Organisation and global scopes are the degenerate cases at the top of the same walk, and a course-scoped assignment covers exactly one row. Keeping the walk in one query matters because it runs on every guarded request; an application-side traversal would issue a round trip per level of the hierarchy, and the depth is a property of the customer's faculty structure rather than something the platform controls.

6.4 Course and Curriculum

Courses are the unit a learner enrols in and the container every other feature hangs from. The implementation lives in `abridgeai/features/courses/`, split by audience rather than by entity: `authoring.py` for the teacher's editing surface, `catalog.py` for the learner's browse and detail views, `assignment.py` for teacher–course bindings, and `administration.py` for the operator actions that cross both.

The curriculum tree

A course owns ordered modules, and each module owns ordered items — lessons, quizzes, and interview configurations addressed uniformly so the learner's next step is a single ordered walk rather than three parallel lists. Ordering is a dense `position` integer per parent under a unique constraint, so two items can never claim the same slot; reordering rewrites the affected range in one statement rather than issuing a move per row.

Publication gating

A course, like every other authored artefact in the system, moves through `draft`, `published`, and `archived`. Draft content is invisible to learners at the query level rather than filtered in the presentation layer, so an unpublished lesson cannot leak through an endpoint that forgot to check.

Neither transition is reversible, and the asymmetry is deliberate. Publication is a one-way door: a published course can never return to draft, because its learning outcomes

double as the graded assessment scale and re-opening it for editing would move the goal-posts under students already enrolled against them. Archival is the separate terminal state, reached from either of the others, and it withdraws a course from new enrolment without disturbing the enrolments already running through it.

What publication requires

Three conditions gate the transition into `published`, each refusing a course that would fail a student in a different way. At least one gradeable unit — a published lesson, quiz or interview — because completion is computed from units, and a course with none can never be completed: as a required course on a career path that locks its stage, and every stage behind it, permanently. At least one teacher, measured against a per-organisation minimum, so a live course is staffed. At least one learning outcome, because a course that never states what it teaches cannot be assessed against anything. All three are evaluated only on the way in, so re-publishing a course that is already live does not re-litigate a rule an administrator has tightened since.

Publication also announces itself: the teachers attached to the course and the students already enrolled in it are notified once, on that same first transition. Dispatch is per recipient and best-effort — a failure is logged and the publication stands, because a message that could not be delivered is not a reason to withdraw a course that is otherwise fit to be live.

One transition, two doors

A course reaches `published` either through the dedicated publish endpoint or by patching its status, and the two must not be allowed to differ. A patch that applied the gradeable-unit gate alone, skipping the staffing and outcome gates, would make a course publishable or not depending only on which endpoint the caller chose, and nothing would surface the divergence: each door would be internally consistent. The announcement compounds it. Notification fires only on the transition, so a course published through a door that stayed silent could never be announced afterwards — the dedicated route would find it already published and stay silent in turn.

The patch therefore does not re-implement the transition; it delegates to it, applying the rest of the payload first so the announcement describes the course as patched rather than as it was a moment earlier. Copying the gates across instead would produce two implementations to be kept in step by hand, which is the arrangement that lets them part. The same permission guards both doors, so they cannot differ on who may open them either, and a refused publish rolls back the whole patch rather than leaving half of

it applied.

Archival has the same shape and the same treatment. Archiving a course that sits on a published career path is refused, because withdrawing it would lock that stage — and every stage behind it — for every enrolled student; writing the status column straight from a patch would walk past that check. Both lifecycle moves are reached through one implementation each, whichever endpoint is called.

Deletion is for drafts; archival ends a published course

Deleting a course is a draft-only operation, for every role including an administrator. Nothing can yet depend on a draft — enrolment refuses a course that is not published, and a published pathway cannot hold an unpublished course — so removing one costs nobody anything. Once a course is published, archival is the only way to end its life. A draft deleted in error is recovered by restoring the soft-deleted row rather than by a state change — an operator action with its own audit trail, not an ordinary edit.

The asymmetry exists because the soft-delete cascade cannot reach far enough. It walks the course's own children, the modules and lessons and items beneath it, but a pathway's reference to a course is a row in a join table that carries no deletion marker and no relationship back to the course. The cascade therefore leaves it standing, pointing at a course that progress queries no longer return: the stage quietly loses a requirement, its remaining work becomes the whole of it, and it can complete — and unlock the stage after it — under students who are mid-pathway. That is the same harm archival is written to prevent, which is why the more destructive door must not be the less guarded one.

When a live course may be archived

Archival is refused while the course still sits on a pathway somebody could reach. Establishing that is subtler than it first appears, because publishing a pathway does not retire the version before it (Section 6.10): a course attached once is recorded in that version permanently.

Asking only whether the pathway is published therefore blocks archival forever. Detaching the course in a new draft and republishing does not help — detachment is a draft-only edit, so the earlier published version keeps its row, and the pathway is still published. A course taken out of a live curriculum months earlier would remain unarchivable, leaving an operator holding only the deletion route, the one door that does real damage.

A version is treated as reachable when new enrolments would land on it — it is the current published version of a still-published pathway — or when somebody is still walking

it, pinned there either by a career enrolment or by a programme path attempt. A version that is neither is history: the course is recorded in it and nothing reads it. So the exit is the ordinary authoring one. Fork a draft, remove the course, publish; the old version stops being the one new students pin to, and once its last student has finished or moved on, the course archives.

Syllabus import

`syllabus_import.py` accepts a structured syllabus and materialises an entire module tree in one operation. It follows the same per-row discipline as the programme roster import: a malformed entry is reported against its position and the remainder still imports, because a syllabus with one bad row is the normal case rather than the exception.

Completion is counted over the visible curriculum

The course's own completion is an aggregate over its units, and the population that aggregate runs on has to be the same one the learner is shown — not merely similar to it. Visibility is decided at two levels, because the curriculum screen builds itself from published modules and then from the published items inside them. A unit therefore counts only when both hold: a published, undeleted lesson, quiz or interview, sitting in a published, undeleted module.

The second condition is easy to omit and expensive to omit, because the failure is silent and asymmetric. Filtering the item but not its container admits a published lesson inside a draft or archived module: absent from the learner's screen, present in the denominator. The student cannot reach it, so the tally can never close, the enrolment never promotes to completed, and the career-path stage behind it stays locked — for every student on that course, indefinitely, with nothing in the interface suggesting why. Archiving a module is an ordinary editorial act, which is what makes the omission reachable in normal use rather than only in a contrived one.

The same population is used by the teacher's roster, the progress summaries and the readiness figures, for the same reason the completion rules have a single implementation: a course that reads complete to the learner and incomplete to their teacher is a defect in whichever of the two computed it alone.

Enrolment and entitlement

Learners reach a course three ways — operator enrolment, a career path or programme that grants it, and self-enrolment by invitation code — and all three converge on one enrolment row whose status is `active`, `completed`, `dropped`, or `waitlisted`. Keeping

the grant paths separate from the enrolment record is what allows a programme path change to release an entitlement without disturbing an enrolment the learner obtained independently.

6.5 Material Ingestion Pipeline

When an instructor uploads a learning material, the file is stored in Garage S3-compatible object storage and a background worker triggers the ingestion pipeline implemented in `abridgeai/features/materials/ingestion/pipeline.py`. The pipeline transforms raw files into searchable, embeddable document chunks in five sequential stages. The stages below describe what each one does; Figure B.3 in Appendix B shows how they are strung together with the upload that precedes them and the teacher curation that follows, including the failure paths that no single stage owns.

Stage 1 — Extraction

The `dispatch_extractor(mime)` factory selects the appropriate extractor based on the file's MIME type. The extraction registry supports PDF, DOCX, PPTX, HTML, plain text, source code, images, audio, and video. Each extractor implements the `BaseExtractor` interface and returns an `ExtractedContent` object carrying the raw text and a `source_type` tag. Audio and video files are transcribed using a speech-to-text model before text extraction proceeds.

Stage 2 — Chunking

The chunker is selected based on `source_type`:

- **Audio / Video** — `TimestampAwareChunker`: preserves segment timestamps so retrieved chunks can be linked back to specific moments in the media.
- **Image** — `TokenAwareChunker(max_tokens=10,000)`: treats the entire extracted text as a single chunk.
- **All other types** — `SemanticChunker` when an LLM gateway is available; otherwise `TokenAwareChunker(max_tokens=500)` as a fallback.

The `SemanticChunker` uses an LLM to identify natural topic boundaries in the text, producing chunks that align with conceptual units rather than arbitrary token windows. Each chunk is then enriched with a `section_title` and a `context_sentence` via the contextual prepending technique described in Section B.

Stage 3 — Embedding

Each enriched chunk is passed to `EmbeddingClient.embed()`, which calls the configured embedding model (default: `text-embedding-3-large`, 3072 dimensions stored as `halfvec` for storage efficiency). Before calling the model, `build_contextual_text()` prepends the chunk's `section_title` and `context_sentence` to the raw content:

```
[Topic: {section_title}] {context_sentence} {content}
```

This contextual prepending follows Anthropic's Contextual Retrieval technique [50], which reduces retrieval failure rates by anchoring each chunk's embedding in its broader document context. Every embedding call writes an `ai_model_calls` audit row recording the model, token counts, and cost.

Stage 4 — Persistence

`DocumentChunk` ORM rows are written to PostgreSQL with denormalised foreign keys to the material version, lesson, module, and course. The `embedding` column stores the `halfvec(3072)` vector. A `content_tsv` column (populated by a database trigger) stores a `tsvector` for BM25 full-text search. A SHA-256 content hash is stored to enable deduplication on re-ingestion.

Stage 5 — Knowledge Graph (optional)

When `settings.knowledge_graph_enabled` is true, the pipeline calls `build_knowledge_graph_for_material_version()` in `abridgeai/ai/knowledge_graph/builder.py`. For each chunk, an LLM extracts a JSON object containing `entities` (concepts with optional definitions and confidence scores) and `relationships` (typed as `RELATED_TO` or `PREREQUISITE_OF`). The extracted graph is upserted into Neo4j, anchored to the course–module–lesson–material hierarchy. This graph is later used during quiz and interview generation to provide structured concept context alongside the vector-retrieved chunks.

Worker and transaction discipline

The pipeline is invoked by an `arq` background worker (`features/materials/workers/ingest.py`). The pipeline itself only flushes to the database session; the worker owns the commit. On any exception, the material version's `processing_status` is set to `failed` and the exception is re-raised so the worker can record the failure without silently swallowing errors.

Where the bytes live, and what stays readable during a re-ingest

Two boundaries are worth stating plainly because they determine what a failure costs. The first is the split between store and database: the object store holds the file itself, and the database holds only a reference to it plus the text derived from it. A failed ingest therefore never loses the upload — the bytes are already durable before the pipeline starts, and re-running the job re-derives everything downstream of them.

The second is the version cutover. A material is a sequence of versions, exactly one of which is flagged current, and the flag moves only once the new version's chunks are written and indexed. Until then retrieval continues to resolve against the previous version, so a re-ingest that fails at chunking or embedding leaves learners and the generation pipelines reading the last good text rather than nothing at all. The failure is recorded as a `processing_status` on the new version and the old one simply remains current, which is why a failed re-ingest is a recoverable inconvenience rather than an outage of the material.

6.6 AI Generation Pipelines

The quiz and interview generation pipelines share a common architectural pattern: a multi-stage orchestrator that threads a single `pipeline_run_id` through every LLM call so all `ai_model_calls` audit rows roll up to one `ai_pipeline_runs` parent. Each stage is a pure function that receives its inputs and returns typed outputs, making stages independently testable and replaceable.

6.6.1 Hybrid Retrieval

Both pipelines begin with a hybrid retrieval stage that combines dense vector search with sparse BM25 full-text search, then fuses the two ranked lists using Reciprocal Rank Fusion (RRF).

Dense retrieval

The `pgvector` retrieval module (`ai/retrieval/pgvector.py`) performs an approximate nearest-neighbour search using the `<=>` cosine distance operator on the `halfvec(3072)` embedding column. The query text is embedded with the same model used during ingestion, and the top-k chunks are returned ordered by ascending cosine distance.

Sparse retrieval

The BM25 module (`ai/retrieval/bm25.py`) queries the `content_tsv` column using PostgreSQL's `ts_rank_cd` function. This captures exact keyword matches that may be missed by the semantic search, particularly for technical terms, acronyms, and proper nouns.

RRF fusion

The two ranked lists are fused in `ai/retrieval/fusion.py` using the weighted RRF formula adapted from the Anthropic Contextual Retrieval cookbook [50]:

$$\text{score}(c) = w_{\text{sem}} \cdot \frac{1}{\text{rank}_{\text{sem}}(c) + 1} + w_{\text{bm25}} \cdot \frac{1}{\text{rank}_{\text{bm25}}(c) + 1} \tag{6.1}$$

where $w_{\text{sem}} = 0.8$ and $w_{\text{bm25}} = 0.2$ by default (configurable via `hybrid_semantic_weight` and `hybrid_bm25_weight` settings). Chunks absent from one list contribute zero from that leg. The fused list is re-ranked by descending score and the top-N chunks are forwarded to the generation stage.

Role-based filtering

A `RoleFilter` post-processor restricts retrieved chunks to those the requesting user is authorised to access, based on their course enrollment and organisation membership. This ensures that quiz questions are generated only from materials the student has been granted access to.

MMR re-ranking (optional)

When `mmr_enabled` is true, Maximal Marginal Relevance [51] is applied after fusion to reduce redundancy in the retrieved set. MMR iteratively selects the chunk that maximises relevance to the query while minimising similarity to already-selected chunks, producing a more diverse context window for the LLM.

6.6.2 Quiz Generation Pipeline

The full quiz generation pipeline (`features/quizzes/ai/pipelines/full.py`) executes six stages in sequence. Figure B.4 in Appendix B places these stages in the wider request they serve — the synchronous dispatch that returns before any of them run, the durable run row they report against, and the poll by which the teacher's browser learns the outcome.

1. **Retrieval** — hybrid retrieval as described above, returning the top-N fused chunks plus optional knowledge graph context.

2. **Ideation** — an LLM call that, given the retrieved chunks and a course outline, produces a list of question *topics* (ideation targets). The outline is either provided by the caller or synthesised from the chunk metadata. The ideation stage oversamples by a factor of 1.5 to give the downstream validation stage room to reject low-quality drafts.

3. **Generation** — for each ideation target, an LLM call generates a multiple-choice question with four options, a correct answer key, and an explanation. Prompts are rendered from Jinja2 templates in `ai/stages/generation/prompts/`.

4. **Validation** — each generated question is passed to a second LLM call that checks factual accuracy, option quality, and alignment with the source chunks. The validator returns a verdict (`accept`, `revise`, or `reject`) with a rationale. Rejected questions are discarded; revised questions are patched in place.

5. **Deduplication** — accepted questions are compared pairwise using embedding cosine similarity. Questions whose embeddings exceed a configurable similarity threshold are deduplicated, keeping the higher-quality variant.

6. **Persistence** — accepted, deduplicated questions are written to the `quiz_questions` and `quiz_question_options` tables as a draft batch, associated with the quiz and the `pipeline_run_id`.

6.6.3 Interview Question Generation

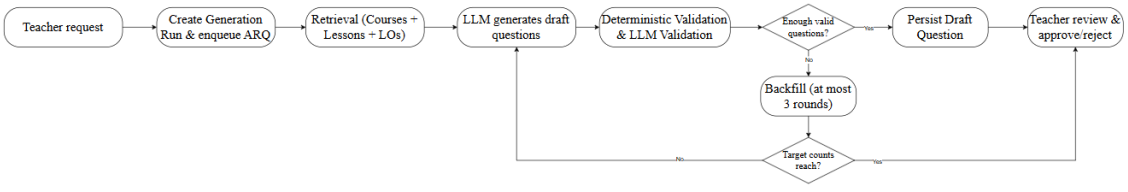


Figure 6.2: Interview Question Generation Pipeline.

As shown in Figure 6.2, the pipeline retrieves authorised course material and learning outcomes, then creates draft open-ended questions with suggested assessment guidance. Basic validation checks grounding and outcome alignment before accepted drafts

are stored for teacher review. Only teacher-approved questions enter a published interview configuration.

6.6.4 LLM Gateway and Audit

All LLM calls are routed through `LLMGateway` (`ai/llm/gateway.py`), which provides a uniform interface over any OpenAI-compatible endpoint. The gateway:

- Selects the model and API key based on the `LLMRole` (e.g., `QUIZ_GENERATION`, `EMBEDDING`, `KG_BUILD`), allowing different roles to use different models or providers.
- Parses LLM responses robustly: it first attempts strict `json.loads`, then extracts from a fenced code block, then uses `raw_decode` to tolerate trailing commentary from models that ignore the `json_object` response format.
- Writes one `ai_model_calls` row per call recording the model name, prompt and completion token counts, computed cost (via a per-model pricing table), latency, and the `pipeline_run_id` grouping key.

Nothing generated reaches a learner unreviewed

The stages above end at the gateway, but the pipeline does not end at the model. Every artefact a generation run produces — quiz questions, interview questions, extracted knowledge — is written in a pending state and requires a human decision before it becomes visible to a student. A teacher approves it, edits it, or rejects it; the original and the edited form are both retained, so the quality of the generation itself stays measurable over time rather than being overwritten by the corrections applied to it.

This gate is the reason the retrieval work above is worth doing rather than merely interesting. Retrieval quality determines how much of the teacher's attention each generated item consumes: grounded, deduplicated, access-filtered context produces drafts that are mostly approved as they stand, whereas poor retrieval produces drafts that are individually plausible and collectively wrong, which costs more to review than to write. The pipeline is therefore tuned for reviewer throughput, not for autonomy — the system never proposes to publish its own output.

6.7 Adaptive Quizzing

Section 6.6 covers how quiz questions are generated. This section covers what happens afterwards: review, publication, the attempt itself, grading, and release. The implemen-

tation is in `abridgeai/features/quizzes/`, whose service layer is deliberately one module per concern — `publish_gate.py`, `grader.py`, `gradebook.py`, `regrade.py`, `overrides.py`, `integrity.py`, `review_visibility.py` — because these rules change independently of one another. Figure B.5 in Appendix B shows the attempt itself end to end, where these concerns — integrity monitoring, spaced repetition and recovery from an abandoned attempt — meet on the same rows.

Human review before publication

Every question carries a `review_status` of `pending`, `approved`, `edited`, or `rejected`. AI-drafted questions enter as `pending` and only approved questions are ever delivered in an attempt. `publish_gate.py` refuses publication while any question is unreviewed, and separately refuses it while any question lacks an expected answer time — the latter because the spaced-repetition scheduler grades recall partly on elapsed time against expectation, so a question with no expectation cannot be scheduled and must not reach a learner.

Per-student rule resolution

A quiz's dates, time limit, and retake rules can be adjusted for an individual or a group. `override_policy.py` resolves the effective rules for one learner in a fixed precedence: the quiz's own settings, then the most generous group adjustment that applies to them, then their personal adjustment. Group adjustments resolve to the most generous rather than the last-written, because an accommodation must never be narrowed by a second one the learner also qualifies for.

The attempt

An attempt moves through `in_progress` to one of `submitted`, `graded`, `abandoned`, or `expired`. Question and option order are shuffled per attempt by `shuffle.py` from a per-attempt seed, so the order is stable across a reload but differs between learners. An attempt still open when its time limit or the quiz's close date passes is resolved automatically — submitted, granted a grace period, or closed ungraded — as the teacher configured, so an attempt abandoned by a closed laptop does not wait on the learner's return.

Grading and the grade of record

`grader.py` marks objectively-answerable questions on submission and sets aside anything it cannot mark with confidence for the queue in `manual_grading.py`. While any answer in an attempt remains unmarked the learner is shown no score, since a partial

score presented as final would mislead. `gradebook.py` then reduces a learner's attempts to one grade of record by the quiz's declared method — highest, first, last, or average — and stores the method and the attempts counted alongside the figure, so a grade can be explained later rather than merely asserted.

Regrading

`regrade.py` retains an internal dry-run and commit foundation for re-evaluating affected attempts, but the capability is not exposed in the current teacher interface. Published questions are frozen and there is no controlled answer-key-correction workflow; exposing regrade without that entry point would produce an action that normally reports no changes, while applying a correction without reconciling spaced-repetition state could leave grades and ease factors inconsistent. The complete correction-to-regrade workflow is therefore deferred to future work.

Integrity signals

`integrity.py` records window blur, tab switches, full-screen exits, and connection loss while an attempt is running. Recording never blocks or fails a submission, signals arriving after submission are discarded, and the weights and flag threshold are fixed at the moment the attempt starts — so editing a quiz mid-cohort cannot change how an attempt already under way is judged. The records are evidence for a teacher to review, not a verdict, and learners never see them.

One client per attempt

`session_guard.py` binds a live attempt to the authentication session that owns it, through the Redis key `quiz_attempt_session:{attempt_id}` held for ninety seconds. Claim, heartbeat, takeover and release are compare-and-set Lua scripts, so two tabs resuming the same attempt cannot both win the race; the frontend additionally holds a Web Locks key per attempt and coordinates handover over a `BroadcastChannel`, falling back to a channel leader handshake where Web Locks is unavailable. A learner moving to another device confirms an explicit takeover.

Two decisions here run against the grain of the rest of the backend and are deliberate. First, answer saves, integrity events and submission *fail closed* when Redis is unreachable, where every other Redis path degrades to PostgreSQL — admitting two concurrent clients to a graded attempt is worse than refusing the write. Second, the key is ephemeral rather than a durable lease: an abandoned client releases the attempt by falling silent. The server keeps the authoritative timer throughout, so closing a tab neither pauses nor forfeits an

attempt; an accidental close leaves it resumable, while a deliberate exit flushes the current answer, releases the key, and stops the camera.

The boundary is worth stating plainly. A copied access token reusing the same authentication session id is indistinguishable from the original client, Redis loss or an administrative flush drops ownership, and a modified client can ignore the frontend tab coordination entirely. These are accepted limits of a session-scoped guard; closing them needs a persistent assessment lease shared by quizzes and interviews, with opaque per-client credentials, generation counters, audit history, invalidation on a Redis reset, and server-side enforcement reaching into the interview transport. That is left to future work.

Required camera

`Quiz.require_camera` defaults to false. Where a teacher enables it, the learner must grant `getUserMedia` video access before starting or resuming, and the workspace blocks whenever the video track is missing, ended, or muted — after a four-second grace period, so a momentary device glitch does not interrupt the attempt. The timer and the ownership heartbeat both continue during that block, because the requirement governs whether the learner may keep working, not whether the clock runs.

As implemented, the check is a presence test and nothing more: the requirement is satisfied by an active video track, and the stream is held only as a local `MediaStream` in the browser — no capture, upload, or analysis path exists in the current code. The flag is copied into the attempt's `integrity_policy_snapshot` at start alongside the signal weights, so a quiz edited later cannot retroactively impose — or lift — a camera requirement on an attempt already under way. Doing anything with the video itself, and the privacy and consent obligations that would follow, is left to future work.

6.8 Mock Interview Sessions

The mock interview subsystem assesses a learner through a structured question-and-answer session. Teachers configure and publish the interview in advance, including its questions, learning outcomes, and assessment rules. Published settings remain stable for the duration of a learner session.

Main handling flow

The learner completes onboarding, then answers each interview question through typed text or optional audio input. The system records the turn, analyses the response, and selects a follow-up or next question. This loop continues until the configured interview

scope is complete, after which terminal evaluation produces the final result and feedback.

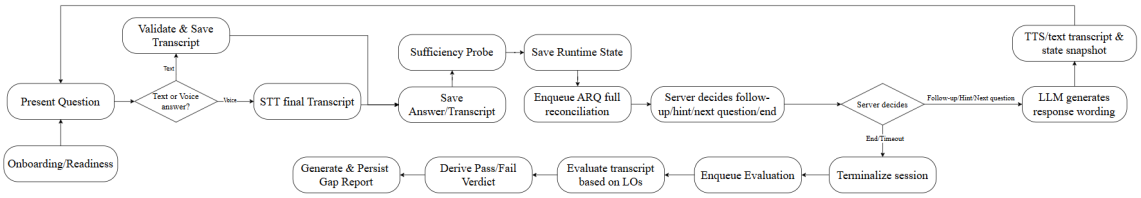


Figure 6.3: Activity Diagram: Mock Interview Main Handling Flow.

In-session processing supports the conversation, while terminal evaluation determines the assessment result. Learners receive their result and improvement guidance. Authorised teachers can review the supporting session record and assessment information.

6.9 Spaced Repetition Implementation

The spaced-repetition subsystem implements the SM-2 model and its extensions in `abridgeai/features/spaced_repetition/`. The main implementation layers are the pure scheduling core, the review transaction, the lesson-unlock read model, the due-card worker, and instructor analytics. The full formulas and process diagrams are specified in Appendix B; this section records the production composition and its main safeguards.

Scheduling core and review transaction

The pure `sm2/` functions derive Q from correctness, hint usage, and response-time ratio, update the Easiness Factor, schedule the next interval, apply jitter, and enforce the interval ceiling. They have no database or network dependencies and are unit-tested at the threshold values for response time, repetition count, and Q. The review service loads or initialises the student's card state, applies these functions, writes the updated state and an immutable before/after `CardReview` row in the caller's transaction, and returns pending events for dispatch only after commit. Consequently, a failed transaction cannot leave either a partial schedule update or a learner-visible notification.

Assessment and lesson progression

Assessment quizzes do not create new review cards, while existing card evidence is still graded. Lesson unlock is implemented by `sm2/lesson_unlock.py` as a read-side gate combining lesson prerequisites, EF coverage, and the optional interview-pass requirement described in Section B.2. Figure B.2 in Appendix B shows the complete gate. The imple-

mentation returns both the verdict and the blocking evidence used by the learner interface; it does not persist an unlock event.

Reminders and analytics

The hourly worker in `features/spaced_repetition/workers/scan_due_cards.py` scans cards with `due_at <= NOW()` and sends at most one in-app reminder per student within the configured cooldown. The quiz-level reminder flag determines whether a student is eligible for a reminder, while the reported count remains the student's full due backlog. Instructor endpoints expose at-risk students, review compliance, and class-level knowledge-retention distributions through the feature's SQL queries.

Robustness and end-to-end cycle

Beyond textbook SM-2, the implementation bounds EF, dampens positive growth for guessable formats, clamps untrusted response times, and clamps teacher-provided initial EF values before persistence. The resulting cycle is: a graded response produces Q, the card state determines the next review, and the due-card worker later reads that state to issue a reminder. Unlock is evaluated separately from the same persisted card state on the learner's next request, so scheduling and progression use one current source of truth without duplicating the algorithmic specification.

6.10 Career Pathways

A career pathway is the route a student walks: an ordered sequence of stages, each holding required courses and a set of electives from which the stage asks for a stated number. The rules that decide what a student has finished and what they may open are specified in Section 5.3.1; this section covers how the subsystem is built around them. It is implemented in `abridgeai/features/career_paths/`, split by audience into an authoring surface for managers and a learner surface for the student walking the path.

A version is the unit of truth

A pathway is never edited in place once students are on it. Publishing freezes a version — its stages, their ordering, their course membership, the required flags and the unlock policies — and every enrolment pins the exact version it started on. A later revision is a copy-on-write fork into a new draft; it does not reach a student already walking the earlier one, and it does not reach a programme version that froze the earlier one either. Every read in the subsystem therefore takes a `version_id` rather than a path identifier,

and the progress a student sees is always measured against the route they were actually given. This is the premise the rest of the section depends on: without it, an author editing a pathway would silently change what enrolled students must finish.

Several versions are published at once, and that is the point

Publishing a pathway does not retire the version before it. Both stay `published`, and so does the one before that: an enrolment is pinned to the version it started on, and that row has to remain published and readable for as long as a student is still walking it. Retiring the predecessor on publish — which is what the policy documents do, one live version per language — would break every student mid-pathway.

The consequence is that "the current version" cannot be a database constraint. A policy can assert that exactly one version is live and have the schema enforce it; a pathway cannot, because several being live simultaneously is the correct state. Current is decided instead by ordering: among the published, undeleted versions of a pathway, the one with the highest version number is what a new enrolment pins to, and the earlier ones are what existing enrolments continue to read.

An ordering rule is only as reliable as the number of places that spell it out. This one is written once and every resolver is built on it — the single-pathway form and the many-pathway form are the same ranked statement under a different filter, rather than two spellings that would have to be kept in agreement by hand. A sibling feature validating a set of pathways before attaching them to a programme asks for the resolution rather than re-deriving it. A second derivation that agrees today is the hardest kind to catch on the day it stops agreeing.

One module owns the rules

Satisfaction, stage completion, unlocking and progress are computed in exactly one place, `services/stages.py`. The learner's progress endpoint, the manager's roster, the readiness worker and the Start endpoint all call into it rather than deriving any part for themselves. The discipline is not stylistic. A second implementation of "finished" can be individually reasonable and still disagree with the first at the margin, with nothing watching for the disagreement; a rule with one implementation cannot drift from itself.

The stage latch is append-only

When a stage evaluates complete for an enrolment, a row is written to `student_stage_progress`, and that row is never updated or deleted. The table deliberately omits the soft-delete and audit mixins the rest of the schema carries, because a soft-deleted latch would be a

way to un-complete a student — the precise failure it exists to prevent.

The need arises because stage completion is *derived* rather than recorded, and derived answers can move backward. A learner can un-tick a lesson, demoting the course beneath them; an author can raise a stage's elective quota. Either would silently re-lock a stage the student had already passed. The asymmetry is deliberate and load-bearing: un-marking a lesson demotes the *course*, so satisfaction stays honest, but it never un-latches the *stage*. Work once finished is finished.

Unlocking chains through the prefix

Unlock is evaluated in a single forward pass, each stage reading the one before it, because a gate may depend on a previous stage's completion and that completion may itself be latched. A gated stage requires the previous stage to have been *reachable* as well as finished — the reasoning is in Section 5.3.1. An unrecognised unlock policy resolves to locked rather than open, so a malformed rule withholds content instead of releasing it.

A lock is a setting, not a verdict

Whether a locked stage actually blocks a student from starting its courses is the author's choice: only `hard` enforcement refuses, while `soft` and `advisory` record the lock, warn the student and let them through. The distinction is routed through a single helper rather than tested at each call site, and that indirection is the fix for a real defect: the Start endpoint once tested the unlocked flag directly and blocked every enforcement level, so an author who had chosen "show a warning, still allow" got a hard block instead. The authoring interface offers these as three distinct choices; blocking all three would make the interface lie about what it does.

Publication is gated; mutation is not

"Every pathway has at least one stage" and "every stage has at least one course" are enforced when a pathway is published, not when it is edited. Enforcing them on the mutation path would make ordinary authoring impossible — a stage must be creatable before it can be filled, and a pathway must exist before it has stages. The gate separates hard failures, which refuse the publish, from soft findings, which are returned so the interface can surface them without blocking an author who means it.

Editing a live pathway is classified, not forbidden

A published pathway with students on it can still be revised, and the subsystem's job is to make the consequence visible rather than to refuse. Every mutation is classified

against the number of students actually walking the path and how far they have reached: an edit is *breaking* when an in-flight student ends up worse off — work added to a stage they have not finished, the goal moved by a raised quota or a lengthened path, or a stage they could previously enter now locked — and safe otherwise. With no active enrolments nothing can be breaking, so authoring a draft is unconstrained.

A separate impact read reports the blast radius before the author commits: how many students are enrolled and how many sit in each stage. Reordering stages is the subtlest case and is handled by reporting rather than repair — moving a stage away from the first position changes what its stored unlock policy means, so the operation returns warnings and leaves the policy as the author wrote it. Silently rewriting it would discard the author's intent at the moment the position that expressed it moved.

Archive, never delete

A published pathway is archived rather than removed. Archiving blocks new selections and invalidates pending change requests that targeted it, while students already walking it continue under their pinned version, with their progress, completion awards and history untouched. Outright deletion is reserved for drafts, where by definition no student can be enrolled.

Readiness is sampled, not recomputed

A nightly worker writes one append-only readiness snapshot per active enrolment, each stamped with the pathway version that produced the score. Readiness is therefore a record of what was true on a given night rather than a figure re-derived from today's rules — an author who revises a pathway, or a student whose course is demoted, changes future points and never past ones. The alternative, recomputing history on read, would silently rewrite a chart every time the underlying pathway moved.

The compatibility projection

Learner-facing pathway endpoints still authorise through `student_career_enrollments`, a per-student row maintained alongside the programme path attempts that are the real source of truth. Starting an attempt ensures the projection exists; ending the last attempt on a pathway releases it. It is named here as what it is — a staged-cutover artefact, kept so the learner surface continued to work while enrolment moved up into the programme layer, not a second authority on who is on which pathway.

6.11 Programme and Path Governance

Section 6.10 covers the pathway itself — its versions, stages and the evaluation that decides what a student has finished. This section covers the enrolment structure above it: which career paths a student holds, how they are acquired, and how one is given up. It is implemented in `abridgeai/features/learning_programs/`. Figure B.7 in Appendix B traces the giving-up case in full, from the learner's request through the dean's decision to the entitlements the approval revokes.

A held path is a row, not a column

The enrolment carries no `career_path_id`. `ProgramEnrollment` owns many `ProgramPathAttempt` rows, and the paths a student currently holds are the active ones. This is what lets a programme permit several paths at once, and it is what keeps a path that has ended readable: closing an attempt writes an `exit_snapshot` of the progress it had earned rather than clearing a field — measured by the same stage evaluation the learner's own view and the manager's roster report (Section 5.3.1), so the frozen figure means what the live one meant the moment before it froze. Each attempt also records a `selection_source` — `program_default`, `student`, or `path_change` — so an audit can distinguish a path the system assigned from one the student chose.

Automatic placement at enrolment

A published programme version must nominate exactly one of its paths as the default, enforced by a partial unique index on `learning_program_version_paths` rather than in application code, so no concurrent edit can produce a second one. Enrolment resolves that default *before* writing any row: if it has since been archived, the whole operation aborts. Resolving it afterwards would have admitted the student and then failed, leaving an enrolment with no path — a state nothing else in the feature can produce or recover from. Both enrolment doors, the hand-picked list and the CSV import, go through the same helper for this reason.

The default is left unstarted, rather than the enrolment refused, in three cases: the student already holds that path in another programme, they are at the organisation's ceiling for concurrent paths, or they have already completed the version the programme pins. The enrolment stands and waits in `awaiting_path` so the student chooses a path for themselves. The last case is what stops a programme completing itself the moment it is joined: an attempt whose pinned version is already finished completes at the next progress evaluation, which would leave no active attempt, close the enrolment against a

programme in which the student had done nothing, and refuse every further path they tried to add. It is judged by the same `is_version_complete_for_user` that evaluation uses, against the same pinned version, so the guard and the sweep cannot reach opposite answers.

Adding is direct; leaving is reviewed

Taking on a further path is a direct write, bounded by two limits. A programme may cap how many of its paths one enrolment holds; the cap is optional and, where set, may not exceed the paths the programme bundles. Where it is unset the programme imposes no limit of its own, and what binds is the organisation-configurable ceiling on concurrent paths a student may hold across all their programmes — the only limit that sees the total, since per-programme caps would otherwise sum, and a student in two programmes capped at one and two could hold three. Leaving one — by switching to another path, or by dropping it outright — creates a `PathChangeRequest` that a Faculty Dean must decide.

Both kinds share one table, discriminated by a `kind` column, because they share everything that matters: the review queue, the partial unique index enforcing one open request per enrolment, the finite switch allowance an approval spends, and the rejection vocabulary. They differ only in whether a destination exists, so the target columns are nullable under a `CHECK` constraint that requires one for a change and forbids one for a drop. A separate table would have duplicated the queue and the allowance, and made the single-open-request rule unenforceable by any one constraint.

One path, one programme at a time

A career path may be bundled by several programmes, but a student may hold it in only one of them at a time. The rule is a partial unique index on `(student_id, career_path_id)` across active attempts, which requires `student_id` on the attempt itself: it otherwise lives on the enrolment, one table away, where no constraint can reach it. A denormalised key that drifts would be worse than none, so a composite foreign key ties `(program_enrollment_id, student_id)` back to the enrolment it was copied from, which makes an attempt whose student disagrees with its enrolment's unrepresentable rather than merely unlikely. The service refuses a duplicate first and names the programme already holding it; the index is what holds when two requests in different enrolments race, which no per-enrolment constraint can see.

Invariants are re-checked at the decision, not trusted from the request

Every precondition is verified twice: once when the student files, and again when the dean decides. The gap between the two is measured in days, and the enrolment can move within it. Approving a change re-confirms that the source attempt is still active and that the target has not since been archived, in which case the request is marked `invalidated` rather than failed. Approving a drop re-confirms that another path would remain in progress, because the student's other path may have been completed, switched out, or dropped by an earlier approval since filing.

Entitlements follow the paths, not the request

Course access granted by a programme is released only when no remaining attempt still justifies it, resolved by counting the student's other active attempts on that path rather than assumed from the path being left. With several paths held at once, a course shared between two of them must survive one ending. Completions are held separately again, as `CourseCompletionAward` rows attached to the course, so a result already earned is never recomputed when the path beneath it changes.

Programme completion

An enrolment completes only when no active attempt remains, so a student holding two paths completes the programme on finishing the second, not the first. Each attempt is judged against the path version it was pinned to, never the path's current head, so a revision published mid-enrolment cannot retroactively change what a student must finish.

Whether an attempt's pinned version has been finished is not decided here. It is asked of the career-path feature through `is_version_complete_for_user`, which runs the stage evaluator specified in Section 5.3.1: every required course satisfied, and each stage's elective quota met. Re-deriving it here would invite a stricter reading than the one the student sees — testing that every course attached to the version was satisfied, say — and the two would part company exactly where the quota bites — a pathway of two required courses and five electives asking for one would show the student 100%, complete their career enrolment, and leave the programme attempt open against four electives nobody owed. Neither side would surface the divergence, each being individually self-consistent. Completion is one rule with one implementation for the same reason the request table is one table: the moment a second copy exists, the two drift.

One distinction is preserved rather than flattened. Evaluated for the student's own pathway, a stage counts as complete if it is *latched* or currently satisfied, so completion

never regresses once earned. Evaluated for a programme attempt there is no latch to read: the attempt pins its own version, which need not be the one the student's career enrolment — and its latches — point at. The attempt is therefore judged live, against the version it actually promised.

A version presenting no stages at all returns not-complete in both paths. An unmeasurable pathway has not been shown to be finished, and what these writes produce is a durable academic record rather than a progress bar; publication guarantees at least one stage per path and one course per stage, so no pathway a student can hold reaches that branch.

The request lifecycle

One lifecycle serves both kinds. A request is filed `pending` and may be acknowledged into `in_progress`, which signals to the student that it is being considered but decides nothing — both states are open, both occupy the one-request-per-enrolment slot, and both are decidable, so a dean may rule straight from the queue without acknowledging first. From either, the dean moves it to `approved` or `rejected`; the student may withdraw it to `cancelled` while it remains open; and the system itself may end it as `invalidated` where the re-checks above fail at the decision. That last transition is the reason the lifecycle has six states rather than five: a request overtaken by events is neither the student's fault nor the dean's refusal, and recording it as a rejection would misattribute both.

The two kinds diverge only at approval. Approving a change closes the outgoing attempt with a progress snapshot and opens a replacement pinned to the target path version; approving a drop closes the attempt and opens nothing. Everything before that point — the queue, the allowance, the open-request rule, the rejection vocabulary — is identical, which is what makes the shared table worth its nullable target columns.

6.12 Caching

Redis carries three unrelated responsibilities in this system, and keeping them distinct matters: an application read cache, a small amount of ephemeral runtime state, and the `arq` job queue. Only the first is a cache in the sense of this section. Cache keys are declared centrally in `core/cache/keys.py` — pattern, TTL and purpose in one place — so no call site hard-codes a key string, and the declaration is what the invalidation listener reads when a write lands.

Table 6.1 lists the active namespaces.

Table 6.1: Redis cache namespaces: key pattern, time-to-live, and what each entry saves.

Key pattern	**TTL**	**Purpose**
`course_content:published:{course_id}`	300 s	The published curriculum tree a learner opens a course to. Shared by every learner on the course; a miss rebuilds it from six batch queries.
`lesson_unlock:{student_id}:{lesson_id}`	60 s	Verdict of the unlock gate, whose prerequisite traversal is recursive and re-evaluated on every read that needs it.
`cards_due:{user_id}:...`	60 s	One page of the due-card queue. The suffix carries the lesson, course, cursor and limit, so invalidation clears the whole prefix.
`kr:{user_id}:{lesson_id}`	300 s	Knowledge-retention estimate $\hat{R}_{s,\ell}$; drifts slowly but is read on every dashboard load.
`compliance:{user_id}:{lesson_id}`	3600 s	Review compliance rate. The heaviest of the metric queries and the slowest-changing.
`material:ingest:progress:{material_version_id}`	3600 s	Live ingest progress. Written outside the ingest transaction so the teacher sees movement before it commits; PostgreSQL stays authoritative.
`quiz_attempt_session:{attempt_id}`	90 s, renewed	Which browser session owns a live quiz attempt. Security-sensitive, so it fails closed rather than open.

Invalidation

TTL is the safety net, not the mechanism. A SQLAlchemy listener maps each written table to the keys it invalidates and issues the deletions itself, so a review that crosses an

unlock threshold is visible on the learner's next read rather than up to a minute later. Two details make that work in practice. Keys whose live entries carry suffixes beyond the declared pattern — the due-card pages — are cleared by prefix rather than by exact match, or every real key would survive the delete. And the shared course-content key is deleted twice: once at flush, and again after the transaction commits, because between those two moments a concurrent reader can refill it from the pre-write snapshot and pin that stale tree for the full TTL, for everyone on the course.

Failure policy

Every read through the cache falls back to PostgreSQL, and every failure to write one is logged and dropped. A dead Redis therefore costs latency and nothing else — no request fails because a cache entry could not be read or stored. The one deliberate exception is quiz-attempt ownership, which is an authorisation decision rather than a cache: when Redis cannot evaluate the atomic claim, the answer is no.

What is not cached

Permissions, authentication principals, presigned URLs and knowledge-graph projections all have declared key namespaces that are not wired into the read path. Declaring a key is not the same as having a cache: each of these needs an invalidation story that is provably complete before it can be turned on, and for permissions and sessions the cost of being wrong is stale access rather than a stale page. They are left inactive deliberately, since an additional database query is cheaper than a revoked permission that keeps working.

6.13 Platform Configuration, Policies and Audit

Runtime settings

Operational limits — concurrent programme enrolments, concurrent career paths per student, retention windows — are runtime settings rather than deployment constants, resolved per organisation by `abridgeai/core/runtime_settings.py`. The effective value is the first of four levels that supplies one: a `system_settings` row for this organisation, a row with a null organisation (the deployment default), the environment variable named by the setting's spec, and finally the code default declared in the settings registry.

The last two levels are what make adding a setting safe. A deployment that has never heard of a new key still resolves it, so a release does not require a configuration change to land first. The resolution API returns every candidate value alongside the effective one

and the level it came from, because an administrator asking "why is this 3?" needs to see which layer won rather than be told the answer.

One row set per organisation is cached in process under a short TTL, since settings are read on many requests and written rarely.

Policy documents

`abridgeai/features/policies/` holds terms, privacy notices, and academic regulations. Versioning is copy-on-write: a published version is immutable and correcting it opens a new draft, so the exact text a reader agreed to remains retrievable. Audience is a set of roles, and the empty set means public — including signed-out visitors.

Audit

Two audit trails run side by side. The typed authentication and access-control trail records sign-in, session, and role-assignment events; the HTTP audit log records mutating requests with actor, method, path, affected row, and a column-level diff, written asynchronously through an actor pool so the request path carries no additional latency.

Both are append-only, and the guarantee is enforced in the database rather than in application code: a trigger rejects `UPDATE` and `DELETE` on the audit tables outright, with deletion permitted only inside a narrowly scoped maintenance session used by the retention job. This is the property that gives the trail its value — a participant cannot remove evidence of what they did, and neither can an administrator.

6.14 Frontend Implementation

The client is a single-page application built with React 19 and TypeScript 5.9 on Vite 8, comprising roughly 1,600 TypeScript modules of which some 850 are route components. It is organised by route rather than by layer: each screen owns a directory holding its page component, a private `_components/` folder, and its tests, so the blast radius of a change is visible from the path.

Routing and server state

Navigation uses TanStack Router, whose typed route definitions make a broken link a compile error rather than a runtime 404. Server state is held entirely in TanStack Query — there is no parallel client-side cache of server data, and no global store mirroring it. Mutations invalidate query keys declared in one registry, so a write refreshes exactly the views that depend on it without each call site deciding for itself what to refetch.

Types generated from the API, not written twice

Request and response types are generated from the backend's OpenAPI schema by `openapi-typescript` into a checked-in declaration file. A `codegen:check` script regenerates and fails on any difference, so a backend contract change that the client has not absorbed breaks the build rather than surfacing as a runtime shape mismatch. Hand-written types remain only for the few endpoints outside the generated schema, and are marked as such.

Design system and theming

Styling is Tailwind 4 over a Material 3 token layer: components consume semantic tokens rather than raw palette values, which is what allows the light and dark themes to be defined once at the token level instead of per component. Dialogs, menus, and other overlay primitives come from a headless component library, so keyboard and focus behaviour is implemented once.

Internationalisation

All user-facing copy resolves through i18next against English and Vietnamese catalogues. Text is never inlined in a component; a missing key is therefore visible as the key itself rather than silently rendering in the wrong language.

Testing

Component and integration tests run under Vitest with Testing Library, against a Mock Service Worker layer that intercepts at the network boundary — so tests exercise the same fetch path as production rather than a substituted client. Playwright covers end-to-end journeys. A bundle-size budget is checked against a recorded baseline on build, so a dependency that materially inflates the download fails the build rather than being discovered in production. The suite's composition and its executed results are reported alongside the backend's in Section 6.15.

A note on type checking

The repository root holds a solution-style `tsconfig.json` that lists project references and no files of its own. Running `tsc -noEmit` against it therefore type-checks *nothing* while appearing to succeed — the build script uses `tsc -b`, which walks the referenced projects, and that is the only invocation whose success is meaningful.

6.15 Testing

Verification of this system is documented in four places, and it is worth stating once which owns what. Section 6.15 — this one — describes how the implementation is verified: the layers the suite is built from, what each layer can establish that the others cannot, what the suite measures, and what is deliberately left outside it. The test cases themselves, traced to the requirements they discharge, are designed in Section 5.10 and listed in full in Appendix H (Volume II); the per-case execution detail behind the totals reported here is in Appendix G. Quality attributes — latency under load against the non-functional targets, frontend audit scores, and generated-question quality — are not verified by this suite at all: they are measured, and those measurements are Chapter 7's.

6.15.1 Test architecture

The suite is layered by what each layer is able to prove, not by the packages it happens to cover. The distinction that matters most is between the tests that may touch a database and the tests that may not.

Table 6.2: Test layers, what each establishes, and what it requires to run.

Layer	**What it establishes**	**Requires**	**Files**
Backend unit	Pure logic and single-module behaviour: scheduling arithmetic, grading, derivation rules, router branches driven by stubbed sessions.	Nothing external	299
Backend integration	Properties that exist only in the database: transaction boundaries, cascade and constraint behaviour, raw SQL, concurrency and race handling.	PostgreSQL + Redis	166
Architectural	Invariants that are otherwise matters of review: audit bypass, org-scoped routing, prompt-injection hardening.	Nothing external	3
Characterisation	Behaviour of the legacy surface preserved across the ORM consolidation and cutover.	PostgreSQL	2
Frontend unit / component	Component rendering, hook state machines, guard and invalidation wiring, formatting and i18n integrity.	jsdom, requests intercepted at the network boundary	229
Frontend end-to-end	Whole journeys through a real browser against a running stack, including a smoke wave and an accessibility wave.	Browser + full stack	9

The split between the first two rows is the load-bearing one. A unit test that stubs the session can assert that a service calls what it should and in what order — the commit-then-dispatch ordering the notification path depends on is verified exactly this way — but it cannot establish that a constraint fires, that a cascade reaches the rows it should,

or that two concurrent writers reach a consistent state. Those properties do not exist in Python; they exist in PostgreSQL, and the tests that assert them run against a real instance with the migrations applied. The architectural layer is the unusual one: rather than testing behaviour, it walks the route table and the model registry to assert that no route escapes organisation scoping and no write escapes the audit trail, so an invariant that would otherwise depend on a reviewer noticing fails the build instead.

6.15.2 Results

Both suites were executed in full on the deployment server. The backend run applies the migrations first and measures branch coverage over the `abridgeai` package in the same pass.

Table 6.3: Executed test results.

	Backend	**Front-end**	**End-to-end**
Passed	6,871	1,668	52
Skipped	16	0	26
Deselected by marker	3	—	—
Expected failures (`xfail` / `xpass`)	5 / 7	—	—
Failed	0	0	0
Test files	472	229	9
Wall-clock duration	13 min 26 s	75.9 s	2 min 6 s

That is 8,591 executed cases with no failures. Coverage over the backend is **85.8%** measured with branch coverage across 45,945 statements, against a configured gate of 75% that fails the build when it is not met. The front-end runs report no coverage figure because the gate is not applied there; their file counts are given above instead.

The end-to-end column is the smallest and the most expensive per case, which is the expected shape: each of those 52 drives a real browser against a running stack. Its 26 skips are conditional rather than disabled — a spec that needs a processed material or a live interview session skips itself when the fixture data is absent, which keeps an environment-dependent scenario from being reported as a failure of the code. Ten of the

52 are accessibility scans, one per learner, teacher and administrator route, each asserting that the page carries no axe violation of serious or critical impact.

Table 6.4 breaks the backend figure down by package. Statement coverage is reported per area rather than the branch-inclusive figure, because statements aggregate exactly across files while branch percentages do not; the 85.8% headline above is coverage.py's own branch-inclusive total for the whole package.

Table 6.4: Backend tesing coverage by package.

Package	**Files**	**Stmts**	**Missed**	**Cover**
features/interviews	154	13,156	1,880	85.7%
features/quizzes	91	7,552	819	89.2%
features/courses	30	4,551	597	86.9%
ai	58	3,663	325	91.1%
features/materials	33	2,606	273	89.5%
features/career_paths	15	1,936	273	85.9%
features/admin	30	1,926	133	93.1%
features/access_control	13	1,682	180	89.3%
features/learning_programs	7	1,497	142	90.5%
core	30	1,470	105	92.9%
features/identity	18	1,281	84	93.4%
features/spaced_repetition	18	1,229	120	90.2%
features/enrollments	15	687	83	87.9%
features/notifications	11	593	77	87.0%
features/progress	13	575	64	88.9%
features/discussions	7	428	67	84.3%
features/policies	6	399	26	93.5%
infrastructure	5	317	31	90.2%
api	1	185	50	73.0%
access_control	3	153	13	91.5%

Continued on next page

Table 6.4 – Continued from previous page

Package	Files	Stmts	Missed	Cover
workers	3	59	0	100.0%
Total	**561**	**45,945**	**5,342**	**88.4%**

The distribution follows the risk rather than the line count. Identity, configuration policy, the administrative surface and the shared `core` layer sit above 92%, being small, widely depended upon, and cheap to exercise. The interview feature carries the largest absolute gap because it is both the largest package and the one with the most real-time behaviour that cannot be reached without live media. The application factory in `api` is the lowest at 73%: it is composition rather than logic — router mounting and middleware registration — and its uncovered lines are the production-only branches that a test client never takes.

6.15.3 Continuous integration and excluded suites

The ordinary command is `uv run pytest`. In continuous integration the migrations are applied before a single coverage run:

```
uv run alembic upgrade head
uv run pytest --cov=abridgeai --cov-report=xml \
  --cov-report=term-missing --cov-fail-under=75
```

The CI environment provides PostgreSQL and Redis service containers, and the job sits alongside linting and security scanning as a required gate. The frontend equivalents are `npm run test:run` for the unit and component suite and `npm run test:e2e` for the browser suite.

Three things are deliberately outside the gated run. The `voice_live` marker guards an opt-in harness (`tests/integration/test_voice_live_harness.py`) that drives English and Vietnamese LiveKit scenarios end to end, checking audible agent output, persisted turns and cleanup; it requires `RUN_LIVEKIT_VOICE_TESTS=1`, LiveKit credentials and a live model endpoint, so it consumes paid external services and cannot be a per-commit gate. The `destructive` marker guards tests that reset shared state. The Playwright suite needs a browser and a running stack, so it is executed against the deployed environment rather than in the per-commit unit job; the end-to-end column of Table 6.3 reports that run. The resulting boundary is deliberate: deterministic state, persistence and race-safety

behaviour is verified continuously, while live STT, TTS and WebRTC validation remains an explicit operational test.

That boundary carries a cost worth recording, because it is the one this suite paid most visibly. An end-to-end assertion is reached only through a chain of environment — browser binaries, the backend's signing key and database, fixture state, and the rendered language — and a break anywhere along it surfaces as dozens of failing assertions rather than as one configuration error. Reading such a run therefore begins by asking whether the failures share a cause, not by treating them as a list of defects: in practice they rarely are one.

One limit should be stated plainly, because no quantity of passing tests addresses it. This suite establishes that the system behaves as specified; it does not establish that the specification improves learning. In particular the scheduling model of Appendix B is verified against its own definition — the derivation of Q, the ease-factor update, the interval rules and their boundary conditions — and not against retention outcomes, which would require a longitudinal cohort study outside this project's scope.

7 Evaluation

This chapter evaluates the aBridgeAI system across four dimensions: frontend user experience (Section 7.1), backend performance under load (Section 7.2), functional-requirement coverage (Section 7.3), and AI model comparison for quiz generation (Section 7.4).

7.1 Frontend Evaluation

Frontend quality is assessed with Google Lighthouse (v13, Chrome 153) against the deployed production site, auditing every page under both the mobile and the desktop emulation profiles. Five representative pages are audited: the public Login page and four authenticated learner pages (Student Dashboard, My Courses, Course Detail, and a submitted Quiz Attempt Review). Authenticated pages are audited with a dedicated test account whose long-lived session is minted with the application's own signing key, so every audit exercises the real authenticated rendering path rather than a login redirect.

Table 7.1 summarises the per-page scores and the average across pages; the full audit reports, including per-page metric details and final-view screenshots, are reproduced in Appendix I (Volume II).

Table 7.1: Lighthouse Audit Results (Lighthouse v13, Chrome 153, Production Deployment)

	Mobile emulation				**Desktop emulation**			
Page	**P**	**A**	**BP**	**SEO**	**P**	**A**	**BP**	**SEO**
Login	85	100	100	100	100	100	100	100
Student Dashboard	96	96	100	100	100	96	100	100
My Courses	98	100	100	100	100	100	100	100
Course Detail	94	96	100	100	100	96	100	100
Quiz Attempt Review	96	95	100	100	100	95	100	100
Average	**93.8**	**97.4**	**100**	**100**	**100**	**97.4**	**100**	**100**

Mobile emulation applies Lighthouse's default constrained profile (slow 4G throttling, 4× CPU slowdown, 412 px viewport); desktop emulation applies the lighter desktop profile (40 ms RTT, 10 Mbps, unthrottled CPU). Under the desktop profile every page scores a perfect 100 in Performance, Best Practices, and SEO, and every page achieves a perfect SEO and Best Practices score under both profiles — the latter reflecting the crawlability,

canonical, and indexing fixes applied during the evaluation iteration. Two audit findings surfaced by the initial dashboard run (a 0.43 cumulative layout shift caused by a collapsing loading skeleton, and the invalid canonical URL on deep links) were fixed during this evaluation and re-measured at zero shift and full SEO score respectively. These automated results support the audited frontend configuration and web-platform practices, but do not independently establish WCAG 2.1 AA conformance without complementary keyboard, contrast, and assistive-technology checks.

7.2 Backend Performance

Backend performance is evaluated with k6 against the deployed system. The methodology deliberately replaces the read-only, single-endpoint scenarios of the capstone project with a multi-user mixed read/write profile tied directly to NFR-2.1, and the evaluation doubles as a remediation loop: three deployment configurations are measured in sequence, each bottleneck found is fixed, and the fix is verified by re-running the identical load.

7.2.1 Test Environment

- **Server**: development machine; PM2-managed Uvicorn serving the FastAPI application, with PostgreSQL 16 (pgvector), Redis 7, Neo4j 5, Garage S3 and LiveKit in Docker
- **Load generator**: k6 v2.3 running from a separate machine on the same LAN, over HTTPS to the production domain — TLS termination included, unlike the previous iteration's plain-HTTP LAN runs
- **Authentication**: 200 synthetic students seeded on the deployment (organisation membership, course enrolment, and 30-day JWTs minted with the application's own signing key), so every virtual user authenticates as a *distinct* student; Google OAuth remains out of scope as an external dependency
- **Configurations measured**, in remediation order:
  - **Config A** — one Uvicorn worker, PostgreSQL at its default `max_connections=100` (the specialized-project deployment)
  - **Config B** — four Uvicorn workers (identical in every other respect)
  - **Config C** — four workers with PostgreSQL `max_connections=300`

7.2.2 Test Scenarios

The peak profile models a live class session. Each level sustains for three minutes with a 2 s think time:

- **Browse (80% of virtual users)**: profile bootstrap, enrolled-course list, notification-badge polling, course progress, spaced-repetition state, and curriculum pages — the per-request pattern of the learner dashboard and course-learn views.
- **Quiz taking (20%)**: the full write journey against one published quiz — start attempt, per-question answer writes (each firing the SM-2 review in-transaction), session heartbeats, integrity-event batches, and final submission with grading. This is precisely the write path and the concurrent-attempts-on-one-quiz case the previous evaluation did not represent.
- **Multiplier matrix**: 1× (50 virtual users — the NFR-2.1 target), 2× (100) and 4× (200), to evidence headroom rather than a single point.

Because every virtual user holds a distinct student token, concurrent attempts contend as they would between real students; the single-session guard is exercised rather than tripped artificially by a shared identity.

Two clauses of NFR-2.1 are deliberately outside this evaluation and are treated as future work (Section 8.3): the ten simultaneous video streams, because no media material is published on the deployment to stream; and the five concurrent AI interview sessions, because each session consumes LLM gateway tokens and enqueues evaluation jobs, making load generation a cost decision rather than a measurement one. The evaluation therefore evidences the concurrent-active-users clause of the NFR over the HTTP control plane.

7.2.3 Results

Table 7.2 presents the scaling matrix across the three configurations, and Table 7.3 the per-endpoint detail at the NFR target level under the final configuration. Raw k6 outputs are archived in the backend repository under `perf/k6/results/`.

7.2.4 Analysis against NFR Targets

- **Bottleneck progression (evidence-driven remediation).** Under Config A the throughput ceiling is 35–37 requests per second regardless of offered load. Server telemetry during the 4× run shows the single Uvicorn process at 110% CPU (one

Table 7.2: Peak-profile scaling matrix (3 s ramp, 3 min sustained per level; browse/quiz split 80/20)

Config	Level	VUs	RPS	p50	p95	p99	Err%
A (1 worker)	1×	50	37.1	661 ms	2.66 s	5.85 s	0.00
A (1 worker)	2×	100	34.7	2.06 s	5.64 s	9.29 s	0.00
A (1 worker)	4×	200	35.5	4.46 s	10.83 s	14.47 s	0.00
B (4 workers)	1×	50	111.0	73 ms	464 ms	731 ms	0.00
B (4 workers)	2×	100	110.9	329 ms	1.73 s	4.13 s	0.00
B (4 workers)	4×	200	112.6	413 ms	5.94 s	11.37 s	0.55
C (4wk + PG300)	4×	200	111.5	539 ms	5.34 s	11.92 s	0.00

Table 7.3: Per-endpoint results at the NFR-2.1 target level (Config B/C, 50 VUs, 3 min, 23,473 requests, 0% errors)

Endpoint	RPS	p50	p95	p99	Max
`GET/me/notifications/unread-count`	25.9	44 ms	311 ms	608 ms	943 ms
`GET/users/me`	13.0	64 ms	386 ms	645 ms	1.09 s
`GET/me/courses`	13.0	143 ms	522 ms	739 ms	1.07 s
`GET/me/progress/courses/{id}`	13.0	56 ms	376 ms	600 ms	1.08 s
`GET/me/cards-due`	13.0	45 ms	326 ms	591 ms	862 ms
`POST/attempts/{id}/answers`	10.2	108 ms	558 ms	883 ms	1.21 s
`POST/attempts/{id}/session/heartbeat`	4.1	33 ms	253 ms	504 ms	828 ms
`POST/quizzes/{id}/attempts`	2.0	185 ms	727 ms	938 ms	1.20 s
`POST/attempts/{id}/submit`	2.0	125 ms	647 ms	936 ms	1.11 s
`GET/me/sr-dashboard-summary`	2.5	110 ms	564 ms	846 ms	1.67 s

core) while the machine idles at 71.8% and PostgreSQL sits at 10% — the process model, not the database, was the wall. Moving to four workers (Config B) lifted the ceiling to 111–113 requests per second (≈ 3×, near-linear per worker) and cut p50 at the NFR level from 661 ms to 73 ms (9×). At 4× overload, Config B then surfaced the *next* bottleneck: 0.55% of requests failed with HTTP 500 traced to `psycopg.OperationalError` — PostgreSQL rejecting connections because four workers (× pool 10 + overflow 10 each) plus the background worker and agents exhausted the default `max_connections=100`. Raising it to 300 (Config C) eliminated the failures: the identical 4× run reports 0.00% errors and zero HTTP 500s, with throughput unchanged (≈111 RPS) because the remaining saturation is worker CPU, not connection scarcity.

- **NFR-2.1: concurrent-active-users clause satisfied at target, with measured headroom.** At the NFR's own level (50 concurrent active users, driven harder than human pacing at a 2 s think time), the final configuration sustains 111 requests per second with p50 73 ms, p95 464 ms, 0% errors and a 1.67 s maximum. The NFR level consumes roughly 45% of measured capacity; latency begins degrading near 2×, and the first request failure appears only at 4× — and disappears again once the connection ceiling is lifted. The video-stream and AI-interview-session clauses of the NFR remain untested for the reasons stated above.

- **NFR-1.4 (dashboard within 2 s) and NFR-2.2 (queries within 0.5 s).** Both pass at the NFR level under the final configuration: every endpoint's p50 is below 400 ms and the slowest observed request is 1.67 s. At 2× the aggregate average remains near 0.5 s; at 4× both are exceeded — degradation beyond design load is graceful (queueing) rather than failure.

- **TC-NFR-2.1 (excess requests handled without crashing).** Across every configuration and level — including 4× the target on a single development machine — no process crashed, no restart occurred, and the sole failure mode observed (database connection exhaustion) was bounded (0.55%), diagnosed, and removed by configuration.

- **Comparison with the specialized-project results.** The previous iteration reported 136–431 requests per second on single-endpoint authenticated GETs; the two figures are not comparable. The earlier profile exercised shared, cacheable read paths with one identity, whereas the final profile is per-user (200 distinct students,

cold per-user caches) and includes the quiz write journey with in-transaction SM-2 scheduling. The final numbers are the more conservative and the more realistic of the two.

7.3 Requirements Scope

This section distinguishes the Functional Requirements Specification (FRS) prepared for the Capstone Project phase from the final Capstone Project FRS. The Capstone Project FRS contains 57 functional requirements (FRs), represented by Modules 1–4 in Appendix A (Volume II). The Capstone Project FRS expands the specification to 106 FRs. Its additional Bonus section contains 49 FRs in Modules 5–8.

Table 7.4 compares the two requirements baselines. The increase reflects broader capstone scope.

Table 7.4: Functional Requirements Scope: Specialized Project and Capstone Project

Requirements baseline	**FR count**	**Interpretation**
Specialized Project FRS	57	Requirements in Modules 1–4 of Appendix A (Volume II).
Bonus	49	Additional requirements in Modules 5–8 of Appendix A (Volume II).
Capstone Project FRS	106	Full final specification: Specialized Project FRS plus Bonus requirements.

7.4 AI Model Comparison for Quiz Generation

The quiz generation pipeline supports multiple LLM backends via the LLM Gateway abstraction. This section evaluates model performance for the quiz generation task across quality, latency, and cost dimensions to determine the optimal production default.

7.4.1 Methodology

Three candidate models were compared on a benchmark of two lessons from the *Data Warehousing & Data Mining* course (Chapter 1: Overview, Chapter 2: Basic Issues), running each model 3 times × 12 questions × 2 lessons = 72 expected questions per model. A JSON-repair fallback (via `gpt-5-chat-latest` with a structure-only system

prompt) recovers malformed outputs, bringing all models to full sample size ($n = 72$ for gpt-oss and gemma, $n = 77$ for llama due to over-generation).

Generation parameters

- Temperature = 0.3, top_p = 0.95
- 12 questions per lesson, fixed Bloom taxonomy mix (60% remember, 30% understand, 10% apply)
- Same prompt template, same retrieved chunks, same JSON schema across all candidates
- Direct LLM gateway call (bypassing ARQ) to isolate the model effect from queue latency

Judge protocol

An independent judge model (`gpt-5-chat-latest`) evaluated each generated question using a **reference-guided chain-of-thought** protocol [52]:

1. **Independently solve** the question using only the source chunks.
2. **Compare** the independent answer with the marked-correct option and the candidate's explanation.
3. **Score** the rubric only after Steps 1–2.

This protocol reduces judge failure rates from 14/20 (default prompt) → 6/20 (plain CoT) → 3/20 (reference-guided), as reported in Table 4 of Zheng et al. [52]. Anti-bias guardrails counter position, verbosity, and self-enhancement biases catalogued in [52, 53].

The judge scores each question on:

- **Groundedness** (strict: explicit textual support required from source chunks)
- **Answer correctness** (does the marked-correct option match the source material?)
- **Judge-solve match** (fraction where the judge's independent answer matches the marked-correct option — structural reliability check)
- **Distractor plausibility** (1–5 scale: 1 = trivially eliminable, 5 = requires deep understanding to distinguish)

- **Overall quality** (1–10 single-answer scale per Zheng et al. [52] Figure 6)
- **Schema compliance** (valid JSON output parseable by the pipeline)

Confidence intervals are 95% non-parametric bootstrap (2,000 resamples at question level). Decoding: temperature = 0.0, greedy (standard for LLM-as-a-judge stability [52]).

7.4.2 Quality Results

Table 7.5 presents the quality evaluation results under the reference-guided judge protocol.

Table 7.5: Quiz Generation Quality by Model (judge: `gpt-5-chat-latest`, reference-guided CoT)

Metric	gpt-oss-120b	gemma-4-31b-it	llama-3.3-70b
Sample size (n)	72	72	77
Groundedness (strict)	94.4%	**100%**	77.9%
Answer correctness	93.1%	**100%**	84.4%
Judge-solve match	93.1%	**100%**	84.4%
Distractor plausibility	**4.07**	3.86	3.97
Overall quality (1–10)	8.62	**8.97**	8.04
Schema compliance	**100%**	**100%**	**100%**

Remark. gemma-4-31b-it achieves perfect scores on all fidelity metrics — the judge independently arrived at gemma's answer for every question (100% solve-match), the strongest reliability signal available under reference-guided judging. gpt-oss-120b's 5.6 pp gap to gemma is attributable to paraphrasing depth rather than factual hallucination: when the judge could not solve a question, it also flagged it as not-grounded.

7.4.3 Latency and Cost

Table 7.6 presents latency per lesson (12 questions generated per call) and cost per 100 questions.

Remark. gpt-oss-120b dominates on both latency (4× faster) and cost (2.2× cheaper than gemma). Per-question latency of 2.7 s enables real-time generation feedback, whereas gemma's 10.8 s requires an asynchronous queue pattern.

Table 7.6: Quiz Generation Latency and Cost by Model

Metric	gpt-oss-120b	gemma-4-31b-it	llama-3.3-70b
Mean latency / lesson	**32.6 s**	125.6 s	81.6 s
p99 latency / lesson	**43.5 s**	140.9 s	95.0 s
Per-question latency (p50)	**2.7 s**	10.8 s	7.7 s
Cost / 100 questions	**$0.0077**	$0.0172	$0.0100

7.4.4 Failure Analysis

Each model exhibits distinct failure patterns under the reference-guided judge:

gpt-oss-120b

(production default) Primary failure mode is *paraphrase-depth*: the model rephrases source material at a level the judge's Step-1 solve cannot independently reproduce from the chunks alone (≈5.6% of questions). These are not factual hallucinations — the judge's groundedness and solve-match failures overlap perfectly, confirming the issue is inference depth rather than content drift.

gemma-4-31b-it

(highest fidelity) 'Zero quality failures across all dimensions. The trade-off is distractor plausibility (3.86 vs gpt-oss's 4.07): gemma writes distractors that are *too obviously wrong* for a learner familiar with the source, reducing exam discrimination power. One raw output required JSON structural repair (trailing prose after closing brace), recovered by the repair pass.

llama-3.3-70b

(not recommended for production) Exhibits *external-knowledge contamination* in ≈22% of questions: the model injects pre-training knowledge (e.g., naming specific vendors not in the source), overgeneralises concepts beyond source scope, or confabulates examples. The reference-guided judge correctly identifies these because its Step-1 solve fails to land on llama's marked-correct option.

7.4.5 Recommendation

Production default: gpt-oss-120b. Fastest (4×), cheapest (2.2×), schema-perfect, highest distractor plausibility. Its 94.4% strict groundedness approaches gemma's 100%

under reference-guided judging — the gap is paraphrasing style, not factual error.

High-stakes fallback: gemma-4-31b-it. For graded assessments where any ungrounded content is unacceptable, gemma's perfect fidelity justifies the latency and cost premium. Schema fragility is mitigated by the JSON-repair pipeline.

All three models under-deliver on Bloom's `apply`-level questions (target 10%, actual <2%). This is addressable at the prompt level with worked examples, independent of model choice.

8 Conclusion

This chapter summarises what the aBridgeAI platform delivers, states the limitations of both the implementation and the evaluation behind it, and records the work that remains.

8.1 Achievements

The platform implements the complete learning loop proposed in Chapter 6, from content ingestion through to programme-level progression. The following capabilities are operational and have been verified through integration testing:

1. **Authentication and Access Control** — Google SSO sign-in and logout, session management with rotating refresh tokens held as keyed digests, two-factor authentication with TOTP and recovery codes, freshness enforcement on sensitive operations, and fully dynamic role-based access control in which permissions are catalogue-driven rather than hardcoded. Assignments resolve across four scopes — global, organisation, organisational unit, and course — with containment decided by a recursive walk of the unit tree.

2. **Multi-tenant Organisation Management** — Tenant provisioning, organisational-unit hierarchies rooted at faculties, staff affiliation, membership lifecycle, domain attachment with optional auto-provisioning, individual invitation that creates account, membership and role in one transaction, and bulk student enrolment from a CSV roster with per-row validation that reports failures without aborting the batch. One account belongs to one organisation, enforced by a partial unique index rather than by convention alone.

3. **Course and Curriculum Management** — Courses moving through draft, published and archived states, subdivided into modules that hold lessons, quizzes and interviews as sibling curriculum items under a uniform ordering. Prerequisites at module and lesson level, syllabus import that materialises a module tree in one operation, invitation-code self-enrolment, and entitlement recorded separately from enrolment so that a programme can grant and withdraw access without disturbing a course reached independently.

4. **Multi-format Material Ingestion** — An extraction registry covering documents, presentations, HTML, plain text, source code, images, and transcribed audio and

video, feeding chunking, embedding and indexing. Materials are immutable versions with a preprocessing quarantine whose teacher decisions survive re-ingestion, and the current-version flag moves only once the new version is indexed, so a failed re-ingest leaves the previous text queryable rather than the material unavailable.

5. **AI Generation Pipelines** — Hybrid retrieval combining dense vector and sparse full-text search, fused by reciprocal rank, diversified by maximal marginal relevance, and filtered by the requester's access before reaching the model gateway. A course knowledge graph supplies structured concept context alongside retrieved chunks. Every generated artefact is written pending and requires a teacher to approve, edit or reject it before any learner sees it; the original and the edited form are both retained.

6. **Adaptive Quizzing** — Attempt delivery with per-attempt shuffling, configurable timing and retake policy, automatic resolution of attempts left open past a deadline, automatic marking with a manual queue for what cannot be marked confidently, a single explainable grade of record, and per-student and per-group accommodations that merge by generosity. Integrity signals are recorded under a policy frozen at attempt start, a live attempt admits one client at a time, and a quiz may require a live camera.

7. **Spaced Repetition and Progression Gating** — The SM-2 core with calibrated early ease-factor dampening, quality derived from observed correctness, hint usage and response-time ratio rather than self-assessment, interval jitter to prevent cohort pile-up, and a due-card scan that raises review reminders. Lesson unlock combines prerequisite completion, the ease-factor floor across a required coverage proportion, and where configured a passing interview.

8. **Mock Interviews** — Teacher-defined outcomes, a course-scoped question bank with sibling variants and semantic duplicate detection, staged onboarding that ensures an equipment fault is never scored as a knowledge gap, adaptive turn-taking in text and voice with real-time transcription and synthesis, per-turn perception that degrades safely rather than breaking a session, prompt-injection and output-leakage guards with a redacted audit trail, and single-reservation evaluation producing a binary verdict with a teacher-facing gap report.

9. **Learning Programmes and Path Governance** — Versioned career pathways organised into stages, learning programmes that bundle them under a faculty, enrol-

ment individually or in bulk against an organisation-configurable ceiling, pathway selection, and dean-adjudicated change and drop requests with a fixed rejection vocabulary. An approval snapshots outgoing progress and preserves earned course completions as awards, so work shared between pathways is credited once and never repeated.

10. **Notifications and Discussions** — A categorised notification inbox with per-category, per-channel preferences over in-app and e-mail delivery, localised message construction, and dispatch isolated so that a delivery failure can never roll back the academic record that raised it. Threaded lesson discussions with soft deletion that preserves thread structure.

11. **Analytics and Reporting** — Learner progress and retention views, teacher cohort analytics identifying at-risk learners and systematic retention gaps, organisation-scoped career-readiness snapshots with drill-down, and administrative observability covering platform statistics, processing-job inspection, and AI cost telemetry attributable by user and pipeline.

12. **Platform Configuration, Policies and Audit** — A closed registry of runtime-tunable settings resolved through four precedence levels with all candidates reported, dry-run preview, mandatory change reasons, and rollback. Versioned policy documents served publicly where their audience is unrestricted. Two audit trails whose append-only guarantee is enforced by database trigger rather than application code.

13. **Web Client** — A React and TypeScript single-page application with typed routing, server state held solely in a query cache with registry-declared invalidation, request and response types generated from the backend's OpenAPI schema and checked at build time, a tokenised design system supporting light and dark themes, and full English and Vietnamese localisation.

8.2 Limitations

The limitations below are separated into two kinds, because they carry different weight. The first concern what the system does not yet do; the second concern what this work has not yet established, and it is the second that bounds the claims this report is entitled to make.

Limitations of the implementation

- **Assessment integrity is deterrent, not proof.** A live attempt is bound to one client through a session-scoped key rather than a durable lease, so a copied token carrying the same session identifier cannot be distinguished from the original client, and a modified client can ignore the browser-side coordination entirely. Where a camera is required, the system verifies only that a video track is live; nothing is captured, so the requirement discourages casual misconduct rather than evidencing its absence. Integrity signals are collected under one fixed policy with fixed weights, which cannot be tuned to the stakes of a particular assessment.

- **Code answers cannot be marked automatically.** `code` is an accepted question type, but a submission is stored as text and routed to a teacher for manual marking. The platform therefore grades its most objectively checkable assessment type by its least scalable means.

- **Administrative movement is constrained.** An account belongs to at most one organisation and there is no transfer workflow, so relocating a person between institutions means discarding their history and re-creating them. The organisational-unit tree supports arbitrary depth, but the administrative and reporting surfaces assume a flat membership beneath a faculty.

- **Institutional obligations are not yet met.** The platform holds identifiable student records but offers no path to export everything held about one person, nor to erase it on request while preserving aggregate reporting.

Limitations of the evaluation

- **No learning-outcome study.** The pedagogical premise of this work — that algorithmically derived quality scoring and spaced scheduling improve durable retention — has not been tested against learners. The scheduler is shown to behave as specified; it is not shown to teach better than the alternative it replaces. Establishing that requires a controlled study over a cohort and a term, which this project's timeframe did not permit.

- **Media and AI-pipeline load coverage.** The final load evaluation exercises the HTTP control plane — per-user reads, the quiz write journey, concurrent attempts, session guards — under a 200-student synthetic cohort, and it drove two deployment remediations (four Uvicorn workers; PostgreSQL `max_connections` 300) that were

each verified by re-running the identical load. It does not load-test media delivery or the AI pipelines: LiveKit WebRTC streams, video playback, and AI interview sessions are absent because no media material is published on the deployment and each interview session consumes LLM gateway tokens. NFR-2.1's video-stream and interview-session clauses therefore remain asserted rather than measured, and the remediation chain was validated only to the point of worker-CPU saturation on a single development machine rather than through a connection-pooling layer.

- **Generation quality is model-assessed.** Quiz quality was scored by an independent judge model under a reference-guided protocol with anti-bias guardrails and bootstrap confidence intervals. That protocol is well founded and materially reduces judge error, but it remains an automated proxy: no subject-matter expert has rated the generated questions, and no learner has sat them. Groundedness and distractor plausibility as measured are properties the judge recognised, not outcomes observed in teaching.

- **Multi-tenant operation is unvalidated at scale.** Organisation provisioning, bulk enrolment, and domain-based auto-provisioning are implemented and integration-tested, but have not been exercised with the number of concurrent tenants, users, or stored materials that an institutional deployment implies.

8.3 Future Work

The work below is recorded as scope rather than as a schedule: each item states what remains to be built and why the system needs it, without committing to an ordering or an allocation. Several follow directly from boundaries stated earlier in this report; the rest are capabilities the present design accommodates but does not yet provide.

Domain identity and records

- **Human-readable business codes for domain entities.** Students, organisations, courses, learning programmes and career paths are identified internally by opaque UUIDs. Institutions administer these entities by their own codes — a student number, a course code, a programme code — and reconciling platform records against a registrar's system currently depends on matching names. Each entity needs a business code that is unique within its scope, stable across edits, and usable in import, search, and export.

- **Data export and erasure.** The platform holds personal data and assessment records for identifiable students, but provides no path to export everything held about one person, nor to erase it on request while preserving the aggregate record. Both are ordinary obligations for an institutional deployment and neither exists today.

Assessment integrity

- **Configurable integrity policy.** Integrity signals are presently recorded under one fixed policy with fixed weights. Institutions differ in how strictly they invigilate, and a formative weekly quiz does not warrant the treatment of a summative examination. What is needed is a security mode selectable per quiz or per organisation, governing which signals are collected, how they are weighted, and what threshold constitutes a flag.

- **Retained and reviewable video.** A quiz may require a camera, but the current implementation only tests that a video track is live; nothing is captured. Making the recording retrievable for review, or analysing it for indications of misconduct, is a substantial addition rather than an extension: it introduces storage, retention, access-control, consent, and jurisdictional obligations that the present design deliberately avoids by never holding the media at all. Those obligations must be settled before the capability is built, not after.

- **Persistent assessment lease.** Ownership of a live quiz attempt is bound to an authentication session in Redis, which is enough to separate two ordinary browsers but not a durable lease. A copied token carrying the same session identifier is indistinguishable from the original client; losing Redis loses ownership; and a modified client can ignore the coordination entirely. Closing these gaps requires a lease shared by quizzes and interviews, with opaque per-client credentials, generation counters, audit history, invalidation on a cache reset, and enforcement extended into the interview transport.

Assessment authoring and delivery

- **Controlled answer-key correction and downstream regrading.** Published quizzes are intentionally frozen in the current release, so a Teacher cannot safely correct an answer key after attempts have been submitted and the Regrade action is not exposed in the interface. A complete workflow must permit only the answer

key — not the prompt, option text or question structure — to change through a reasoned, revisioned correction; preview the affected answers, attempts, grades and completion states; and apply the change only after explicit confirmation. Commit must recompute the grade of record and Course completion, latch newly eligible pathway stages without revoking historical stage completion or awards, reconcile the affected spaced-repetition reviews and ease factors without double-counting them, refresh downstream assessment and progress views, and retain the correction and its effects in the audit trail.

- **Code questions with execution.** `code` is already an accepted question type, but a submitted answer is stored as text and routed to a teacher for manual marking. What is missing is the surface and the machinery around it: an editor with language support in the learner's workspace, and a sandboxed runner that executes a teacher-authored test suite against the submission and returns the result as the mark. This turns the platform's weakest assessment type — the one that cannot currently be graded automatically at all — into its most objective.

Collaboration

- **Media in discussions.** Lesson discussions accept text only. A learner asking about a diagram, an error message, or a rendering problem is usually trying to show something, and describing it in prose is both laborious and lossy. Attachments need the same storage, access-control, and moderation treatment that learning materials already receive rather than a separate path.

Organisation and pathway administration

- **Transferring an account between organisations.** An account belongs to at most one organisation, and this release provides no transfer: an account is offboarded by ending its membership, and a genuine move must be performed as a deletion and a re-creation, which discards the person's history. A transfer must be an explicit, audited transaction that reconciles roles, faculty assignments, enrolments, content ownership, and reporting scope in one step.

- **Stronger constraints on pathway changes, with automatic rejection.** Every pathway-change request currently reaches a dean, including those that could never be approved. Requests that fail a determinable rule — an exhausted switch allowance, a target outside the enrolled programme version, an archived target, a

request duplicating an open one — should be refused at submission with the reason shown to the student, leaving deans only the requests that genuinely require academic judgement. Institutions should additionally be able to express their own eligibility rules, such as a minimum progress threshold or a deadline in the academic calendar.

- **Deeper faculty hierarchies.** The organisational-unit tree supports faculties, departments and further sub-units, but the administrative surfaces and the reporting views assume a faculty with a flat membership beneath it. Institutions organise themselves more deeply than that — divisions within departments, programme groups within divisions — and both dean authority and cohort reporting need to resolve correctly at any depth rather than only at the top.

Platform configuration

- **Richer setting types.** The runtime settings registry admits boolean, integer and floating-point values only. Operational knobs that are naturally textual or enumerated — a default locale, a provider selection, a retention policy name — cannot be expressed and must therefore remain deployment constants, which is the very coupling the registry exists to remove.

Performance at media scale

- **Load-testing the media and AI clauses of NFR-2.1.** The load evaluation covers the HTTP control plane up to four times the NFR's concurrent-user target, but the NFR's other two clauses — ten simultaneous video streams and five concurrent AI interview sessions — remain unmeasured. Verifying them needs a deployment with published video material, a LiveKit media path driven by real WebRTC clients or a media-capable load generator, and an LLM budget for interview sessions, none of which the current development deployment provides. The interview control plane (session start, token minting, onboarding, state polling) is already scripted and cost-gated in the load suite, so extending coverage is a matter of environment rather than new tooling.

- **A connection-pooling layer before further worker scaling.** The measured bottleneck progression — single worker CPU, then PostgreSQL `max_connections`, then worker CPU again — was cleared by configuration each time. Scaling the worker count materially beyond four should first introduce a pooling layer such

as PgBouncer between the application and PostgreSQL, so that capacity growth spends database connections linearly rather than per-worker, and should re-run the archived peak profile to locate the next wall.

8.4 Concluding Remarks

aBridgeAI set out to close the distance between what a curriculum certifies and what an employer interviews for, and the platform now implements that loop end to end. Material uploaded by a teacher becomes a retrieval corpus; that corpus grounds generated assessment that no learner sees until a teacher has approved it; performance on that assessment drives a scheduler that decides when the material should be seen again; the resulting card state gates progression; and a mock interview asks the learner to explain what the quiz established they could recognise. Around that loop sits the governance structure an institution actually needs — faculties, versioned career pathways, programme enrolment, and dean-adjudicated pathway change — so that progress means something to a registrar and not only to a dashboard.

Two commitments shaped the design more than any individual feature, and both were kept. The first is that a model never decides anything a teacher is accountable for: every generated artefact is a draft awaiting human approval, and the retrieval work exists to make review cheap rather than to make review unnecessary. The second is that any rule which judges a learner is frozen at the moment it starts applying to them — the integrity policy onto the attempt, the pathway version onto the enrolment, the programme version onto the student — so that editing a curriculum can never retroactively change what somebody already under way is measured against. Both cost more to build than their absence would have, and both are the reason the system's outputs can be defended to the person they affect.

What remains is honest to state plainly. The engineering is in place and behaves as specified; the pedagogical claim behind it is not yet demonstrated. Whether deriving the quality signal from observed behaviour rather than self-assessment produces better retention than standard SM-2 is a question that needs a cohort, a term, and a control group, and answering it is the natural continuation of this work. The contribution offered here is a platform on which that question can now be asked — one where the scheduling state, the assessment record, and the progression decision are all recorded well enough to be studied.

References

[1] Mathura Shanmugasundaram and Arunkumar Tamilarasu. "The impact of digital technology, social media, and artificial intelligence on cognitive functions: a review". In: *Frontiers in Cognition* Volume 2 - 2023 (2023). ISSN: 2813-4532. DOI: 10.3389/fcogn.2023.1203077. URL: https://www.frontiersin.org/journals/cognition/articles/10.3389/fcogn.2023.1203077.

[2] Studio MG. *Cognitive Load and AI in Education*. Accessed: 2026-03-08. n.d. URL: https://www.studio.mg/articles/cognitive-load-and-ai-in-education.php.

[3] Richard E. Mayer and Roxana Moreno. "Nine Ways to Reduce Cognitive Load in Multimedia Learning". In: *Educational Psychologist* 38.1 (2003), pp. 43–52. DOI: 10.1207/S15326985EP3801_6.

[4] Alexander Renkl and Robert Atkinson. "Structuring the Transition From Example Study to Problem Solving in Cognitive Skill Acquisition: A Cognitive Load Perspective". In: *Educational Psychologist - EDUC PSYCHOL* 38 (Mar. 2003), pp. 15–22. DOI: 10.1207/S15326985EP3801_3.

[5] Wozniakand and Piotr A. *The True History of Spaced Repetition*. SuperMemo website. June 2018. URL: https://www.supermemo.com/en/blog/the-true-history-of-spaced-repetition.

[6] Lorna Smith Benjamin. *Interpersonal Diagnosis and Treatment of Personality Disorders*. 2nd ed. New York: Guilford Press, 1996.

[7] Junyao Ye, Jingyong Su, and Yilong Cao. "A Stochastic Shortest Path Algorithm for Optimizing Spaced Repetition Scheduling". In: *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22)*. Association for Computing Machinery, 2022, pp. 4381–4390. DOI: 10.1145/3534678.3539081. URL: https://doi.org/10.1145/3534678.3539081.

[8] Jiahao Zhao. *LECTOR: LLM-Enhanced Concept-Based Test-Oriented Repetition for Adaptive Spaced Learning*. 2025. URL: https://arxiv.org/pdf/2508.03275.

[9] Duolingo, Inc. *halflife-regression*. https://github.com/duolingo/halflife-regression. GitHub repository, accessed 2026-03-08. 2016.

[10] Open Spaced Repetition. *anki-sm-2: Python implementation of Anki's SM-2 spaced repetition scheduler*. https://github.com/open-spaced-repetition/anki-sm-2. GitHub repository, accessed 2026-03-08. n.d.

[11] Sven Greving and Tobias Richter. "Examining the Testing Effect in University Teaching: Retrievability and Question Format Matter". In: *Frontiers in Psychology* 9 (2018), p. 2412. DOI: https://doi.org/10.3389/fpsyg.2018.02412.

[12] Anki. *Anki*. https://apps.ankiweb.net. Accessed: 2026-03-16. 2026.

[13] Quizlet. *Quizlet*. https://quizlet.com. Accessed: 2026-03-16. 2026.

[14] Instructure. *Canvas LMS*. https://www.instructure.com/canvas. Accessed: 2026-03-16. 2026.

[15] Moodle. *Moodle*. https://moodle.org. Accessed: 2026-03-16. 2026.

[16] Duolingo. *Duolingo*. https://www.duolingo.com. Accessed: 2026-03-16. 2026.

[17] John Sweller, Jeroen J. G. van Merriënboer, Fred and Paas. "Cognitive Architecture and Instructional Design: 20 Years Later". In: *Educational Psychology Review* 31 (2019), pp. 261–292. DOI: https://doi.org/10.1007/s10648-019-09465-5.

[18] Drew Perkins. *Cognitive Load Theory Revisited*. We Grow Teachers, accessed 2026-03-11. Oct. 2024. URL: https://wegrowteachers.com/cognitive-load-theory-revisited/.

[19] Björn de Koning. "Best of Both Worlds? Combining Physical and Mental Self-Management Strategies to Support Learning from Split-Attention Examples". In: *Education Sciences* 14 (Mar. 2024), p. 284. DOI: 10.3390/educsci14030284.

[20] Hermann Ebbinghaus. *Memory: A Contribution to Experimental Psychology*. Teachers College Press, 1885. DOI: https://doi.org/10.1037/10011-000.

[21] Wiseheart et al. "Distributed Practice in Verbal Recall Tasks: A Review and Quantitative Synthesis". In: *Psychological Bulletin* 132 (May 2006), pp. 354–380. DOI: 10.1037/0033-2909.132.3.354.

[22] Nate Kornell. "Optimising learning using flashcards: Spacing is more effective than cramming". In: *Applied Cognitive Psychology* 23.9 (2009), pp. 1297–1317. DOI: https://doi.org/10.1002/acp.1537.

[23] Piotr A. Woźniak, Edward J. Gorzelańczyk, and Jerzy A. Murakowski. "Two Components of Long-Term Memory". In: *Acta Neurobiologiae Experimentalis* 55.4 (1995), pp. 301–305. DOI: 10.55782/ane-1995-1090.

[24] Hermann Ebbinghaus. *Memory: A Contribution to Experimental Psychology*. Teachers College Press, 1885. DOI: https://doi.org/10.1037/10011-000.

[25] Henry L. Roediger and Jeffrey D. Karpicke. "Test-enhanced learning: Taking memory tests improves long-term retention". In: *Psychological Science* 17.3 (2006), pp. 249–255. DOI: 10.1111/j.1467-9280.2006.01693.x.

[26] Robert Bjork. "Memory and Meta-memory Considerations in the Training of Human Beings". In: Apr. 1994, pp. 185–205. ISBN: 9780262279697. DOI: 10.7551/mitpress/4561.003.0011.

[27] Jarrett Ye. *FSRS: Free Spaced Repetition Scheduler*. Accessed: 2026-03-14. 2023. URL: https://github.com/open-spaced-repetition/fsrs4anki.

[28] J. Martin Corbett. *Critical Cases in Organisational Behaviour*. Macmillan, 1994. ISBN: 9780333577516.

[29] Zachary A. Pardos and Neil T. Heffernan. "Modeling Individualization in a Bayesian Networks Implementation of Knowledge Tracing". In: (2010). Ed. by Paul De Bra, Alfred Kobsa, and David Chin, pp. 255–266.

[30] Chris Piech et al. *Deep Knowledge Tracing*. 2015. arXiv: 1506.05908 [cs.AI]. URL: https://arxiv.org/abs/1506.05908.

[31] Lalit Pandey. "Pandey et al., 2019". In: (Aug. 2020).

[32] Mohammad Khajah, Robert V. Lindsey, and Michael C. Mozer. *How deep is knowledge tracing?* 2016. arXiv: 1604.02416 [cs.AI]. URL: https://arxiv.org/abs/1604.02416.

[33] Kurt VanLehn. "The Behavior of Tutoring Systems". In: *International Journal of Artificial Intelligence in Education* 16.3 (2006), pp. 227–265. DOI: 10.3233/IRG-2006-16(3)02. URL: https://journals.sagepub.com/doi/abs/10.3233/IRG-2006-16%283%2902.

[34] Nelson Cowan. "The magical number 4 in short-term memory: A reconsideration of mental storage capacity". In: *Behavioral and Brain Sciences* 24.1 (2001), pp. 87–185. DOI: 10.1017/s0140525x01003922.

[35] Jochen Ranger. "A Note on the Hierarchical Model for Responses and Response Times in Tests of van der Linden (2007)". In: *Psychometrika* 78 (July 2013). DOI: 10.1007/s11336-013-9324-6.

[36] John Reeve and W. Lee. "Students' Classroom Engagement Produces Longitudinal Changes in Classroom Motivation". In: *Journal of Educational Psychology* 106 (2014), pp. 527–540. DOI: 10.1037/a0034934.

[37] Jakub Pokrywka et al. "Modeling Spaced Repetition with LSTMs". In: *Proceedings of the 15th International Conference on Computer Supported Education (CSEDU)*. 2023. DOI: 10.5220/0000173200003470.

[38] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, et al. "ChatGPT for good? On opportunities and challenges of large language models for education". In: *Learning and Individual Differences* 103 (2023), p. 102274.

[39] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.

[40] Congmin Min et al. *Towards Practical GraphRAG: Efficient Knowledge Graph Construction and Hybrid Retrieval at Scale*. 2025. URL: https://arxiv.org/pdf/2507.03226?.

[41] Northflank. *Why we ditched Next.js and never looked back*. Accessed: 2026-05-22. 2024. URL: https://northflank.com/blog/why-we-ditched-next-js-and-never-looked-back.

[42] Appwrite. *Why developers are leaving Next.js for TanStack Start, and loving it*. Accessed: 2026-05-22. 2024. URL: https://appwrite.io/blog/post/why-developers-leaving-nextjs-tanstack-start.

[43] TECHSY. *Next.js vs React + Vite 2026: Need a Framework?* Accessed: 2026-05-22. 2026. URL: https://techsy.io/en/blog/nextjs-vs-react-vite.

[44] Hygraph. *Vite vs. Next.js: A side-by-side comparison*. Accessed: 2026-05-22. 2024. URL: https://hygraph.com/blog/vite-vs-nextjs.

[45] TanStack. *TanStack Start vs Next.js Comparison*. Accessed: 2026-05-22. 2024. URL: https://tanstack.com/start/latest/docs/framework/react/comparison.

[46] Versatik. *WebRTC vs WebSockets: AI Voice Communications*. Accessed: 2026-03-13. 2025. URL: https://versatik.net/en/news/webrtc-websockets-future-ai-voice-voicebots.

[47] Tiger Data. *Pgvector vs. Qdrant*. Accessed: 2026-03-13. 2025. URL: https://www.tigerdata.com/blog/pgvector-vs-qdrant.

[48] Neo4j, Inc. *Why Graph Databases? Native Graph Processing and Storage*. Accessed: 2026-05-22. 2024. URL: https://neo4j.com/why-graph-databases/.

[49] Redis Ltd. *Redis Benchmark: Performance and Latency*. Accessed: 2026-05-22. 2024. URL: https://redis.io/docs/latest/operate/oss_and_stack/management/optimization/benchmarks/.

[50] Anthropic. *Introducing Contextual Retrieval*. https://www.anthropic.com/news/contextual-retrieval. Accessed: 2026-09-14. Sept. 2024.

[51] Jaime Carbonell and Jade Goldstein. "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries". In: *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '98)*. Association for Computing Machinery, 1998, pp. 335–336. DOI: 10.1145/290941.291025. URL: https://doi.org/10.1145/290941.291025.

[52] Lianmin Zheng et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena". In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 2023. URL: https://arxiv.org/abs/2306.05685.

[53] Haitao Li et al. "LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods". In: *arXiv preprint arXiv:2412.05579* (2024). URL: https://arxiv.org/abs/2412.05579.

[54] LiveKit, Inc. *LiveKit React SDK*. Accessed: 2026-05-22. 2024. URL: https://docs.livekit.io/reference/components/react/.

[55] The PostgreSQL Global Development Group. *PostgreSQL Documentation: Reliability and the Write-Ahead Log*. Accessed: 2026-05-22. 2024. URL: https://www.postgresql.org/docs/current/wal-reliability.html.

Appendices

Appendix A Technology Selection Comparisons

Section 5.1 states each technology decision and the constraint that drove it. The comparison matrices behind those decisions are collected here: one per component, scoring the option chosen against the alternatives that were seriously considered. They are reference material for a reader checking the reasoning rather than following it, which is why they sit apart from the argument they support.

A.1 Backend Framework

Table A.1: Backend Framework Comparison

Feature	**FastAPI**	**Django REST**	**Express.js**
Language	Python	Python	JavaScript / Node.js
AI/ML Integration	Native (LangChain, scikit-learn, NumPy)	Native but synchronous ORM bottleneck	Requires Python sidecar microservices
Concurrency Model	Async I/O (`asyncio`)	Sync-first (WSGI); async support limited	Single-threaded event loop
Performance	High (∼180MB cold start, async-native)	Moderate (∼250MB, sync overhead)	Moderate (∼120MB, CPU-bound limits)
Auto Schema / Docs	Built-in (OpenAPI, Swagger UI)	Requires `drf-spectacular` add-on	Requires manual setup

A.2 Frontend Framework

Table A.2: Frontend Framework Comparison

Feature	**Vite + React + TanStack Query**	**Next.js (App Router)**	**Angular**
Bundle Size	~42KB (minimal runtime) [43]	~92KB (RSC runtime overhead) [43]	Large (full framework)
HMR Speed	<50ms (native ESM) [44]	Slower (Turbopack improving) [44]	Moderate
Complexity	Low (standard React patterns)	High (RSC boundaries, App Router, Flight payload) [41]	High (Zone.js, decorators)
Data Fetching	Fine-grained (TanStack Query: cache, invalidation, optimistic updates) [45]	Coarse (fetch caching, requires manual React Query integration) [45]	RxJS (steep learning curve)
LiveKit SDK	Official React SDK [54]	Official React SDK [54]	No official SDK
SSR / SEO	Not needed (authenticated SPA)	Built-in (unnecessary overhead for this use case)	Angular Universal (complex)

A.3 Real-Time Communication

Table A.3: Real-Time Communication Comparison

Feature	LiveKit	Twilio	Socket.io
Protocol	WebRTC (UDP)	SIP / WebRTC	WebSockets (TCP)
Voice Latency	Ultra-low (<200ms) [46]	Moderate (cloud routing overhead)	High (TCP head-of-line blocking)
AI Agent SDK	Built-in (Agents SDK, VAD, STT)	Requires manual pipeline	No native support
STT Integration	Integrated (Whisper, Deepgram)	Separate service required	Separate service required
Self-Hostable	Yes (open-source)	No (vendor lock-in)	Yes

A.4 Primary Database and Vector Search

Table A.4: Primary Database & Vector Search Comparison

Feature	PostgreSQL + pgvector	Qdrant	Pinecone
Vector Throughput (99% Recall)	471.57 QPS [47]	41.47 QPS [47]	N/A (managed, opaque)
ACID Transactions	Full [55]	None	None
SQL + Vector in one query	Yes (JOIN + <-> operator)	No	No
Operational Overhead	Low (single service)	Moderate (separate cluster)	Low (managed SaaS)
Self-Hostable	Yes	Yes	No (vendor lock-in)

A.5 Graph Database

Table A.5: Graph Database Comparison

Feature	Neo4j	ArangoDB
Graph Traversal	Native (Cypher, index-free adjacency)	Native (AQL, multi-model)
Query Language	Cypher (declarative, readable)	AQL (verbose, general-purpose)
Plugin Ecosystem	APOC (graph algorithms, ML, path expansion)	Foxx microservices (custom JS)
Python Driver	Official (`neo4j` package)	Official (`python-arango`)
Community & Docs	Large, mature	Moderate

A.6 Caching and Task Queue

Table A.6: Caching & Task Queue Comparison

Feature	Redis	Memcached	RabbitMQ
Throughput	>100K ops/sec [49]	>100K ops/sec	Moderate (AMQP overhead)
Latency	<1ms [49]	<1ms	Higher (protocol overhead)
Data Structures	Rich (lists, sets, sorted sets, streams)	Key-value only	Queue-only
Task Queue Support	Yes (via `arq`, `Celery`)	No	Yes (native)
Persistence	Optional (RDB / AOF)	None	Yes

Appendix B Algorithm and Process Flow Specifications

This appendix carries the specification of the spaced-repetition scheduling model, which the body of the report states only in summary, together with the flow diagrams for those processes whose control flow is difficult to follow from prose alone. A flow earns a diagram here on one criterion: that its shape is unusual enough that a linear description of it misleads. Each of the seven crosses several processes, branches on conditions that are not obvious from the endpoint that starts it, and has at least one path that exists only to handle a failure or an attempt at abuse. The diagrams show the branches and the failure paths together, which prose in sequence cannot. They are ordered as the system is exercised rather than as the packages are named: the learner's own loop first, then the authoring pipelines that fill it, then the assessment those pipelines produce, and finally the pathway structure above all of them.

Spaced repetition: why this is an appendix

The scheduling model below governs how aBridgeAI decides when a learner sees a card again. It is specified here rather than in the body of the report because the present iteration does not evaluate it: establishing that a scheduling policy improves retention requires a longitudinal cohort study that this project's timeline does not accommodate, and reporting a design as a contribution without evidence for it would overstate what has been shown. What the system does implement is described in Section 6.9; what follows is the specification that implementation realises, retained in full so that the algorithm is reproducible and available for evaluation in later work.

B.1 Hybrid SM-2 and Q Derivation

aBridgeAI adopts SM-2 as the scheduling foundation for quiz-based SR, extending it with two modifications: replacement of self-reported Q with an algorithmically derived value grounded in objective response signals, and introduction of a calibration weight that dampens early EF growth until sufficient retrieval evidence has accumulated. The theoretical motivation for these departures is established in Section 3.2.1; this section specifies their formal implementation.

Teacher-Defined Expected Response Time

Each quiz card carries a teacher-assigned expected response time T_{exp}, set explicitly during the card review phase before publication. This value reflects the teacher's domain judgment of the duration within which a student demonstrating confident, independent recall should be able to answer the question. T_{exp} is a mandatory field — no card may be published without it — and may be revised by the teacher at any point prior to or during deployment. Because T_{exp} is teacher-defined rather than formula-derived, it incorporates contextual knowledge of the question's cognitive demand that no surface-feature heuristic could reliably approximate: a teacher setting T_{exp} for a TCP troubleshooting question draws on their understanding of how long a competent student requires to reason through the problem, not merely how long the question takes to read.

Time Ratio

Each student response is normalised against the card's expected response time via a dimensionless time ratio ρ:

$$\rho = \frac{T_{\text{actual}}}{T_{\text{exp}}}, \tag{B.1}$$

where T_{actual} is the elapsed time in milliseconds between question presentation and answer submission. A value of $\rho < 1$ indicates the student responded faster than expected; $\rho = 1$ indicates exact correspondence with the teacher's expectation; and $\rho > 1$ indicates the response exceeded the expected window. Because ρ is dimensionless and normalised against a card-specific baseline, it is comparable across questions of differing complexity — a student answering a hard question in exactly the expected time produces the same ρ as a student answering an easy question in its expected time.

Q Value Assignment

The quality response value $Q \in \{0, 1, 2, 3, 4, 5\}$ is determined by two signals in sequence: first, whether the answer was correct; second, for correct answers, whether a hint was used. Within each branch, Q is resolved as a continuous function of ρ rather than through fixed threshold boundaries, ensuring that the mapping from response latency to recall quality varies smoothly with the ratio rather than producing discontinuous jumps at arbitrary cutoff values.

An incorrect answer yields $Q = 0$ unconditionally, reflecting the position established in Section 3.2.1 that partial credit for failed recall creates an advancement pathway inconsistent with competency-based progression.

For correct answers where a hint was used, the student has demonstrated assisted rather than independent recall. The ceiling is therefore $Q = 2$, with the value resolved by:

$$Q = \max(1,\ 2 - \lfloor \rho \rfloor), \quad Q \in \{1, 2\}. \tag{B.2}$$

This maps $\rho < 1$ and $\rho \in [1, 2)$ to $Q = 2$, and $\rho \geq 2$ — indicating the student took more than twice the expected time even with a hint — to $Q = 1$. No hint usage can produce $Q \geq 3$, preserving the SM-2 convention that scores of 3 and above require independent retrieval [5].

For correct answers where no hint was used, Q is mapped from ρ over the range $\{3, 4, 5\}$ as:

$$Q = \max(3,\ 5 - \lfloor 2\rho \rfloor), \quad Q \in \{3, 4, 5\}. \tag{B.3}$$

This assigns $Q = 5$ when $\rho < 0.5$ — the student answered in less than half the teacher's expected time, consistent with effortless retrieval; $Q = 4$ when $0.5 \leq \rho < 1$ — the student answered within the expected window, consistent with controlled but fluent recall; and $Q = 3$ when $\rho \geq 1$ — the student exceeded the expected time, consistent with laboured reconstruction despite eventual correctness [6]. The floor of 3 ensures that any independent correct response, regardless of latency, is treated as a passing recall event and does not trigger an interval reset. The complete assignment rule is summarised in Table B.1.

Table B.1: Q value assignment rules based on correctness, hint usage, and time ratio ρ. Boundaries at $\rho = 1$ and $\rho = 2$ correspond to the teacher-defined expected response time and twice that value respectively; the boundary at $\rho = 0.5$ corresponds to half the expected time.

Correctness	**Hint Used**	**Time Ratio ρ**	Q
Incorrect	—	any	0
Correct	Yes	$\rho \geq 2$	1
Correct	Yes	$\rho < 2$	2
Correct	No	$\rho \geq 1$	3
Correct	No	$0.5 \leq \rho < 1$	4
Correct	No	$\rho < 0.5$	5

The boundaries in the no-hint branch — $\rho = 1$ and $\rho = 0.5$ — require no external calibration because they derive directly from the teacher-defined T_{exp}: $\rho = 1$ is exactly the teacher's competency expectation, and $\rho = 0.5$ is half of it. Similarly, the hint branch boundary at $\rho = 2$ represents twice the expected time, a duration at which continued difficulty despite hint exposure reasonably indicates weak encoding rather than momentary hesitation. All boundaries are therefore interpretable as multiples of T_{exp} rather than as independently chosen constants, and their justification is grounded in the semantics of the teacher-defined baseline rather than in arbitrary numerical choices.

EF Update and Interval Scheduling

The derived Q value is passed into the standard SM-2 EF update rule:

$$EF_{\text{new}} = EF_{\text{old}} + (0.1 - (5 - Q)\,(0.08 + (5 - Q) \cdot 0.02))\,, \tag{B.4}$$

with EF bounded below by $EF_{\text{min}} = 1.3$. The review interval follows the standard SM-2 recurrence:

$$I(n) = \begin{cases} 1 & \text{if } n = 1 \\ 6 & \text{if } n = 2 \\ I(n-1) \cdot EF & \text{if } n > 2, \end{cases} \tag{B.5}$$

where n denotes the repetition count. If $Q < 3$, the item is considered failed: I resets to 1 day and n returns to zero regardless of prior history. The EF penalty is applied before the reset, ensuring that repeated failures progressively reduce the interval growth rate even as the absolute interval restarts.

Interval Jitter

To prevent review storms caused by the simultaneous scheduling of multiple cards to the same date, a jitter factor $\varepsilon \sim \mathcal{U}(-0.1,\ 0.1)$ is applied multiplicatively to each computed interval:

$$I_{\text{final}} = I(n) \cdot (1 + \varepsilon). \tag{B.6}$$

The magnitude of ε is small enough to have negligible effect on retention outcomes while distributing scheduled reviews across a window of approximately $\pm 10\%$ of the nominal interval, preventing the concentration of review obligations on single calendar dates.

Interval Ceiling and Card Retirement

As established in Section 3.2.1, the SM-2 recurrence is unbounded above. aBridgeAI bounds it with an organisation-configurable ceiling $I_{\max}$ applied after jitter, so that the ceiling is a true upper bound rather than one the jitter may exceed:

$$I_{\text{scheduled}} = \min\left(I(n) \cdot (1+\varepsilon),\ I_{\max}\right). \tag{B.7}$$

The default $I_{\max} = 36500$ days is adopted from Anki, where the equivalent setting carries the same default, and at that value the ceiling does not bind: the longest interval reachable through eight consecutive maximal-EF reviews is approximately 1465 days. The direction of the control is worth stating explicitly, because it is counter-intuitive. Lowering $I_{\max}$ does not reduce review load; it guarantees that no item drifts out of sight for longer than the configured period, and therefore *increases* the number of reviews a student is asked to perform. It is a retention floor, not a workload ceiling.

A second, independent parameter converts that bound into a terminal state. When card retirement is enabled and a review earns an interval exceeding $I_{\max}$, the card ceases to be scheduled: it retains its EF, its interval, its repetition count and its full review history, but holds no next due date and is therefore absent from every due-card read. Retirement requires the computed interval to exceed $I_{\max}$ strictly, so an installation may set the ceiling to the cadence at which it wishes items to settle without that cadence itself triggering retirement.

Retirement is terminal in both directions. A subsequent encounter with the item — in a later quiz, for instance — is graded and recorded as a review, because the retrieval did occur, but does not return the card to the queue; and a failed review neither retires a card nor revives a retired one, since the reset to a one-day interval addresses an item still being consolidated rather than one already released.

This parameter is disabled by default, and the reason is theoretical rather than operational. Under the Two-Component Model described in Section 3.2.1, retrievability decays continuously between reviews for every item regardless of its stability; no state exists in which an item is no longer subject to forgetting. A retirement point therefore cannot be justified as a claim about memory. It is justified instead as a bounded-scope decision: where the material belongs to a course of finite duration, an institution may reasonably decide that review obligations should not outlive the enrolment that created them. The distinction matters for interpretation of the resulting data — a retired card records that scheduling stopped, not that the item was permanently learned.

Calibration Weight

SM-2 initialises EF at 2.5 for all items, implicitly assuming average retention ability from the first review. In an institutional context where EF influences lesson unlock eligibility, an overestimated early EF produces intervals that grow too quickly before the algorithm has accumulated sufficient evidence of the student's retention capacity for that item. To address this, aBridgeAI applies a calibration weight α to positive EF increments during the first three repetitions of any card:

$$EF_{\text{new}} = \begin{cases} EF_{\text{old}} + \alpha \cdot \Delta EF & \text{if } n \leq 3 \text{ and } \Delta EF > 0 \\ EF_{\text{old}} + \Delta EF & \text{otherwise,} \end{cases} \tag{B.8}$$

where ΔEF denotes the increment produced by the standard update rule and $\alpha = 0.6$. Negative increments — produced when $Q < 4$ — are applied at full magnitude regardless of n, ensuring that evidence of forgetting is incorporated immediately while evidence of retention accumulates more conservatively. Once $n > 3$, $\alpha = 1$ and the standard update rule applies without modification. This asymmetric initialisation ensures faster convergence onto the student's true retention profile than an optimistic starting assumption would permit, consistent with the principle that conservative initial models require fewer corrective cycles than overconfident ones [5].

B.2 Individualised Learner Modeling

A foundational design decision in aBridgeAI is the isolation of all SM-2 state variables at the level of the student-card pair rather than the card itself. The Easiness Factor EF, review interval I, repetition counter n, and next scheduled due date are maintained as independent records for each unique combination of student and knowledge item. Two students reviewing the same card therefore accumulate entirely separate EF trajectories reflecting their individual retrieval histories, and no student's scheduling is influenced by the aggregate performance of peers on shared material. This isolation is a theoretical necessity as much as an implementation choice: the same observable response pattern may reflect different underlying knowledge states in different learners depending on prior experience, domain familiarity, and cognitive strategy [28, 33]. Aggregating state across learners would systematically misrepresent individual knowledge trajectories and undermine the validity of any progression gate derived from the model.

Per-Student EF Trajectory

Because EF evolves independently for every student-card pair, each student accumulates a distinct EF trajectory for every card they review. A student who consistently answers a card correctly and quickly — producing high Q values and therefore upward EF drift — will be scheduled to review that card at progressively longer intervals, reflecting growing memory stability. A student who repeatedly fails the same card will see its EF driven toward the floor of 1.3, producing frequent short-interval reviews until stable retention is demonstrated. These trajectories are entirely independent: the high-performing student's EF on a card has no influence on the struggling student's schedule for the same card, and vice versa.

The calibration weight $\alpha = 0.6$ applied during the first three repetitions, specified in Section B.1, further ensures that early EF growth is conservative across all student-card pairs. Since EF is initialised at 2.5 for every new card encounter regardless of the student's general performance profile, the calibration phase provides a card-level adjustment that complements the per-student isolation: each student begins each new card at the same conservative starting point and diverges from there according to their own retrieval evidence.

Lesson-Level EF Distribution

Because EF is tracked at the card level, each student possesses a distribution of EF values across all active cards within a lesson. Let $\mathcal{C}_\ell$ denote the set of active cards in lesson ℓ, and let $EF_{s,c}$ denote the current Easiness Factor of student s on card c. The set of EF values $\{EF_{s,c} \mid c \in \mathcal{C}_\ell\}$ constitutes the student's memory stability profile for that lesson — a card-granular representation of which knowledge components are well-consolidated and which remain fragile. This profile is the basis from which both the lesson unlock condition and the Knowledge Retention Estimate are derived.

Lesson Unlock Eligibility

A student satisfies the unlock condition for lesson ℓ — becoming eligible to progress to the subsequent lesson — when the proportion of their cards whose EF meets or exceeds a teacher-configured minimum threshold $EF_{\min}^{\text{unlock}}$ reaches a teacher-configured coverage threshold τ:

$$\frac{\left|\left\{c \in \mathcal{C}_\ell \mid EF_{s,c} \geq EF_{\min}^{\text{unlock}}\right\}\right|}{|\mathcal{C}_\ell|} \geq \tau. \tag{B.9}$$

The numerator counts the cards in the lesson for which the student's EF has reached

the required floor; the denominator is the total number of active cards in the lesson; and $\tau \in (0, 1]$ is the minimum proportion required before the unlock condition is satisfied. Setting $\tau = 1.0$ requires every card to meet the EF floor before progression, enforcing zero tolerance for weak knowledge components. Lower values of τ permit advancement when a large majority of cards are consolidated, accommodating instructors who wish to allow progression despite isolated difficult items. Both $EF_{\text{min}}^{\text{unlock}}$ and τ are configured per lesson by the teacher, allowing unlock stringency to vary across lessons according to the criticality of the material.

The card set $\mathcal{C}_\ell$ is determined by which quizzes contribute cards at all. A quiz may be designated as an assessment rather than as practice, in which case answering its questions opens no new cards, and its questions consequently form no part of the denominator above. This prevents a summative assessment from governing progression through the same mechanism that measures formative retrieval practice: without it, an examination administered under time pressure would generate cards whose depressed EF values determine whether the student may proceed. Questions the student already holds cards for are still graded normally when they appear in such a quiz, since the retrieval occurred and the evidence is admissible; what the designation withholds is the creation of new scheduling obligations, not the recording of performance.

Where a teacher additionally configures an interview section for the lesson, the unlock condition above is necessary but not sufficient: the student must also satisfy the interview pass condition before progression is granted. The combined gate ensures that both retention of factual knowledge — measured by the quiz EF distribution — and demonstrated application of that knowledge — measured by the interview outcome assessment — are required for lesson completion.

B.3 Student Performance Metrics

aBridgeAI aggregates per-card SM-2 state into three learner-facing metrics that together characterise a student's cognitive standing within a lesson. All three are derived exclusively from the student's own retrieval history, with no dependence on peer performance or instructor-assigned grades. They are surfaced in both per-student and class-wide views: per-student views allow instructors to identify individuals requiring targeted intervention, while class-wide aggregations expose systematic knowledge gaps shared across the cohort that may warrant revision of the source material or quiz configuration rather than individual remediation.

Knowledge Retention Estimate

The *Knowledge Retention Estimate* $\hat{R}_{s,\ell}$ summarises the student's memory stability profile for lesson ℓ as a single normalised score in $[0, 1]$. Each card's EF is first normalised against the operational range of the Easiness Factor, then averaged across all active cards in the lesson:

$$\hat{R}_{s,\ell} = \frac{1}{|\mathcal{C}_\ell|} \sum_{c \in \mathcal{C}_\ell} \frac{EF_{s,c} - EF_{\min}}{EF_{\max} - EF_{\min}}, \tag{B.10}$$

where $EF_{\min} = 1.3$ is the SM-2 floor and $EF_{\max} = 2.5$ is the initialisation value, defining the full operational range. A card at the floor contributes 0 to the sum; a card at the ceiling contributes 1. The resulting $\hat{R}_{s,\ell}$ therefore reflects the average proportion of the possible EF range that has been achieved across the lesson's cards, with values approaching 1 indicating widespread stable retention and values approaching 0 indicating widespread difficulty or recent failure resets.

Cards with repeated $Q = 0$ outcomes are driven toward $EF_{\min}$, pulling $\hat{R}_{s,\ell}$ downward and surfacing persistent knowledge gaps without requiring explicit gap detection logic. This metric is presented to both students and teachers as a continuous progress indicator without exposing the underlying per-card EF values directly, preserving a clean separation between the diagnostic signal and its raw parameters.

Progression Readiness

Progression Readiness is a binary eligibility signal derived directly from the unlock condition specified in Section B.2. It evaluates whether the proportion of the student's cards meeting $EF_{\min}^{\text{unlock}}$ satisfies the coverage threshold τ, returning either eligible or ineligible. Unlike $\hat{R}_{s,\ell}$, which reflects average stability across all cards, Progression Readiness is sensitive specifically to the distribution of cards relative to the unlock floor: a student with high average $\hat{R}_{s,\ell}$ may nonetheless be ineligible if $\tau = 1.0$ and one card remains below $EF_{\min}^{\text{unlock}}$, while a student with moderate $\hat{R}_{s,\ell}$ may be eligible if the majority of their cards are above the floor and $\tau < 1.0$. The two metrics are therefore complementary: $\hat{R}_{s,\ell}$ describes the overall health of the lesson's knowledge profile, while Progression Readiness answers the specific question of whether the unlock condition has been met.

Review Compliance Rate

The *Review Compliance Rate* $\rho_{s,\ell}$ measures the proportion of due cards that the student reviewed within a configurable grace window Δt_{grace} following their scheduled date:

$$\rho_{s,\ell} = \frac{|\{c \in \mathcal{C}_\ell \mid t_{\text{reviewed}} \leq t_{\text{due}} + \Delta t_{\text{grace}}\}|}{|\{c \in \mathcal{C}_\ell \mid t_{\text{due}} \leq t_{\text{now}}\}|}, \tag{B.11}$$

where the numerator counts cards reviewed on time and the denominator counts all cards whose due date has passed as of the current moment t_{now}. The grace window Δt_{grace} acknowledges that students cannot review at the exact moment a card becomes due, and is configured at the system level.

This metric exists specifically to address a structural consequence of the no-decay design: as established in Section 3.2.1, aBridgeAI does not apply automatic EF penalties to overdue cards. A student who stops reviewing therefore retains their last recorded EF values indefinitely, and $\hat{R}_{s,\ell}$ remains frozen at its last state — presenting an artificially healthy retention picture to both student and teacher. $\rho_{s,\ell}$ fills this gap: a student whose compliance rate is falling while $\hat{R}_{s,\ell}$ remains stable is exhibiting engagement drift on unreviewed material, a condition the teacher dashboard flags as a warning signal. The integrity of EF as a measure of demonstrated retrieval performance is preserved — only reviews that actually occurred contribute to EF — while disengagement remains visible through a dedicated metric rather than being silently absorbed into a decaying parameter.

The relationship between the three metrics is summarised in Table B.2.

Table B.2: Summary of student performance metrics and their roles within the aBridgeAI learner modeling layer.

Metric	**Type**	**What It Measures**	**Primary Purpose**
Knowledge Retention Estimate $\hat{R}_{s,\ell}$	Continuous $[0, 1]$	Average normalised EF across all cards in the lesson	Diagnostic — overall memory stability health
Progression Readiness	Binary	Proportion of cards meeting $EF_{\min}^{\text{unlock}}$ relative to τ	Gate — lesson unlock eligibility
Review Compliance Rate $\rho_{s,\ell}$	Continuous $[0, 1]$	Proportion of due cards reviewed within grace window	Signal — engagement drift detection

B.4 Review Scheduling Flow

Figure B.1 traces one review from the answer that triggers it to the state it leaves behind, across the three layers that carry it: the API surface that decides whether a review fires at all, the service that derives Q and reschedules the card, and the transaction that persists the result and releases its side effects.

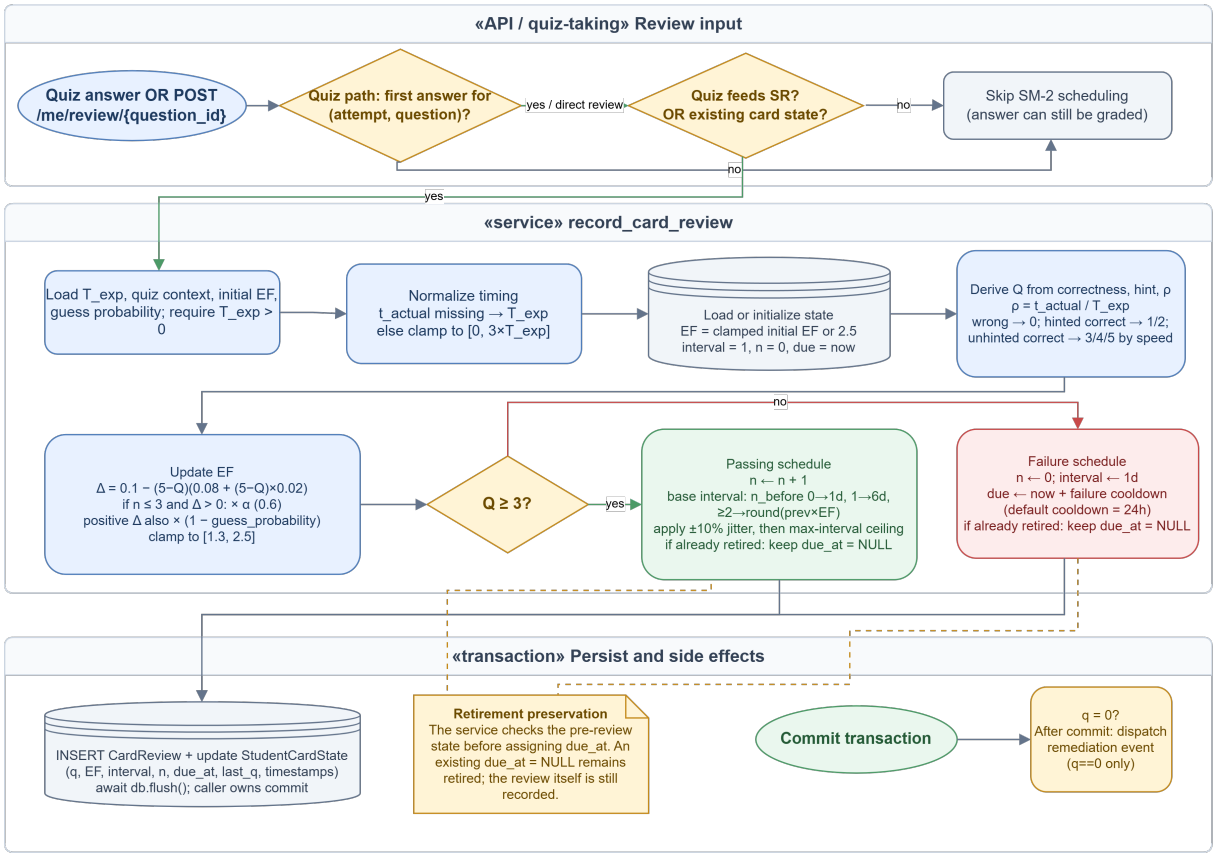


Figure B.1: SM-2 review scheduling: from graded answer to persisted card state.

B.5 Lesson Unlock Gate

The gate's three conditions are specified in Section B.2 and its implementation in Section 6.9. Figure B.2 shows what happens between a learner clicking a lesson and the server deciding whether to hand over its body.

The diagram exists mainly to make two things unmistakable. The first is that the verdict is computed on *read* and nowhere else: there is no unlock event, no service that fires when a threshold is crossed, and no stored eligibility column — only a memoised computation and a cache key that writes invalidate. The second is what the gate does

with a malformed prerequisite graph. The traversal is recursive, so a cycle would not terminate; the implementation detects one, logs it, and treats the cyclic edge as *satisfied*. That is a deliberate choice to fail open on a data error the learner did not cause, and it is the kind of decision a reader will assume is an oversight unless the diagram shows it as a labelled branch.

Two further details are visible only here. Navigation metadata and lesson content sit on opposite sides of the boundary — the curriculum tree tells the learner a lesson exists and is locked, while every read that would return its body, its resources, a signed download URL, or its knowledge graph passes through the gate — so a locked lesson is discoverable but not readable. And the whole gate sits behind a single deployment switch, which returns eligible immediately when set: an escape hatch for the case where a misconfigured threshold locks a cohort out of its own course mid-term.

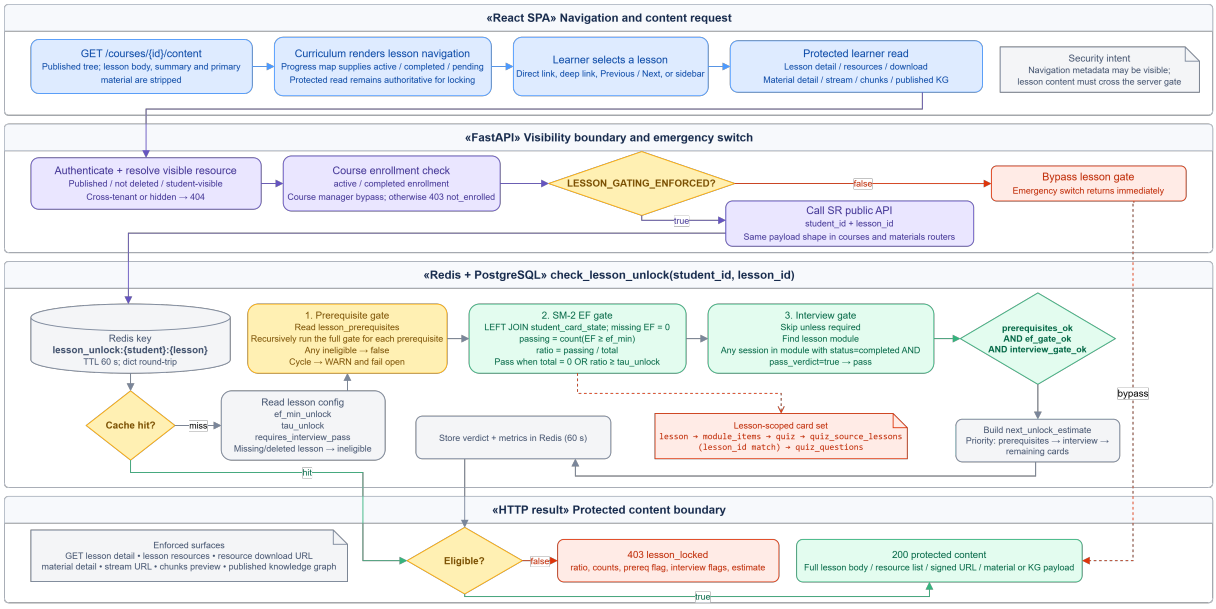


Figure B.2: Lesson unlock gate: enrolment and visibility checks, the three unlock conditions, and the protected-content boundary.

B.6 Material Ingestion

The implementation is described in Section 6.5. Figure B.3 traces one uploaded document from the teacher's file picker to the knowledge graph a learner eventually reads, across the three processes that carry it.

Three properties of this flow are the reason it is drawn rather than only described. The bytes never pass through the API: the browser uploads directly to object storage against

a presigned session, and the API's job afterwards is to verify that what was promised actually arrived — which is why a `HEAD` check for a phantom or truncated object appears as a rejection path with no counterpart anywhere else in the system. The work then leaves the request entirely, so every stage after the enqueue runs in a worker whose failure is recorded against the material version rather than returned to anyone. And the last lane is not automatic at all: the AI-extracted graph is a *seed* for a teacher to curate, publishing is blocked until they do, and an unavailable graph yields an in-memory placeholder rather than an empty one — a distinction with no visible difference until the teacher tries to publish.

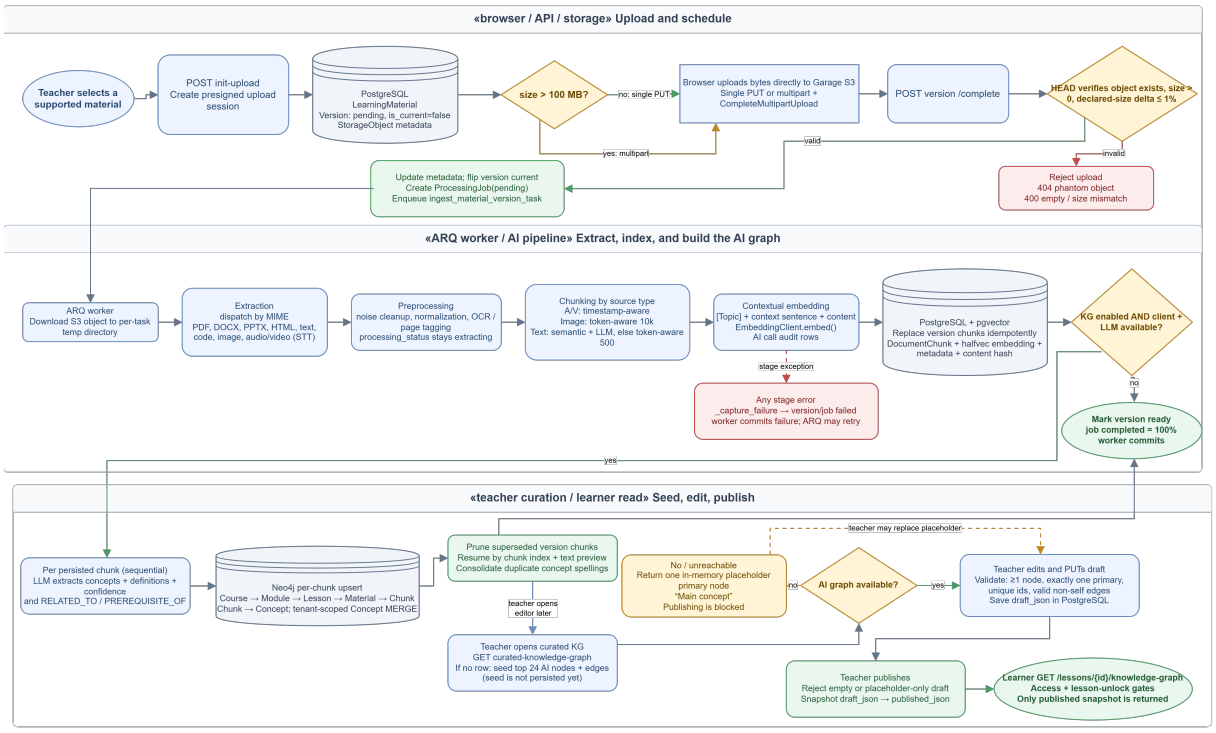


Figure B.3: Material ingestion: upload and scheduling, worker extraction and indexing, and teacher curation of the knowledge graph.

B.7 Quiz Generation

The implementation is described in Section 6.6.2. Figure B.4 traces a generation request from the teacher's click to the questions that land in their review queue.

What the diagram makes visible is that the HTTP request and the work it asks for are two different things with two different lifetimes. The endpoint validates, guards, creates a durable run row, enqueues, and returns `202` — at which point the request is over and

nothing has been generated. Everything of substance happens afterwards in a worker, and the client learns of it only by polling the run it was handed. The consequences of that split are what the remaining branches encode: a run orphaned by a worker crash is recovered by a separate sweep rather than by the caller; progress is checkpointed on its own short-lived database session so a report of progress survives a pipeline transaction that later rolls back; and the two generation modes diverge into genuinely different pipelines, topic mode over-generating to absorb the losses that validation and de-duplication will inflict, coverage mode budgeting per outline section before it generates anything at all.

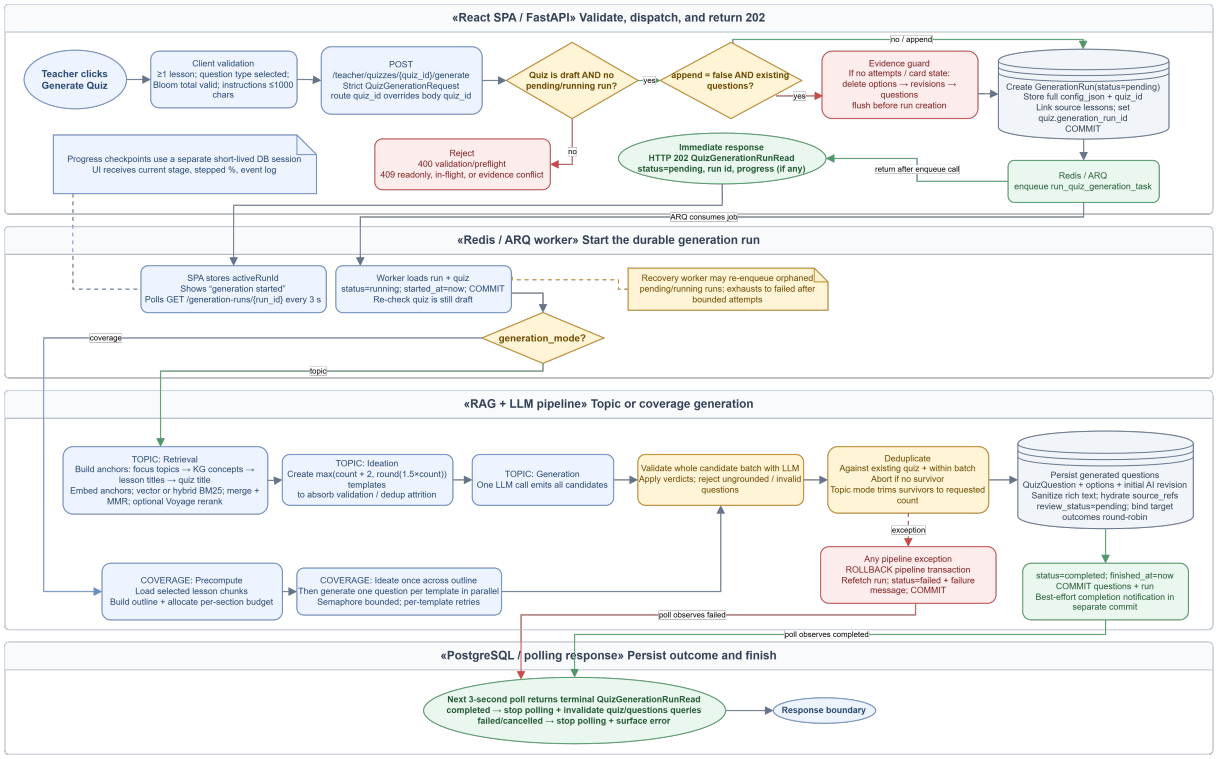


Figure B.4: Quiz generation: synchronous validation and dispatch, the durable generation run, the topic and coverage pipelines, and the terminal poll.

B.8 Quiz Attempt Lifecycle

The implementation is described in Section 6.7. Figure B.5 follows one attempt from the intro screen to the result, and it is the longest flow in the system because an attempt is the one place where assessment integrity, spaced repetition and crash recovery all bear on the same rows at the same time.

Three of its properties are worth naming before reading it. Everything that could change beneath a live attempt is *frozen at creation*: the question order and option shuffle,

the timing, and the integrity policy are all snapshotted onto the attempt row, so editing the quiz mid-attempt cannot alter what the student is being measured against. Ownership is a lease rather than a check — a Redis entry claimed before the database commit and renewed by heartbeat — which is what makes an explicit takeover on a second device possible without two tabs writing to one attempt. And the SM-2 review fires on the *first* saved answer to each question only: a later edit still changes the score, because the answer row is upserted, but it does not reschedule the card, or a student could tune their own review queue by toggling answers.

The diagram also carries the paths that exist only for things going wrong. An attempt left open past its deadline is closed by a cron sweep rather than by the student, and the sweep chooses between submitting it and expiring it according to the quiz's own policy. A submit flushes every unsaved answer first and aborts if any of those writes fails, rather than finalising a partial record. What the result exposes is itself gated: a quiz needing manual grading returns no score at all, and the visibility policy decides separately whether the student may see their score, their correctness, or the answers.

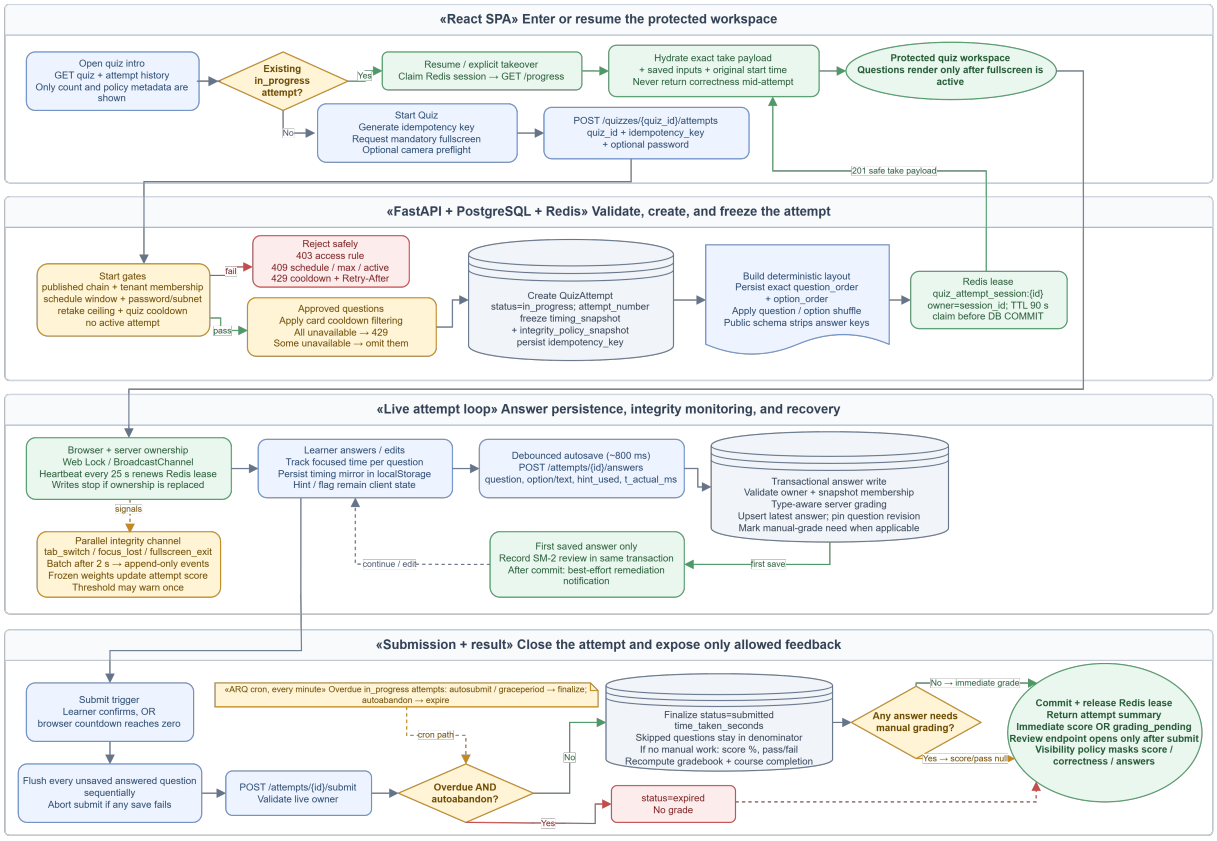


Figure B.5: Quiz attempt lifecycle: entry and resumption, attempt creation and freezing, the live answer loop with integrity monitoring, and submission.

B.9 Career Pathway Stage Evaluation

The rules this evaluator applies are specified in Section 5.3.1. Figure B.6 shows the order in which it applies them for one student against one pinned pathway version: stages are walked in position order, each one's required and quota-limited elective counts are resolved into a live completion, a latched completion already recorded is honoured over a live one that has since lapsed, and the unlock verdict for each stage reads the stage before it — which is why the pass runs forward and only once.

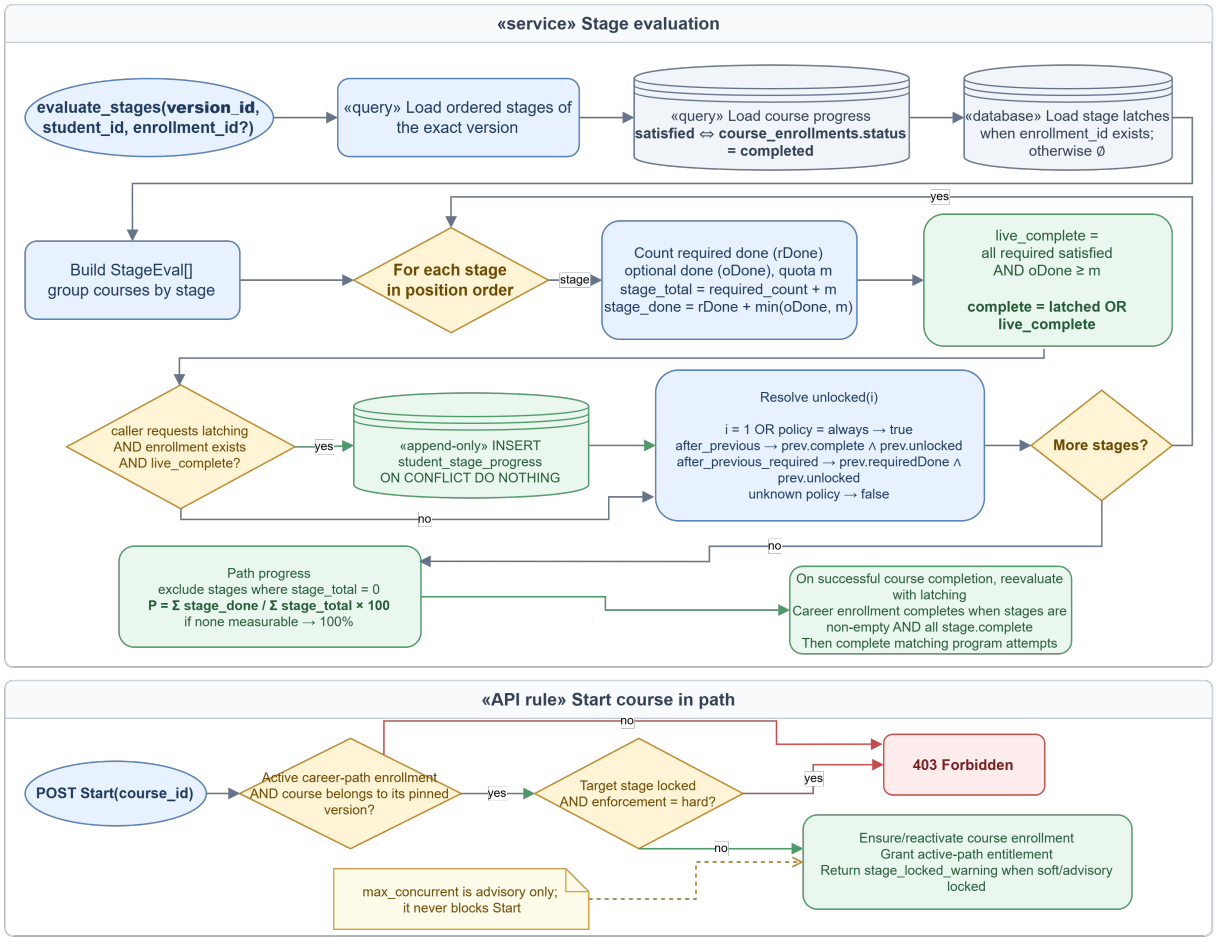


Figure B.6: Career pathway stage evaluation: from pinned version to per-stage completion, unlock, and overall progress.

B.10 Path Change and Attempt Closure

The governance model is described in Section 6.11. Figure B.7 traces a student's request to switch or drop a career path from creation through a dean's decision to the

downstream access that decision revokes.

This flow is drawn because approving a request is not a status update. The approval runs inside one transaction that closes the source attempt, writes an immutable `exit_snapshot` of the progress it had earned, creates the replacement attempt pinned to the target version, and then reconciles every entitlement the old path had granted. That reconciliation is where the subtlety lies: grants are transferred only for courses the two paths share and revoked otherwise, a course enrolment is dropped only when no other live entitlement still covers it, and completed enrolments and their awards are preserved regardless. A student who switches paths therefore keeps what they finished, keeps access to what both paths need, and loses only what the old path alone justified.

The guards are also checked twice, at different times and for different reasons. Creation checks that the request is coherent; approval re-checks the same invariants against the state as it stands at the moment of decision, because a target path can be archived, or a budget exhausted, between the request and the review. A change request whose target has since disappeared is invalidated rather than approved, and no attempt is closed — the one outcome that must never be reached by default, since closure is irreversible and the snapshot it writes is the only record of what the student had earned.

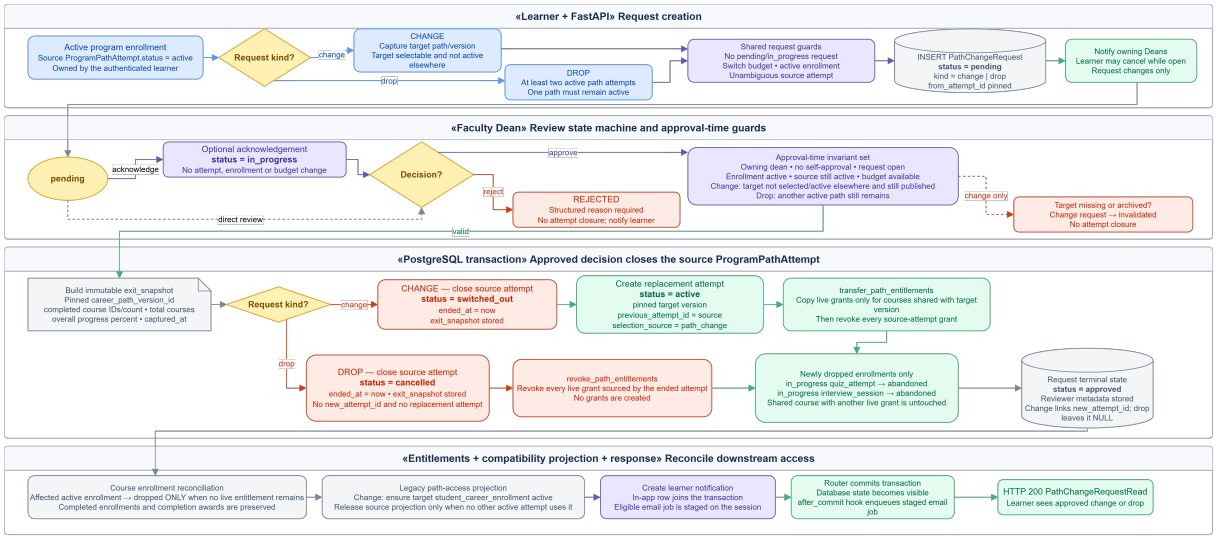


Figure B.7: Path change and attempt closure: request creation, the dean's review state machine, the approval transaction, and downstream entitlement reconciliation.

Appendix C Logical and Physical Database Design

The Entity-Relationship Diagram in Section 5.7 states what the main database holds and how its entities relate. The two views below restate that same structure at the levels beneath it: the logical design as the relations, attributes and keys the schema declares, and the physical design as what is actually created in PostgreSQL — column types, indexes, and the constraints that enforce the model rather than merely describing it.

C.1 Logical Database Design

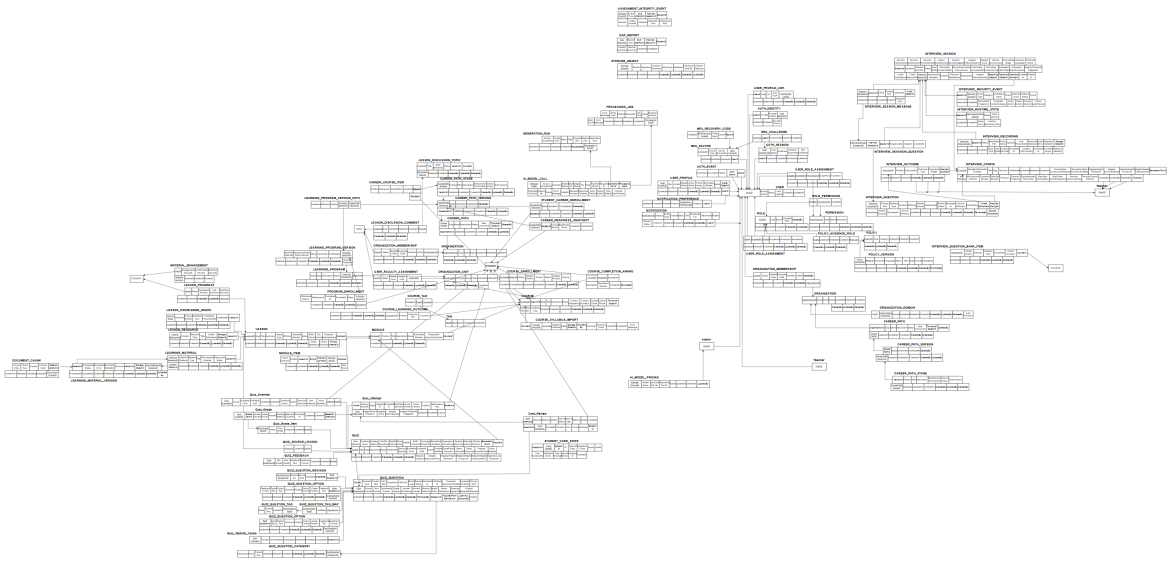


Figure C.1: Logical Database Design

C.2 Physical Database Design

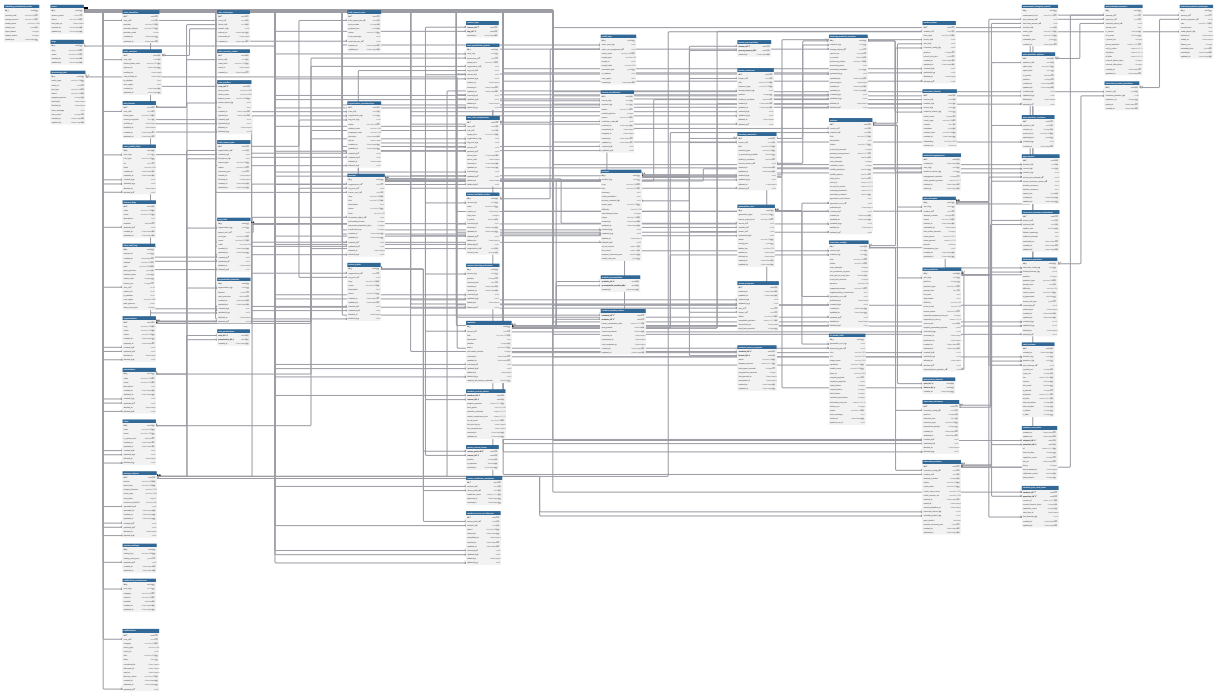


Figure C.2: Physical Database Design

Appendix D Activity Diagrams Specification

D.1 Authentication Flow

This activity diagram details the user authentication workflow in aBridgeAI. It shows how users log in through Google SSO, how the system verifies the local account and MFA requirement, creates an authenticated session, resolves roles and permissions, and handles logout.

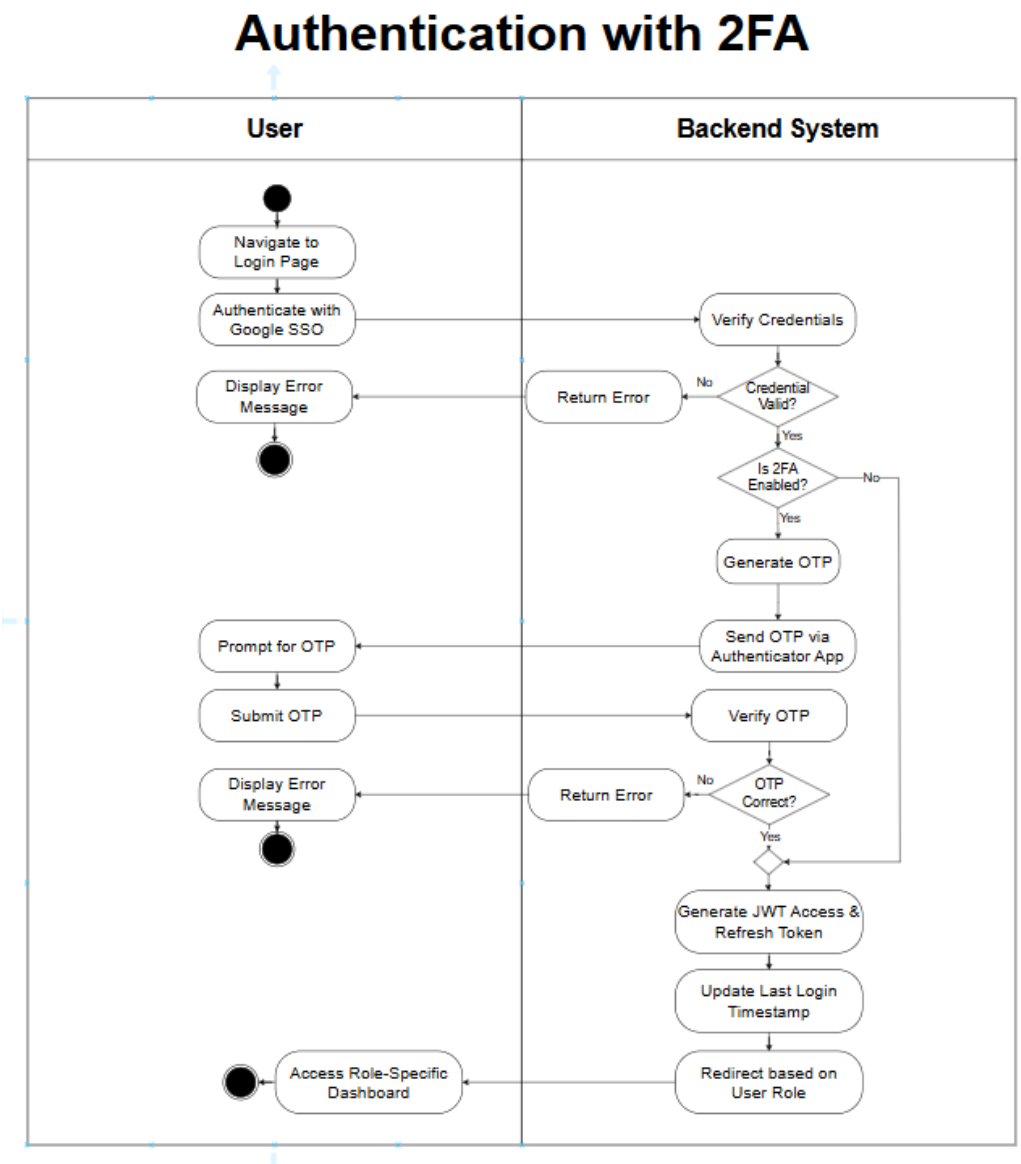


Figure D.1: Activity Diagram: User Login with 2FA

D.2 Smart Content Ingestion Flow

This activity diagram details the material ingestion workflow in aBridgeAI. It shows how teachers upload learning materials directly to object storage, how the backend verifies the upload and queues processing, and how the ingestion worker extracts, chunks, embeds, and stores content for retrieval.

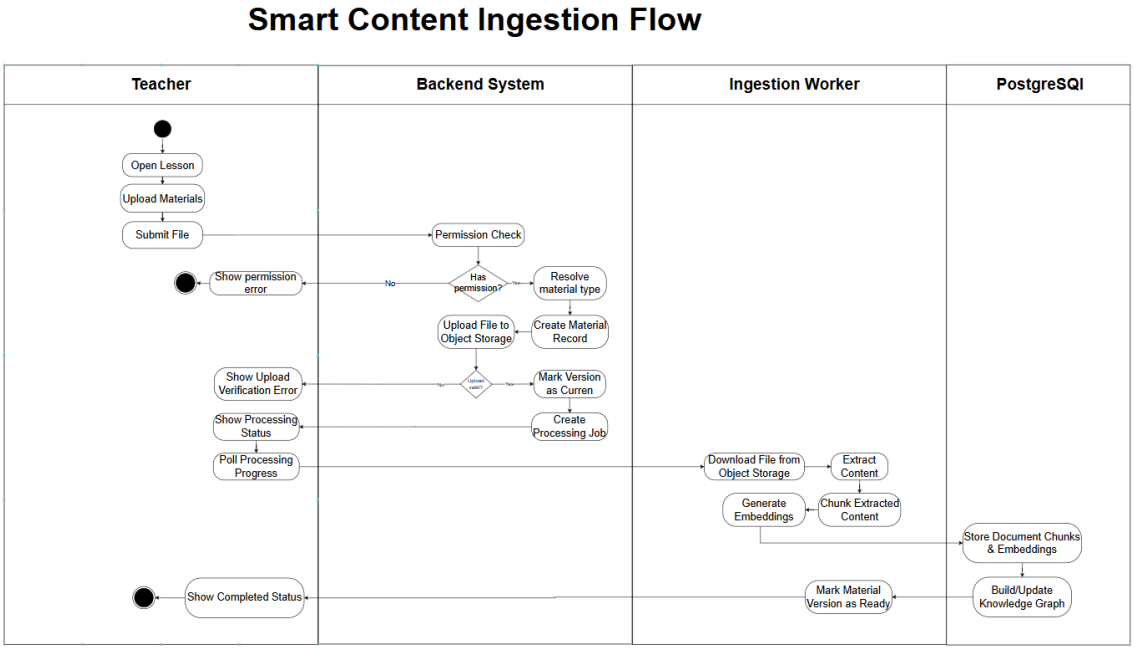


Figure D.2: Activity Diagram: Smart Content Ingestion and AI Parsing

D.3 Human-in-the-Loop Quiz Generation Flow

This diagram maps the assessment creation lifecycle. It demonstrates the "Human-in-the-Loop" architecture where the teacher configures parameters, the AI generates a draft assessment based on the Knowledge Graph, and the teacher performs the final review and pedagogical adjustments before publishing the quiz to the students.

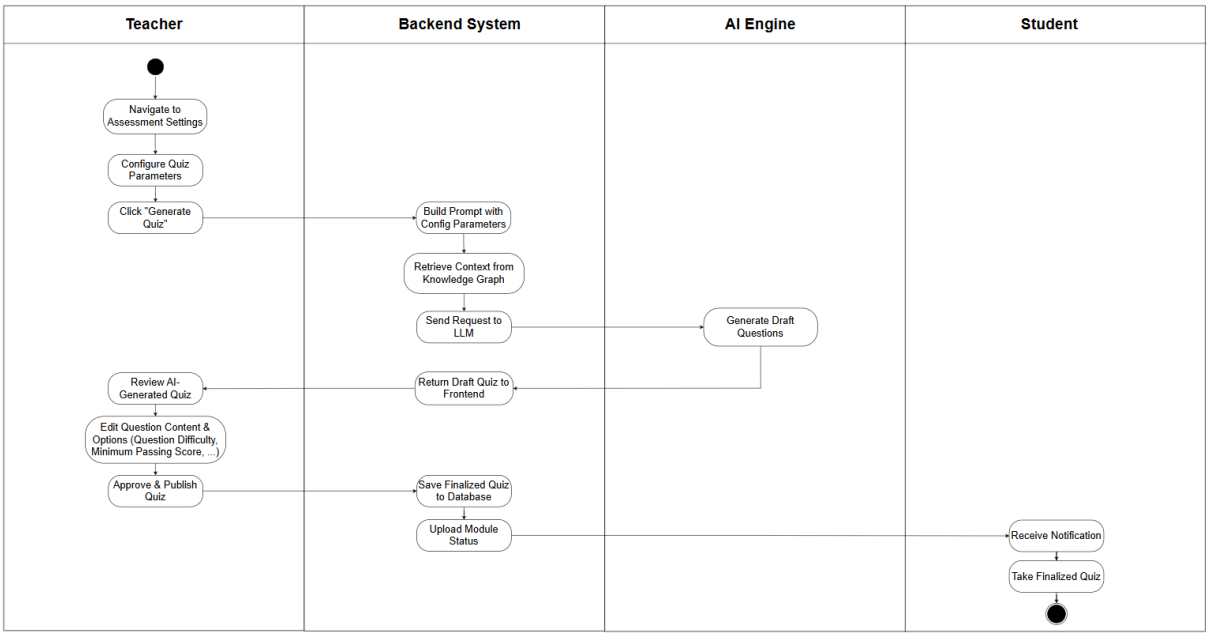


Figure D.3: Activity Diagram: HITL Quiz Generation and Execution

D.4 Native Mock Interview Flow

The previous raster activity diagram described an older generic interview flow and has no editable source. It is replaced by the current native activity trace.

1. Start or resume the persisted interview session; complete identity, audio, language, preparation, and readiness onboarding.

2. Optionally connect a warm LiveKit room during onboarding; dispatch the native interviewer only after readiness and start the assessment clock.

3. Receive either a final speech transcript or a typed `lk.chat` turn. For typed input, validate the action/key and persist a receipt before acknowledgement.

4. Fold the answer into durable runtime state. Apply a fast provisional sufficiency probe, persist the state version, then server-select the next question or retain the current question within configured follow-up and hint bounds.

5. Refresh the conversational state note. The LLM may phrase the next response and invoke guarded tools, while server policy remains authoritative for progression and completion.

6. Publish TTS audio, transcription, and control snapshot; queue deferred reconciliation for richer analysis. On reconnect, republish a snapshot and re-read the current question.

7. On model end or hard stop, close intake, drain bounded active work, flush transcript writes, submit once, enqueue terminal evaluation, and publish the final result when evaluation completes.

This textual activity trace is aligned with Section 6.8; it avoids presenting obsolete raster artwork as the deployed native architecture.

Appendix E Sequence Diagrams Specification

E.0.1 Authentication

This sequence diagram maps the execution flow of the user authentication process.

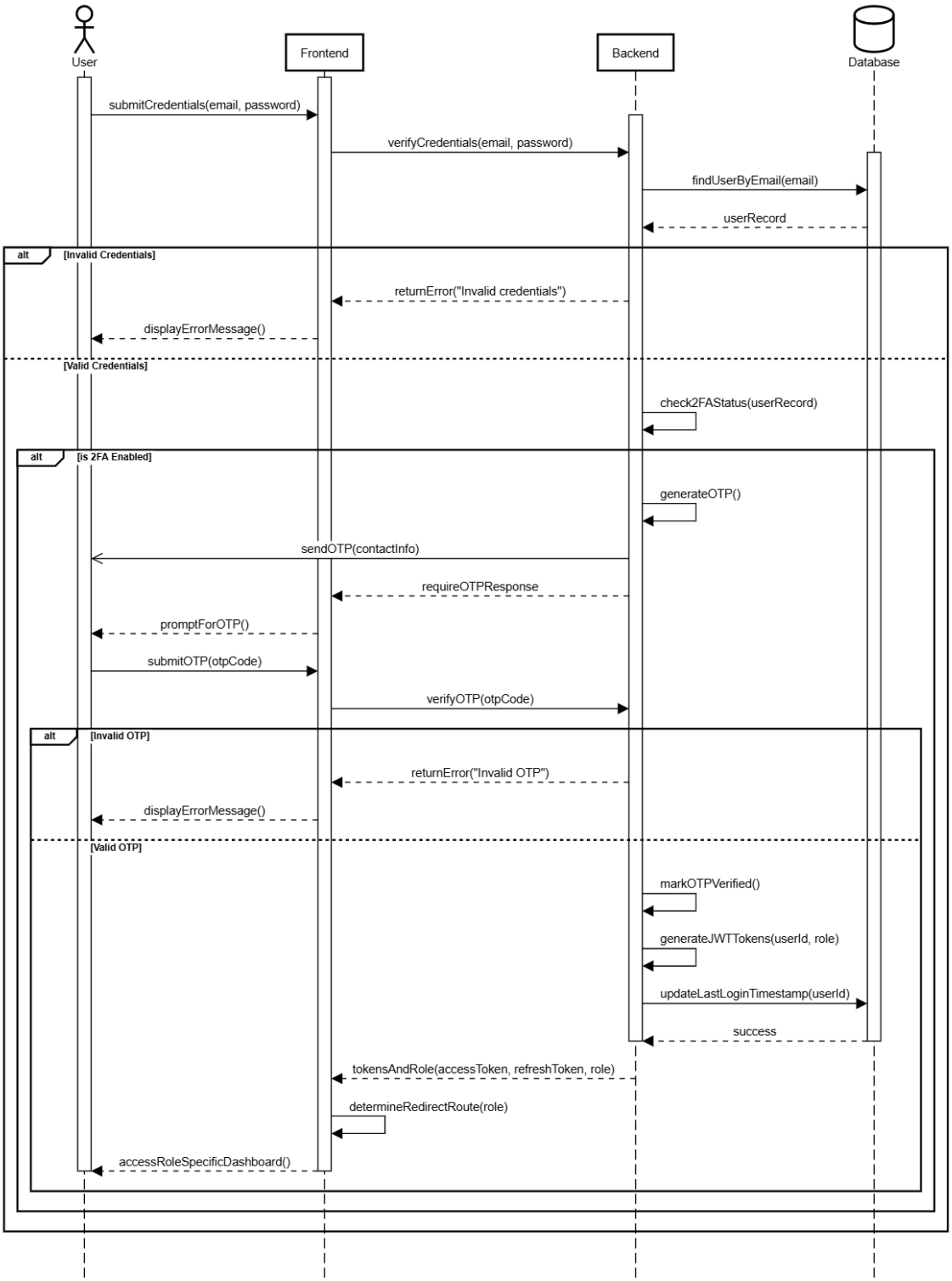


Figure E.1: Sequence Diagram: Authentication

E.0.2 Smart Content Ingestion

This sequence details the interactions when a teacher uploads course materials. It highlights the asynchronous chunking, embedding generation, and vector storage processes required to build the RAG pipeline.

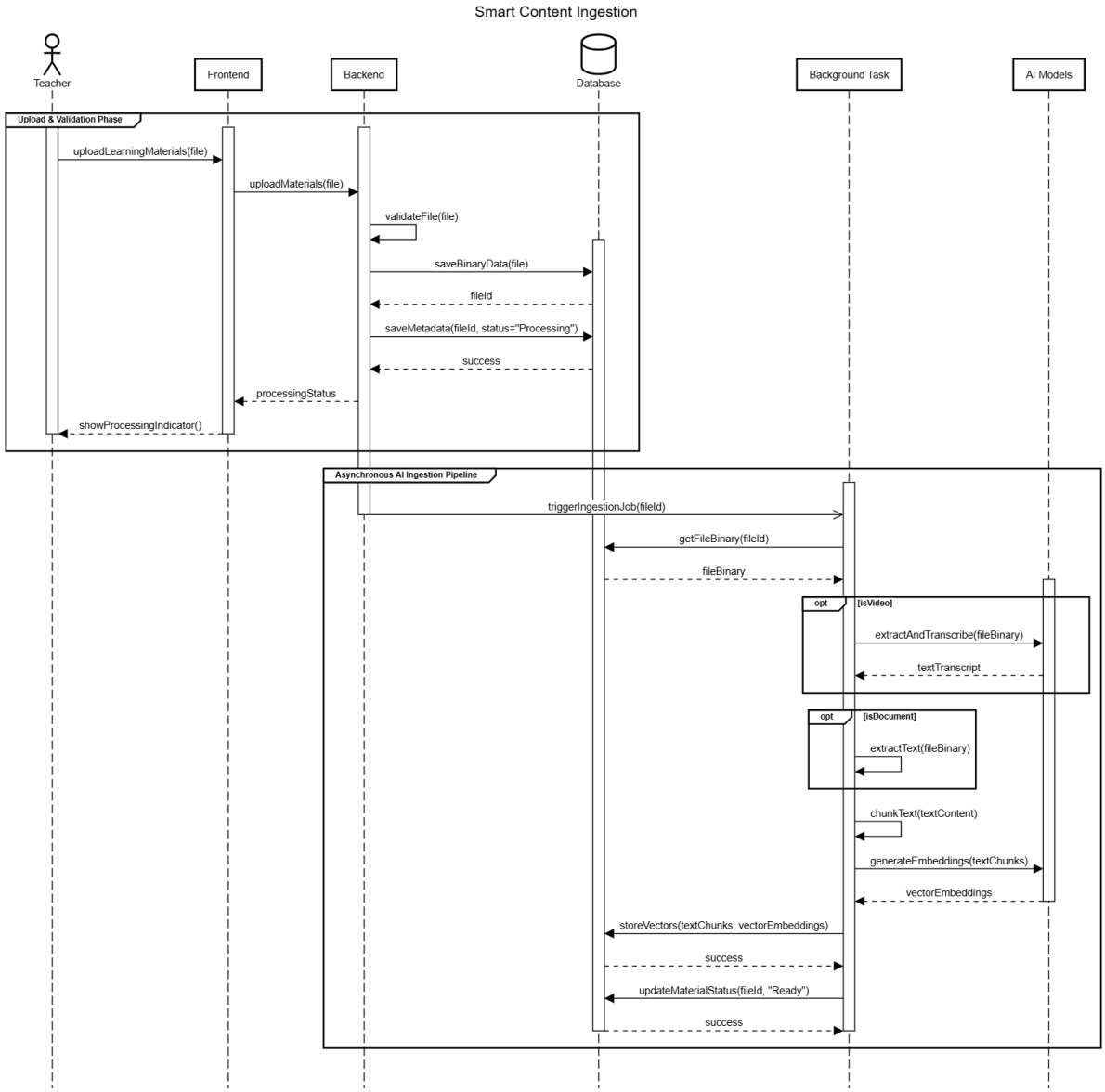


Figure E.2: Sequence Diagram: Smart Content Ingestion

E.0.3 Human-in-the-Loop Quiz Generation

This sequence diagram maps the assessment creation lifecycle, emphasizing the "Human-in-the-Loop" architectural pattern. While the AI Engine performs the compu-

tationally heavy task of generating questions via RAG based on the Course Knowledge Graph, the Teacher retains full control. The workflow includes a review and editing phase, allowing the instructor to adjust question content, difficulty levels, and passing thresholds before publishing the final quiz to the students.

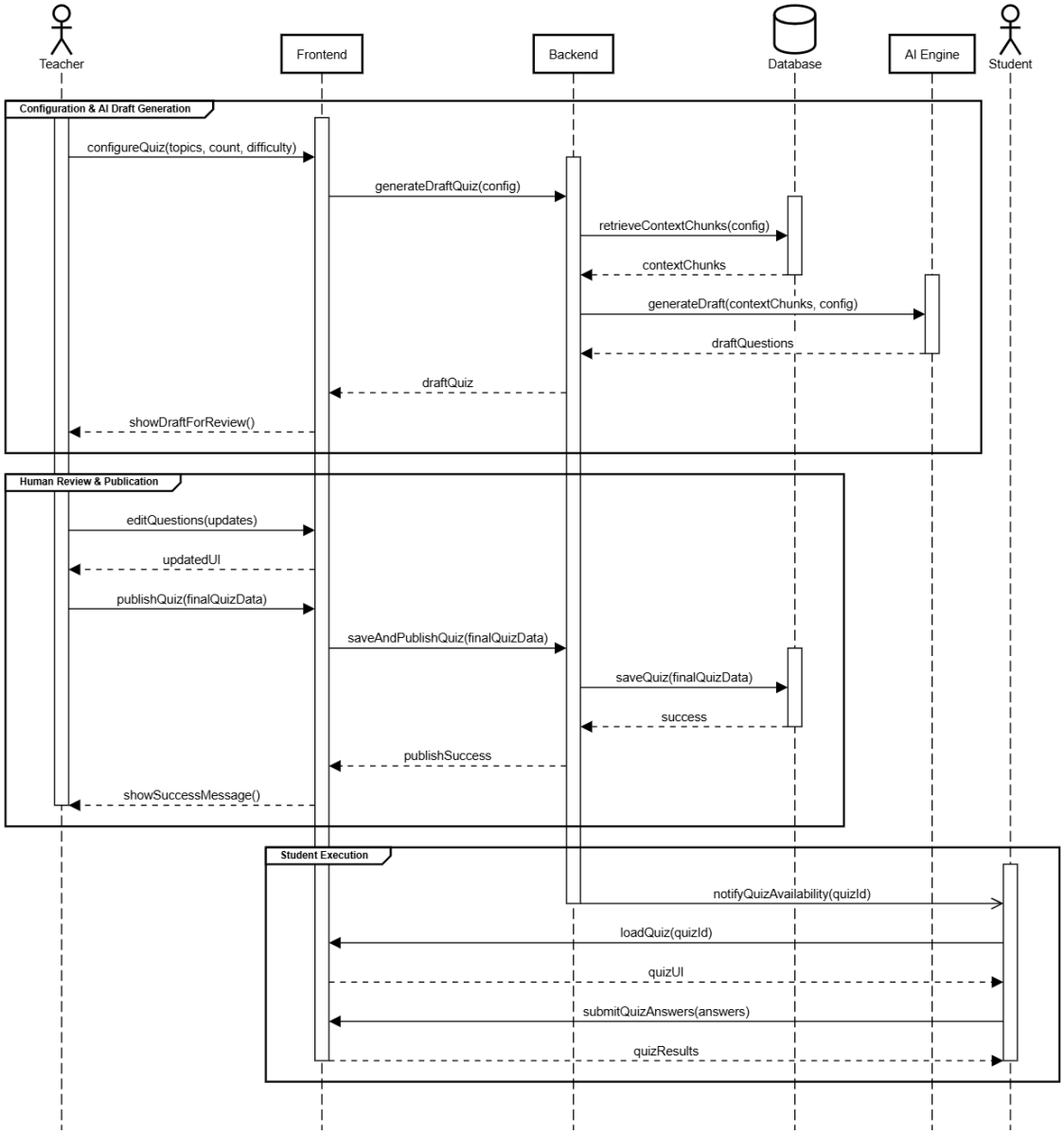


Figure E.3: Sequence Diagram: HITL Quiz Generation and Execution

E.0.4 Native Mock Interview Session

The current interview runtime is a stateful native LiveKit agent rather than the obsolete raster sequence previously included here. The diagram source for those figures is

unavailable and their Qdrant/routed description no longer matches the implemented system; the following textual trace is therefore the authoritative specification.

1. The learner starts or resumes a persisted session and completes onboarding. The API may mint a warm LiveKit token during setup, but dispatches the interviewer only after readiness.

2. The browser joins the LiveKit room over WebRTC. FastAPI supplies ownership checks, room token, and dispatch metadata; it does not relay the audio media stream.

3. The native worker loads durable runtime state and approved question candidates, then starts an `AgentSession` with STT, VAD, LLM, TTS, and persistent chat context.

4. A spoken answer reaches the worker as a final STT transcript. A typed answer reaches `lk.chat`; its receipt is persisted before acknowledgement so retries reuse the idempotency key.

5. The server folds the answer into provisional runtime state, persists it with optimistic versioning, and either advances to a server-selected question or records a bounded follow-up state. The LLM subsequently generates only the conversational wording.

6. The worker publishes TTS audio, transcript, and ordered control snapshots. Deferred `arq` reconciliation improves provisional coverage; terminal evaluation independently judges the persisted transcript.

7. A model completion request and the hard-stop timer converge on one finalization barrier that drains bounded in-flight work, submits once, queues evaluation, and closes the room after closing-audio playout.

The full native implementation and authority boundary are described in Section 6.8.

Appendix F Class Diagram Specification

This appendix expands the three domain packages summarised in Section 5.2. For each package it lists the entities, the relationships that carry design intent, and — where a reader might reasonably expect a different shape — the reason the model does not take it.

F.1 Identity & Access

F.1.1 Roles are assignments, not subclasses

A natural first reading of an education platform is that `Student` and `Teacher` are subclasses of an abstract `User`. The system deliberately does not do this. There is one concrete `User` entity holding identity and authentication state; every capability is derived at request time from role assignments.

Three properties of the domain make inheritance the wrong tool:

- **One person holds several roles at once.** A doctoral teaching assistant is an instructor on one course and an enrolled student on another. Under inheritance that person needs two rows, two identities, and two sets of credentials.
- **A role is scoped, not global.** "Instructor" is never true in the abstract — it is true *of a particular course*. `user_role_assignments` carries a `scope_kind` (`global`, `organization`, `org_unit`, `course`) plus the matching foreign key, so the same role means different things in different places. A subclass has no room to hold that scope.
- **Roles change without the person changing.** Granting and revoking is an insert and a soft delete; under inheritance it would be a change of class, which no ORM models comfortably and which would invalidate every existing reference to the row.

Entity	Role in the model
`User`	Identity and authentication state. One concrete class; no subclasses.
`UserProfile`	Display name, avatar and contact details, separated so that identity can exist before a profile is completed.
`Role`	A named bundle of permissions (`code`, e.g. `course_instructor`).
`Permission`	The atomic capability a guard checks, e.g. `course.update`.
`UserRoleAssignment`	Binds a user to a role *at a scope*. The join carrying `scope_kind` and the scope's foreign key.
`UserPermissionGrant`	A direct grant, for the exceptional case that does not justify a role.
`AuthSession`	An issued session; the unit that MFA and logout act on.
`AuthEvent`	The typed authentication and access-control audit trail (append-only).

Table F.1: Identity & Access entities.

A permission check therefore resolves as: *user* → live assignments whose scope contains the target resource → the union of those roles' permission codes → does that set contain the required code?

F.2 Course & Ingestion

F.2.1 File bytes are not stored in the database

Material content lives in object storage (S3-compatible), never in a database column. A `LearningMaterialVersion` holds `storage_object_id`, a foreign key to a `StorageObject` row that records the bucket and key; the bytes themselves are fetched from, and streamed to, the object store.

The reason is operational rather than stylistic. Lecture material is measured in tens or hundreds of megabytes per file; holding it in Postgres would push every backup, replica and point-in-time restore to the same scale, and every read of a row would carry the risk of dragging the payload with it. Object storage also gives presigned, expiring URLs, so a client downloads directly from the store without the API service proxying the bytes.

Entity	**Role in the model**
`Course`	The container a learner enrols in; owns modules and materials.
`Module` → `Lesson`	The ordered curriculum tree beneath a course.
`LearningMaterial`	A logical document attached to a lesson (“Week 3 slides”), independent of which upload is current.
`LearningMaterialVersion`	One uploaded revision, carrying `storage_object_id` and the processing status of the ingestion pipeline.
`StorageObject`	Bucket and key of the stored bytes. The boundary between the database and the object store.
`DocumentChunk`	A retrievable passage of a version, with its text, metadata, content hash, and a 3072-dimensional embedding.

Table F.2: Course & Ingestion entities.

Versioning is what allows a material to be re-uploaded without invalidating the work derived from it: the old version’s chunks and embeddings remain queryable until the new version finishes processing, so retrieval never falls into a gap mid-ingestion.

F.3 Assessment & Interview

Entity	Role in the model
`Quiz` → `QuizQuestion`	The assessment definition and its items.
`QuizAttempt`	One learner's run at a quiz; the unit grading and integrity events attach to.
`QuizAttemptAnswer`	The per-question response within an attempt.
`InterviewConfig`	The teacher-authored definition of a mock interview.
`InterviewSession`	One learner's run, holding the conversation and its evaluation.
`GapReport`	The aggregated outcome of a session: which learning outcomes the answers did and did not evidence.
`AssessmentIntegrityEvent`	Browser integrity signals recorded during an attempt or session; append-only.

Table F.3: Assessment & Interview entities.

The two assessment modes meet at the `GapReport`: quiz attempts establish whether a learner can recall and apply the material, the interview session probes whether they can explain it, and the report reconciles both against the course's declared learning outcomes.

F.4 Programme & Path Governance

F.4.1 A held pathway is an attempt, not a foreign key

The obvious model gives `ProgramEnrollment` a `career_path_id` column: one student, one pathway. The system deliberately does not do this. An enrolment owns a collection of `ProgramPathAttempt` rows, and the pathway a student is on is read from the live ones.

Three properties of the domain make the single column the wrong tool:

- **A student may hold several pathways at once.** A programme version declares how many may be held concurrently, so the relationship is one-to-many by definition and a scalar column cannot express it.
- **A pathway that ends must remain readable.** Changing or dropping a pathway closes the attempt against an `exit_snapshot` of the progress it had earned; over-

writing a column would destroy exactly the record the decision has to be justified against later.

- **How a pathway was acquired is itself evidence.** `selection_source` distinguishes a pathway the system assigned from the programme default, one the student chose, and one a dean's approval produced. An enrolment that was never the student's own choice reads differently in an audit from one that was, and a foreign key carries no room to say which it is.

Each attempt pins a career-path *version*, not the pathway's current head, so a student continues to be judged against the curriculum they joined even after the pathway is revised beneath them.

F.4.2 Leaving a pathway is a reviewable record

Taking on a pathway and laying one down are deliberately asymmetric. Adding is a direct write within the programme's declared limit; leaving — whether by switching to another pathway or by dropping one outright — creates a `PathChangeRequest` that a Faculty Dean must decide.

Both kinds share one table rather than each having their own, because they share everything that matters: the review queue, the rule of one open request per enrolment, the finite switch allowance an approval spends, and the rejection vocabulary. They differ only in whether a destination exists, so `kind` discriminates them and the target columns are nullable, constrained so that a change must carry a destination and a drop must not. The alternative — a second table — would have duplicated the queue and the allowance, and made "one open request per enrolment" a rule no single constraint could enforce.

Entity	Role in the model
`LearningProgram`	The offering a faculty owns; the unit that is published and enrolled into.
`LearningProgramVersion`	One frozen revision, carrying the pathway switch allowance and the number of pathways a student may hold at once. An edit opens a new draft rather than mutating this.
`LearningProgramVersionPath`	The join binding a version to one career-path version, with its order and the `is_default` flag. A partial unique index permits at most one default per version.
`ProgramEnrollment`	One student on one programme, pinned to the version they joined.
`ProgramPathAttempt`	One pathway held, with its state, `selection_source`, and the `exit_snapshot` written when it ends. Many per enrolment.
`PathChangeRequest`	A request to change or drop a pathway, discriminated by `kind`, with its justification, review state, and the dean's decision.
`CourseCompletionAward`	An earned course result, held independently of any pathway so that it survives one ending and is never recomputed.

Table F.4: Programme & Path Governance entities.

The award is what makes a pathway change cheap for the student: completions are attached to the course, not to the attempt, so work shared between two pathways is credited once and never repeated when the pathway beneath it changes.

Appendix G Implementation Details

This appendix carries the material that Chapter 6 states only in summary: the full lifecycle vocabulary the system enforces, the cross-cutting conventions every feature package observes, the end-to-end path a learner takes through those packages, the two subsystems built entirely to established patterns, and the per-case detail behind the reported test results.

G.1 Entity lifecycle reference

Every lifecycle below is enforced by a database `CHECK` constraint rather than by application code alone, so a state outside the declared set cannot be written by any path — including a migration or a manual correction. The table is the authoritative vocabulary; the transitions between the states, and the reasons for them, are described in the corresponding subsection of Chapter 6.

Table G.1: Entity lifecycle states enforced by database constraints.

Lifecycle	**Carried by**	**States**	**Note**
Publication (generic)	courses, modules, lessons, quizzes, interview configs, career paths, learning programmes, policy versions	`draft`, `published`, `archived`	Published is frozen against edits that would disturb work already done. Where the lifecycle is version-based an edit opens a new draft; where it is not, publication is one-way and a published course or quiz never returns to draft. Archived is closed to new use and never deleted, and for quizzes and learning programmes it is additionally immutable.
Question review	quiz questions, interview questions	`pending`, `approved`, `edited`, `rejected`	AI-drafted items enter pending. Only approved items are delivered to a learner.
Quiz attempt	`quiz_attempts`	`in_progress`, `submitted`, `graded`, `abandoned`, `expired`	Time limit and close date drive the automatic transitions; grading may pause on manual marking.

Continued on next page

Table G.1 – Continued from previous page

Lifecycle	Carried by	States	Note
Interview session	interview_sessions	in_progress, completed, timed_out, abandoned, failed	Evaluated exactly once, under a reservation held for the duration of the run.
Interview onboarding	interview_sessions	identity_check, audio_check, language_check, preparation, readiness, completed	No answer is recorded until onboarding completes, so equipment faults are not scored.
Programme enrolment	program_enrollments	awaiting_path, active, completed, withdrawn, cancelled	Enrolment onto a version with a default path becomes active immediately, never awaiting_path. Completes when no attempt remains active.
Path attempt	program_path_attempts	active, completed, switched_out, cancelled	One row per path held. switched_out follows an approved change; cancelled, an approved drop or a withdrawal. Completed is written when the pinned version's stages all evaluate satisfied.

Continued on next page

Table G.1 – Continued from previous page

Lifecycle	Carried by	States	Note
Path request	`path_change_requests`	`pending`, `in_progress`, `approved`, `rejected`, `cancelled`, `invalidated`	Shared by change and drop. invalidated is reached when the target disappears before the decision.
Course enrolment	`course_enrollments`	`active`, `completed`, `dropped`, `waitlisted`	Reached by operator enrolment, a programme or path entitlement, or an invitation code.
Lesson progress	`lesson_progress`	`not_started`, `in_progress`, `completed`	Feeds both the learner's progress display and the unlock evaluation.
Career enrolment	`student_career_enrollments`	`active`, `completed`, `dropped`	The learner's standing on a career path, independent of any programme that granted it. Completes on the same stage evaluation, which never regresses once a stage is latched.
Notification delivery	`notifications`	`pending`, `sent`, `failed`, `cancelled`	A failure is logged and swallowed; it can never roll back the event that raised it.

Continued on next page

Table G.1 – Continued from previous page

Lifecycle	Carried by	States	Note
Organisation membership	`organization_memberships`	`active`, `invited`, `inactive`, `suspended`, `left`	Membership scope is what dean and manager authority is resolved against.
User account	`users`	`active`, `invited`, `inactive`, `suspended`	Disabling an account revokes every live session.

Where completion is written

Three of the lifecycles above complete as one cascade rather than independently, and the table cannot show that because the transition crosses all three. The evaluation described in Section 5.3.1 is performed once, against the career-path version in question; where it holds, the learner's career enrolment is marked `completed`, every programme path attempt on that path whose *own* pinned version also evaluates complete is closed as `completed`, and a programme enrolment left with no attempt still `active` is marked `completed` with the date recorded. The middle step is qualified deliberately: two programmes may pin different versions of the same path, and a result earned against one version must not close an attempt that promised another.

G.2 Cross-cutting implementation notes

Soft deletion and referential policy

Entities that carry academic meaning are soft-deleted rather than removed, and any parent carrying the soft-delete mixin declares `ON DELETE NO ACTION` on its children. Hard deletion therefore cannot happen by cascade; it happens only where a test or a maintenance job walks the live foreign-key graph explicitly. The effect is that a mistaken delete fails loudly against a constraint rather than silently removing a subtree of results.

Feature independence

Cross-feature calls are confined to each feature's `api/public.py`, and the restriction is machine-checked by import-linter contracts in continuous integration rather than left to review. A feature that reaches into another's internals fails the build, which is what

keeps the boundaries in the previous sections real rather than aspirational.

Concurrency

Decisions that must not race — a path request being approved, an interview being evaluated, a grade of record being recomputed — take a row lock or a reservation before reading the state they will act on, and re-verify their preconditions inside it. Optimistic locking is used instead where a retry is cheap and a conflict is expected to be rare, notably on interview turn state, where a lost race surfaces as an explicit stale-state error rather than an overwrite.

G.3 The learner journey across subsystems

The preceding sections each describe one feature package. Read end to end, they compose into a single path that is worth tracing once, because the handoffs between packages are where the design decisions actually bind.

A learner enters through a programme enrolment, which pins a programme version and places them on its default career path. That path entitles a set of courses; entitlement is recorded separately from enrolment, so a later path change can withdraw access without disturbing a course the learner reached by another route. Within a course, modules hold lessons, quizzes and interviews as sibling items in one ordered walk. A lesson's materials have already passed through ingestion — extracted, chunked, embedded and indexed — and that derived corpus is what the generation pipelines retrieve from when a teacher drafts a quiz or an interview question bank, every draft of which waits for human approval before any learner sees it.

The learner then attempts the quiz. Grading produces a grade of record; the same responses seed spaced-repetition cards, whose ease factors and intervals evolve on a schedule of their own. The unlock rule reads both: prerequisite completion, the quiz result, and the proportion of the lesson's cards standing above the configured ease floor. Passing it opens the next lesson, and where the module requires it, the mock interview probes whether the learner can explain what the quiz established they can recognise. Completion propagates upward — units satisfy a course, courses satisfy a career-path stage, stages satisfy the pathway, and the pathway completes the programme enrolment the learner started from.

Two properties hold along the whole of that path and are easier to see here than in any one section. Nothing generated by a model reaches a learner without a human approving it, and every rule that judges a learner is frozen at the moment it starts applying to them — the integrity policy onto the attempt, the pathway version onto the enrolment, the

programme version onto the student — so that editing the curriculum never retroactively changes what someone already under way is being measured against.

G.4 Unlock evaluation in detail

Whether a lesson opens for a particular learner is decided by three inputs of different kinds, and the distinction between them matters when a result looks wrong.

The first two are per-lesson configuration: which lesson, if any, is a prerequisite, and the standard the learner must reach — the minimum ease factor, the proportion of the lesson's cards that must stand above it, and whether a passing interview is also required. The third is per-learner state: the learner's actual card ease factors, their quiz grade, and their interview verdicts. Configuration is authored once and applies to a cohort; state is personal and changes with every review.

The evaluation is deliberately conservative about missing data. A prerequisite on which the learner has no card history at all is not treated as failed but as unevaluated, and the distinction is surfaced rather than collapsed: a learner who has never attempted the prerequisite needs a different intervention from one who attempted it and fell short. Collapsing the two would present an untouched lesson and a failed one identically, which is the most common way a gating rule becomes untrustworthy to the teacher relying on it.

G.5 Work performed outside the request path

Several operations are too slow, too failure-prone, or too unrelated to the caller to run inside an HTTP request, and are dispatched to the background worker pool instead: material ingestion, quiz and interview generation, notification e-mail, audit writes, and the retention sweeps that expire old records.

They differ in how failure is handled, and the differences are deliberate rather than incidental. Ingestion and generation failures are *reported*: the job's status is recorded against the entity and surfaced to the teacher who started it, because a teacher who uploaded a document is waiting on an outcome and silence would read as success. Audit and notification failures are *swallowed*: they are logged, but they never propagate into the transaction that triggered them, because losing a dean's approval to a failed inbox write would be far worse than losing the notification. Retention sweeps are *retried*, being idempotent by construction — deleting what is already deleted is a no-op — so a transient database error simply means the sweep runs again on its next schedule.

G.6 Deployment topology

The target deployment is a single-page application served to the browser, a FastAPI service handling HTTP lifecycle and control requests, an `arq` worker pool consuming background jobs, and a configuration-controlled native LiveKit interview worker. Browser interview media connects directly to LiveKit Cloud; FastAPI mints room tokens and dispatch metadata but does not relay WebRTC audio. PostgreSQL with `pgvector` remains authoritative for relational, vector, session, and transcript data; Redis provides caching, rate limiting and queueing; Garage provides S3-compatible object storage; and Neo4j holds the course knowledge graph. External model and speech providers sit outside these boundaries.

Two boundaries in that picture carry more weight than the rest. The first is the trust boundary crossed by learner-supplied content: uploaded material and interview utterances originate outside the system and are treated as untrusted wherever they meet a model prompt, which is what the interview input and output guards exist to enforce. The second is the object store path: clients upload and download through presigned URLs issued by the API rather than streaming bytes through it, so the API handles the authorisation decision while the store handles the transfer. That split is why material size is bounded by storage policy rather than by request timeouts. The native agent is enabled in the target deployment; source defaults retain a routed compatibility runtime when the native flag is disabled.

Subsystems built to established patterns

The two subsystems below are documented here rather than in Chapter 6 because neither is novel. OAuth 2.0 sign-in with rotating refresh tokens, TOTP second factors, category-scoped notification preferences and threaded comments are well-established patterns, implemented here as the literature and the relevant standards prescribe. They are recorded in full because the system depends on them and because a few of the decisions taken within them — the invite-only pre-registration gate on OAuth, the keyed MAC over refresh tokens, and the rule that a notification failure never rolls back the academic record that triggered it — are specific to this platform rather than inherited from the pattern.

G.7 Authentication and Identity

The identity subsystem handles user registration, Google OAuth sign-in, JWT session management, and multi-factor authentication (MFA). It is implemented in `abridgeai/features/identity/` and exposes four router groups: `/auth`, `/me`, `/users` (admin), and `/mfa`.

Google OAuth flow

The platform uses Google OAuth 2.0 as its sole external identity provider. The sign-in flow is implemented in `features/identity/services/login.py` and proceeds as follows:

1. The frontend redirects the user to Google's authorisation endpoint with the configured `client_id` and `redirect_uri`.
2. Google redirects back with an authorisation code, which the frontend forwards to `POST /auth/google/callback`.
3. The backend exchanges the code for a Google profile via `infrastructure/google_oauth.py`, which calls Google's `tokeninfo` endpoint.
4. A pre-registration gate checks that the email already exists in the `users` table with `status = 'active'`. New OAuth profiles from unregistered emails are rejected with HTTP 403, keeping the platform invite-only.
5. On success, an `AuthIdentity` row linking the user to the Google provider subject is created (or reused if it already exists), and a new `AuthSession` row is written with a hashed refresh token.
6. The response returns a short-lived JWT access token (configurable TTL, default 15 minutes) and a long-lived refresh token.

JWT and session management

Access tokens are signed with HS256 using a secret loaded from the `JWT_SECRET` environment variable. The `core/security` module provides `create_access_token()`, `hash_secret()`, and `generate_token()` (32-byte URL-safe random). Refresh tokens are stored in `auth_sessions` as keyed HMAC-SHA256 digests — the plaintext is never persisted — and compared in constant time. A keyed MAC rather than a password hash is the right primitive here: the input is a 32-byte random token with no guessable structure, so the work factor that protects a human-chosen password buys nothing, while the

server-held key means a leaked database alone does not yield usable refresh tokens. Token rotation is enforced: each `POST /auth/refresh` call invalidates the old session row and issues a new one.

Multi-factor authentication

TOTP-based MFA is implemented in `features/identity/services/mfa.py`. Enrollment generates a TOTP secret, returns a provisioning URI for authenticator apps, and stores the encrypted secret only after the user verifies a valid code. Subsequent logins that have MFA enabled require a second `POST /mfa/verify` call before the access token is issued. The MFA gate is enforced at the router level via a FastAPI dependency that checks the `mfa_verified` claim in the JWT.

Role-based access

Every authenticated request resolves the caller's roles and permissions through the access control subsystem (Section 6.3). The identity layer attaches the resolved `UserContext` — containing user ID, organisation memberships, and permission set — to the request state so downstream route handlers can perform permission checks without additional database queries.

The session lifecycle

There is one way in and several ways out. Google OAuth is the only sign-in route — the platform holds no passwords and has no local credential to verify — so every session begins with the pre-registration gate above, and an unrecognised Google profile is refused before any session row exists. Where the account has MFA enrolled, the callback yields a token that carries no `mfa_verified` claim and therefore opens no protected route; only the subsequent verification exchanges it for a usable access token. The second factor is thus a gate on issue, not a check applied afterwards.

A live session then persists until something ends it. Ordinary expiry is the common case, the access token being short-lived and each refresh rotating the session row so a replayed refresh token finds its row already invalidated. Explicit logout terminates the presented session. Disabling an account terminates all of them at once, which is why the reverse index from user to session IDs exists: revocation must not depend on scanning the session keyspace, and an administrator disabling an account expects it to take effect on the next request rather than at the next token expiry.

G.8 Notifications and Discussions

Notifications

Dispatch is implemented in `abridgeai/features/notifications/`. A notification carries a category drawn from a fixed vocabulary — course updates and announcements, material processing, quiz and interview generation outcomes, path-change review, lesson discussion, and system messages — and learners hold preferences per category rather than per event, so a single toggle governs a whole class of message instead of accumulating switches.

Message text is built in `messages.py`, which holds one function per notification moment and renders English or Vietnamese from the recipient's own locale. Keeping the copy in typed builders rather than templates is what makes a new locale a compile-time obligation instead of a silent fallback to English.

Delivery tracks `pending`, `sent`, `failed`, and `cancelled`, with e-mail dispatched by a background worker so the request that triggered it never waits. Crucially, a notification failure is swallowed and logged rather than propagated: the write that caused it — a dean's approval, a completed import — is an academic record change, and losing it because an inbox row could not be written would be far worse than a missing notification.

Discussions

`abridgeai/features/discussions/` implements threaded discussion on lessons and courses: topics carry comments, comments carry direct replies, and both are soft-deleted so a removed post leaves the thread's shape intact rather than orphaning the replies beneath it. Authorship and edit windows are enforced in the service layer, and a new comment raises a `course_discussion` notification through the same category machinery as everything else.

The dispatch path

A domain event reaches a recipient through a fixed sequence. The event resolves to a category; the category is tested against that recipient's stored preference for the channel in question, and a pair with no stored preference falls back to a declared default rather than to silence; the recipient's locale selects the builder; and the builder produces the text. From there the two channels diverge. The in-app row is written inside the triggering transaction, so an inbox entry and the change it reports commit together or not at all. E-mail is handed to a background worker instead, because an outbound SMTP call is slow and may fail for reasons that have nothing to do with the academic record being written.

That divergence is where the swallowed failure sits. The boundary is drawn immediately around dispatch: anything that goes wrong beyond it — a worker error, a rejected address, an unreachable relay — is logged and dropped, and can move the delivery row to `failed`, but can never roll back the dean's approval or the completed import that triggered it. The states `pending`, `sent`, `failed` and `cancelled` therefore describe the fate of the message alone, never the fate of the transaction behind it.

G.9 Test execution detail

Section 6.15 reports the executed totals and the coverage achieved per package. The tables below list the individual test cases behind those totals, grouped by the module they exercise.

G.9.1 Authentication and Access Control

Table G.2 details the test cases covering the authentication and access control module.

Table G.2: Test Results: Authentication and Access Control

TC-ID	Description	Test File	Result
TC-1.1.1	Google SSO login callback	`test_oauth_flow.py`	Pass
TC-1.1.2	Idempotent logout	`test_oauth_flow.py`	Pass
TC-1.2.1	Session creation and refresh-token rotation	`test_identity_full.py`	Pass
TC-1.2.2	Session revocation on account disable	`test_identity_queries.py`	Pass
TC-1.3.1	TOTP verification and recovery-code consumption	`test_mfa_router.py`	Pass
TC-1.3.2	Recovery-code reuse prevention	`test_mfa_gate.py`	Pass
TC-1.5.1	Recursive permission resolution	`test_access_control_full.py`	Pass
TC-1.5.2	Endpoint enforcement by role	`test_access_control_membership_resolution.py`	Pass

G.9.2 Course and Material Management

Table G.3 details the test cases for course and material management features implemented in this iteration.

Table G.3: Test Results: Course and Material Management

TC-ID	Description	Test File	Result
TC-3.1.1	Course lifecycle (draft → publish)	`test_courses_e2e.py`	Pass
TC-3.1.2	Course archive and restore	`test_courses_e2e.py`	Pass
TC-3.3.1	PDF material upload and ingestion trigger	`test_materials_e2e.py`	Pass
TC-3.3.2	Ingestion pipeline processing	`test_materials_worker.py`	Pass
TC-3.7.1	Student material access (presigned URL)	`test_materials_learner_router.py`	Pass

G.9.3 Quiz Generation and Taking

Table G.4 details the test cases for the AI quiz generation and student quiz-taking features.

Table G.4: Test Results: Quiz Generation and Taking

TC-ID	Description	Test File	Result
TC-4.1.1	AI quiz generation from PDF content	`test_quiz_full_pipeline.py`	Pass
TC-4.1.2	Teacher review (approve/reject/edit)	`test_quiz_authoring_service.py`	Pass
TC-4.3.1	Student quiz session submission	`test_quiz_taking_service.py`	Pass
TC-4.3.2	Per-question response and feedback	`test_quizzes_e2e.py`	Pass